ULM on ICE

Building a Computer on an FPGA, Step by Step

Michael C. Lehn

Contents

Preface 4

I Getting Started 5

1 The Big Picture 6
1.1 What an FPGA Is . . . 6
1.2 A Lookup Table Is a Multiplexer . . . 6
1.3 Flip-Flops, and Why the Clock Matters . . . 7
1.4 The Board . . . 8
1.5 From a Description to a Configured Chip . . . 8
1.6 Solutions . . . 10

2 Step 01: Logic Gates 11
2.1 The Circuit . . . 11
2.2 The Description in SystemVerilog . . . 11
2.3 From test.sv to the Chip . . . 13
2.4 Simulation . . . 15
2.5 Pictures of a Design . . . 16
2.6 Solutions . . . 17

3 Step 02: Flip-Flops 18
3.1 The Circuit . . . 18
3.2 The Description in SystemVerilog . . . 18
3.3 What yosys Makes of It . . . 20
3.4 How Fast May the Clock Be? . . . 20
3.5 Where Are the Pins? . . . 21
3.6 Simulation . . . 21
3.7 Solutions . . . 23

II From Counters to a Computer 24

4 Step 03: A Counter on the 7-Segment Display 25
4.1 The Circuit . . . 25
4.2 Vectors and Constants . . . 25
4.3 Counters . . . 28
4.4 Using a Module: dev_hex . . . 29
4.5 Driving the 7-Segment Display . . . 29
4.6 What yosys Makes of It . . . 33
4.7 What nextpnr Makes of It . . . 34
4.8 Simulation . . . 35
4.9 What Is Next . . . 35
4.10 Solutions . . . 36

5 Step 04: Storing the DIP Switch 37
5.1 The Circuit . . . 37

5.2 The Description in SystemVerilog . . . 38
5.3 What yosys Makes of It . . . 39
5.4 The Button . . . 39
5.5 What nextpnr Makes of It . . . 40
5.6 Simulation . . . 41
5.7 What Is Next . . . 42
5.8 Solutions . . . 43

6 Step 05: A Mini-ALU 44
6.1 The Mini-ALU from the Course . . . 44
6.2 The Description in SystemVerilog . . . 46
6.3 The Button . . . 51
6.4 What yosys Makes of It . . . 52
6.5 Simulation . . . 53
6.6 What Is Still Missing . . . 54
6.7 Solutions . . . 55

7 Step 06: The Serial Line 56
7.1 Sending Bits One After the Other . . . 56
7.2 The Top Module . . . 57
7.3 Parameters . . . 60
7.4 Baud Ticks from 12 MHz . . . 61
7.5 The Transmitter . . . 63
7.6 The Receiver . . . 64
7.7 A Latch from an Incomplete case . . . 66
7.8 What yosys and nextpnr Make of It . . . 67
7.9 Talking to the Board . . . 67
7.10 Simulation . . . 68
7.11 What Is Next . . . 70
7.12 Solutions . . . 71

8 Step 07: An I/O Device with Buffers 72
8.1 What the Design Does . . . 72
8.2 Interfaces: Named Bundles of Wires . . . 73
8.3 Packages and Include Files . . . 75
8.4 A FIFO . . . 77
8.5 Memory: Block RAM . . . 80
8.6 The I/O Device . . . 82
8.7 Simulation . . . 88
8.8 Resources . . . 90
8.9 What Is Next . . . 90
8.10 Solutions . . . 91

9 Step 08: A Program Loader 92
9.1 What the Design Does . . . 92
9.2 The Single-Port RAMs of the iCE40UP5K . . . 93
9.3 for Loops Are Replicated Hardware . . . 95
9.4 A 64-bit Memory from Four 16-bit Ones . . . 96
9.5 What the Shifters Cost . . . 102
9.6 Two Users, One Memory . . . 103
9.7 The Loader . . . 106

9.8 A Declaration Is Not a Connection . . . 110
9.9 What Is Next . . . 111
9.10 Solutions . . . 112

10 Step 09: The ULM on ICE **113**
10.1 The Instruction Set . . . 113
10.2 The Architecture . . . 114
10.3 The Register File and the ALU . . . 116
10.4 The Decoder . . . 118
10.5 The Control Unit . . . 121
10.6 One Instruction, Cycle by Cycle . . . 127
10.7 Running a Program . . . 128
10.8 Resources and Speed . . . 131
10.9 What Could Be Improved . . . 133
10.10Solutions . . . 134

A The make Targets **135**
A.1 Overview . . . 135
A.2 Simulation . . . 135
A.3 Synthesis and Programming . . . 136
A.4 Pictures of a Design . . . 137
A.5 make report . . . 137
A.6 make floorplan . . . 138
A.7 make clean . . . 138

B Glossary **140**

Preface

The ULM, the *Ulm Lecture Machine*, is the computer we use in our lectures to explain what a machine does with a program. It has sixteen 64-bit registers, a memory, instructions of 32 bits, and a few instructions to read and write characters. Until now it existed as a simulator. In these notes we build it as real hardware: on an FPGA, a chip whose circuits are not fixed when it leaves the factory but are configured later.

We do not build it in one go. The repository `ulm-on-ice` consists of nine directories, `01_logic_gates` to `09_ulm_on_ice`, and each of them is a small, complete project that you can simulate, put on the FPGA and try out. The first one connects two buttons to two LEDs through a few logic gates, the last one is the ULM, running programs that we type in through a terminal. Each step adds one new idea, and each chapter of these notes is one step.

On the way you should learn three things:

- what an FPGA is, and what it can and cannot do;
- how we describe a circuit in SystemVerilog, and how the tools turn this description into a configuration of the FPGA;
- how a computer is built from parts that you know: gates, flip-flops, multiplexers, adders, registers and memory.

We assume that you know logic gates and flip-flops, and that you have built small circuits with a tool like CircuitVerse: an adder, a multiplexer, a register. This is all the hardware background we need. The rest is introduced when a step needs it, and only as much as that step needs.

Most of all, we treat SystemVerilog as what it is: *a textual representation of circuits*. It looks like a programming language, and this is the source of most misunderstandings. A line of SystemVerilog does not do something at some point in time. It says how wires are connected. Whenever a construct is new, we first ask which circuit it describes.

The notes are a tutorial, so they are meant to be worked through with the repository and the board at hand. Every listing of SystemVerilog is taken directly from the repository, and all the output shown was produced by the tools on a Mac (yosys 0.70, nextpnr 0.11.1, Verilator 5.052, sv2v 0.0.13). Your output may differ in details.

The board is an iCEBreaker, with the small button-and-LED module that comes with it and the 7-segment display module from the same manufacturer. The idea for the material grew out of the course *HPC0*, where we build a Mini-ALU in CircuitVerse, first look at its description in SystemVerilog, and then put it on exactly this board.

Part I

Getting Started

Chapter 1

The Big Picture

Before we describe a single circuit, we need a mental model of the chip that will contain it. This chapter answers three questions: what is inside an FPGA, what is on our board, and which tools bring a description in SystemVerilog onto the chip.

1.1 What an FPGA Is

A processor, say the CPU of your laptop, is a fixed circuit. It was designed once, and all chips of this type contain the same gates connected by the same wires. What changes is the program it runs.

An FPGA, a *field programmable gate array*, is different. It is a chip that consists of a large number of small, identical building blocks and of wires between them, and both are *configurable*: which Boolean function a block computes, and which blocks are connected to which. The configuration is loaded into the chip when it is switched on. So the circuit itself is what we change, not a program running on it. If we configure an FPGA as a processor, then this processor can in turn run programs, and that is exactly what we do in the last step.

The building blocks of our FPGA, a Lattice iCE40UP5K, come in a few kinds:

Logic cells. The workhorse. A logic cell consists of a lookup table with four inputs, a flip-flop, and a little extra logic for adding numbers. The iCE40UP5K has 5280 of them.

Routing. Wires between the logic cells, with switches that decide which wires are connected. Most of the area of an FPGA is routing, not logic.

I/O pins. The connections to the outside world, here buttons, LEDs, the display and the USB chip on the board.

Memory blocks. Ready-made memory, because building memory from flip-flops would use up the logic cells quickly. We meet them in Chapter 8.

The lookup table is the part that needs explaining, and it is simpler than its name suggests.

1.2 A Lookup Table Is a Multiplexer

You know the multiplexer from CircuitVerse: it has data inputs, control inputs, and one output, and the control bits select which data input appears at the output. A multiplexer with four control bits selects one of $2^4 = 16$ data inputs.

Now take such a 16-to-1 multiplexer and do not connect its data inputs to signals. Instead, fix each of them to a constant 0 or 1 when the chip is configured. These 16 constants are the *configuration bits* of the lookup table, and the four control inputs are its actual inputs (Figure 1.1). This is a *lookup table*, or *LUT* for short.

What does such a LUT compute? The four input bits, read as a binary number $i = 8x_3 + 4x_2 + 2x_1 + x_0 \in \{0, \ldots, 15\}$, select the configuration bit t_i. So the LUT computes the Boolean function

$$f\colon \{0,1\}^4 \to \{0,1\}, \qquad f(x_3, x_2, x_1, x_0) = t_{8x_3+4x_2+2x_1+x_0},$$

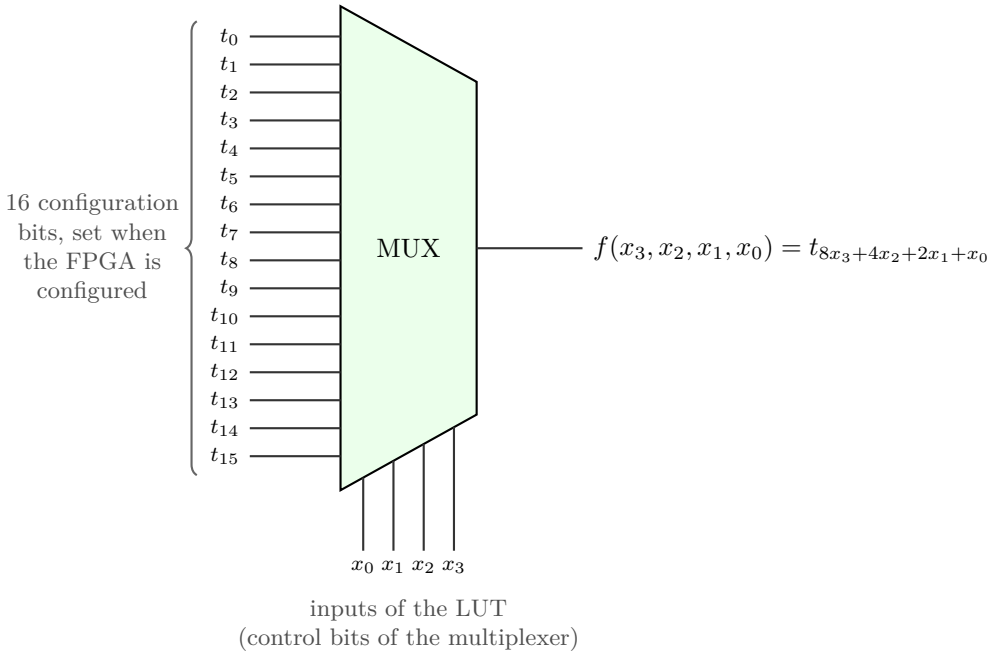


Figure 1.1: A lookup table with four inputs: a 16-to-1 multiplexer whose data inputs are constants. The inputs of the LUT select one of them.

and the 16 configuration bits are nothing but the last column of its truth table. Every Boolean function of four variables has a truth table with 16 rows, so a LUT can compute *every* such function: there are $2^{16} = 65536$ of them, one for each way to choose the configuration bits.

This is the whole trick of an FPGA. There are no AND or OR gates on the chip waiting to be connected. A circuit that you draw with gates is translated into Boolean functions of at most four inputs each, and every one of these functions becomes a LUT with the right 16 bits. A function of more than four inputs is split into several LUTs, the output of one feeding an input of the next.

Exercise 1.1. Which configuration bits $t_0, \ldots, t_{15}$ make a LUT compute the function $f(x_3, x_2, x_1, x_0) = x_3 \wedge x_2$? Which ones make it compute $x_0 \dot{\vee} x_1 \dot{\vee} x_2 \dot{\vee} x_3$?

1.3 Flip-Flops, and Why the Clock Matters

A circuit of LUTs alone is combinational: its outputs depend only on its current inputs. To store something we need flip-flops, and every logic cell has one right behind its LUT. The output of the LUT can go out of the logic cell directly, or into the flip-flop, which takes it over at the rising edge of a clock.

On our board a crystal oscillator delivers a clock of 12 MHz, that is $12 \cdot 10^6$ rising edges per second, one every 83.3 ns. All flip-flops in our designs use this one clock (or are meant to, see Section 3.5). Between two edges, the signals have to travel from the flip-flops through LUTs and wires to the next flip-flops and settle there. This takes time, and so the tools tell us after placing the design how fast the clock could be. For our larger designs this will be a real limitation.

1.4 The Board

The iCEBreaker (Figure 1.2) carries the FPGA, the 12 MHz oscillator, a flash memory that stores the configuration, and a USB chip. The USB chip has two jobs: it writes the configuration into the flash, and it provides a serial line between your computer and the FPGA, which we use from Chapter 7 on.

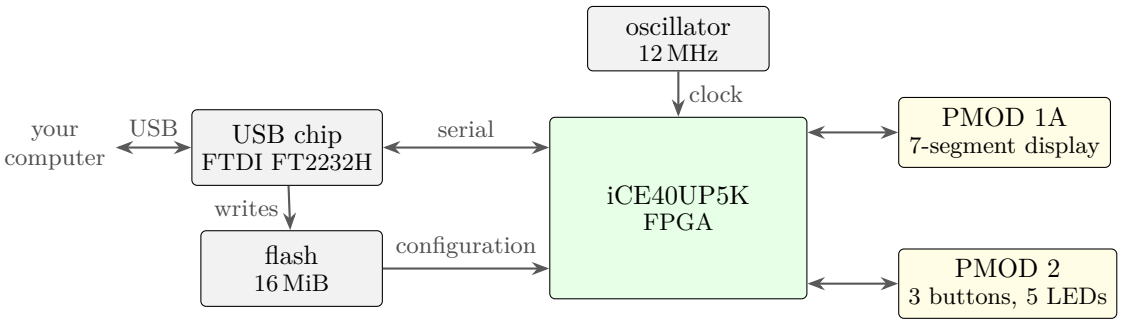


Figure 1.2: What is on the board, and how the parts are connected.

The FPGA forgets its configuration when the power goes off. At power-up it reads the configuration from the flash, and so the flash is what we program.

On the right edge of the board are connectors in the PMOD format, a simple standard for small extension modules. On one we plug the module with three buttons and five LEDs, on another the 7-segment display with two digits.

Which wire of the FPGA leads to which button is not something the FPGA knows. The file `icebreaker.pcf` at the top of the repository gives the pins of the chip names that we can use in our designs. A few lines of it:

```
# 12 MHz clock
set_io -nowarn CLK        35
...
# LEDs and Buttons (PMOD 2)
set_io -nowarn LED1       26
set_io -nowarn LED2       27
...
set_io -nowarn BTN1       20
set_io -nowarn BTN2       19
```

So pin 35 of the chip is called `CLK`, and pin 20 `BTN1`. When a design has an input called `BTN1`, it is connected to the first button.

1.5 From a Description to a Configured Chip

Between the SystemVerilog file and the configured chip there are four tools, and it pays to know what each of them does. Each one reads a file and writes the next (Figure 1.3).

sv2v translates SystemVerilog into plain Verilog. Verilog is the older language from which SystemVerilog grew; yosys understands only part of SystemVerilog, but all of Verilog. For our first designs the translation hardly changes anything; later, SystemVerilog features such as interfaces and packages are translated into Verilog that means the same.

yosys *synthesizes* the design: it turns the description into a *netlist*, a list of the building blocks of our FPGA (LUTs with their 16 bits, flip-flops, memory blocks) and of the wires between them. It also simplifies the logic as far as it can. Nothing is placed on the chip yet.

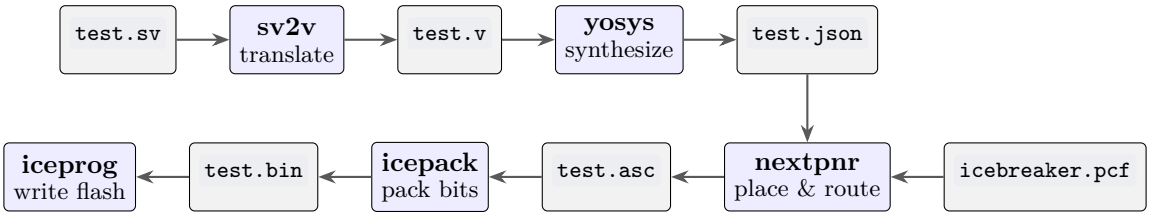


Figure 1.3: The tools between a description in SystemVerilog and the configuration in the flash of the board. `make synth` runs everything up to `test.bin`, `make prog` also `iceprog`.

nextpnr *places* every building block of the netlist on one of the physical blocks of the chip and *routes* the wires, that is, it chooses the switches of the routing so that the right blocks are connected. Here the pin names from `icebreaker.pcf` come in. Then it computes how long the signals take, and how fast the clock may be.

icepack converts the result into the bit pattern that the FPGA loads at power-up, the *bitstream*.

iceprog writes the bitstream through USB into the flash and restarts the FPGA, which then loads it.

We do not call these tools by hand. Every step directory contains the same `Makefile` (it is a link to the one in `01_logic_gates`), and its targets do it for us. The design is always in `test.sv`, with the top module called `test`.

Next to this path to the chip there is a second one that does not need the board at all: *simulation*. The tool Verilator translates the design into a C++ class, and a small C++ program, the *testbench*, sets the inputs of this class step by step and records how all signals change. We look at these recordings in a waveform viewer. Chapter 2 shows both paths for our first design. Appendix A lists all `make` targets.

1.6 Solutions

Exercise 1.1. For $x_3 \wedge x_2$, the output is 1 exactly when $8x_3 + 4x_2 + 2x_1 + x_0 \geq 12$, that is, for the indices 12, 13, 14, 15. So $t_{12} = t_{13} = t_{14} = t_{15} = 1$ and all other bits are 0. Written from t_{15} down to t_0, as yosys does it (Section 2.3): `1111000000000000`.

For $x_0 \dot{\vee} x_1 \dot{\vee} x_2 \dot{\vee} x_3$, the output is 1 exactly when the index i has an odd number of ones in binary: $i = 1, 2, 4, 7, 8, 11, 13, 14$. From t_{15} down to t_0: `0110100110010110`.

Chapter 2

Step 01: Logic Gates

Our first design has no state at all: two buttons, two LEDs, and a few gates in between. This is little hardware, but enough to meet the basic parts of SystemVerilog, to follow the design through all the tools of Figure 1.3, and to simulate it. Everything is in the directory `01_logic_gates`.

2.1 The Circuit

Call the two buttons a and b (pressed means 1) and the two LEDs out_1 and out_2 (1 means on). The circuit computes

$$out_1 = (a \wedge b) \vee a, \qquad out_2 = a \vee b.$$

You would draw it in CircuitVerse as in Figure 2.1: an AND gate and two OR gates. In fact, this figure was not drawn by hand: it was generated from the SystemVerilog description by `make png`, which we come back to in Section 2.5.

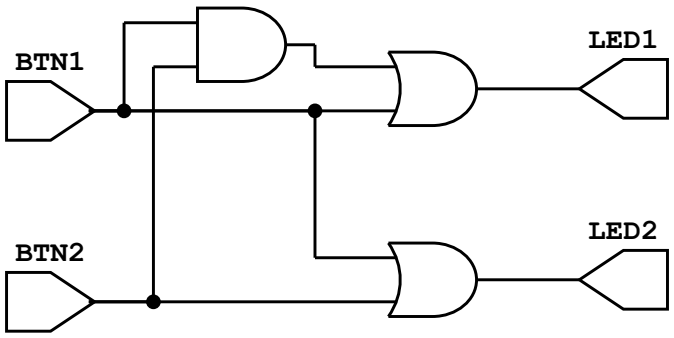


Figure 2.1: The circuit of step 01, as drawn by `make svg` from `test.sv`. The pins on the left and right are the ports of the module.

2.2 The Description in SystemVerilog

This is the complete description:

01_logic_gates/test.sv

```
module test (
    input logic BTN1,
    input logic BTN2,
    output logic LED1,
    output logic LED2
);

    logic a, b;
    assign a = BTN1;
    assign b = BTN2;
```

```
    logic out1, out2;
    assign LED1 = out1;
    assign LED2 = out2;

    always_comb begin
        out1 = a & b | a;
        out2 = a | b;
    end

endmodule
```

Read it as a description of the circuit in Figure 2.1, piece by piece.

A module is a box with pins. Everything between `module` and `endmodule` describes one circuit, here called `test`. The list in parentheses are its *ports*: the wires that leave the box, each one an `input` or an `output`. In CircuitVerse terms, a module is what you get when you turn a circuit into a subcircuit, and the ports are its inputs and outputs.

Because `test` is the outermost module, the *top module*, its ports are the pins of the chip. Their names are not arbitrary: `BTN1`, `BTN2`, `LED1` and `LED2` are names from `icebreaker.pcf` (Section 1.4), and that is how nextpnr knows which pin of the FPGA each port is.

`logic` **declares a signal.** The line `logic a, b;` declares two wires inside the box, each carrying one bit. The type `logic` says only that the signal carries a logic value; whether it ends up as a plain wire or as the output of a flip-flop follows from how we use it, as we see in the next chapter.

`assign` **connects.** `assign a = BTN1;` says that the wire `a` is permanently connected to the input `BTN1`. It is not an assignment in the sense of a programming language, which happens once when the line is executed. It is a statement about the circuit that holds all the time: whatever value `BTN1` has, `a` has it too. The four `assign` statements only give the pins more convenient names. This separates the logic of our design (`a`, `b`, `out1`, `out2`) from the question which pin is which, and we can change one without the other.

`always_comb` **describes combinational logic.** The block

```
always_comb begin
    out1 = a & b | a;
    out2 = a | b;
end
```

says: `out1` and `out2` are computed from the signals on the right-hand sides, by a circuit without memory. Whenever an input changes, the outputs follow. The operators are the Boolean operations you know:

SystemVerilog	logic	name	
`&`	$\wedge$	AND	
`|`	$\vee$	OR	
`^`	$\dot{\vee}$	XOR	(exclusive or)
`~`	$\neg$	NOT	

As in arithmetic, where $\cdot$ binds stronger than $+$, `&` binds stronger than `|`, so `a & b | a` means `(a & b) | a`.

Here the two statements in the block are independent, so their order does not matter at all. We could also have written `assign out1 = a & b | a;`, which describes the same circuit. `always_comb` becomes useful when the logic needs case distinctions, which we see in later steps.

Pitfall. SystemVerilog looks like a programming language, and the `=` in `always_comb` looks like an assignment that is executed at some point. It is not. The whole module describes a circuit that exists all the time, and all of its parts work in parallel. There is no "first line" that runs before the second, just as there is no first gate in Figure 2.1.

Exercise 2.1. Simplify $(a \wedge b) \vee a$. What does that mean for `LED1`?

2.3 From test.sv to the Chip

In the directory `01_logic_gates`, the command `make synth` runs the tools of Figure 1.3 up to the bitstream. It prints the commands it runs:

```
$ make synth
yosys -ql test.log -p 'synth_ice40 -abc9 -dffe_min_ce_use 4 -device u -dsp -top test -json test.json' test.v
nextpnr-ice40 --pcf-allow-unconstrained --up5k --package sg48 --json test.json --pcf ../icebreaker.pcf --asc test.asc
icepack test.asc test.bin
```

Before that, `make` has called `sv2v` to produce `test.v`:

```
$ make test.v
sv2v -v --define=SYNTHESIS --write=test.v test.sv
```

The option `--define=SYNTHESIS` matters only from step 08 on: there, parts of a file are meant for simulation only, and `SYNTHESIS` switches them off.

What yosys makes of it

yosys writes its report to `test.log`. The interesting part is at the end, a list of the building blocks of the netlist:

```
2.60. Printing statistics.

=== test ===

        +----------Local Count, excluding submodules.
        |
        6 wires
        6 wire bits
        6 public wires
        6 public wire bits
        4 ports
        4 port bits
        1 submodules
        1   SB_LUT4
```

Four ports, six wires, and exactly *one* building block, an `SB_LUT4`: one lookup table with four inputs. Three gates became a single LUT, and `LED1` needs no logic at all. If you solved Exercise 2.1, you know why: $(a \wedge b) \vee a = a$, so `LED1` is simply connected to `BTN1`, and yosys found that out by itself. `LED2` needs a LUT for $a \vee b$.

Which LUT? The netlist `test.json` contains, for every building block, its type and its parameters. For our LUT:

```
{
  "LED2_SB_LUT4_O": {
    "type": "SB_LUT4",
    "parameters": {
      "LUT_INIT": "1111111111110000"
    }
  }
}
```

`LUT_INIT` are the 16 configuration bits of Section 1.2, written from t_{15} on the left down to t_0 on the right. The netlist also says how the LUT is connected: `BTN1` drives input x_3, `BTN2` drives x_2, and the inputs x_1 and x_0 are fixed to 0. So the LUT selects the bit

$$t_{8\cdot\texttt{BTN1}+4\cdot\texttt{BTN2}},$$

which is $t_0 = 0$ if no button is pressed, and $t_4 = t_8 = t_{12} = 1$ if at least one is. That is $\texttt{BTN1} \vee \texttt{BTN2}$, exactly as it should be. The other twelve bits are never selected; yosys sets them to 1, but any value would do.

What nextpnr makes of it

nextpnr places the LUT on one of the 5280 logic cells, connects the four pins, and reports what it used:

```
Info: Device utilisation:
Info:              ICESTORM_LC:       3/   5280     0%
Info:                     SB_IO:       4/     39    10%
```

`ICESTORM_LC` are the logic cells, `SB_IO` the I/O pins. Three logic cells for one LUT? The other two produce the constants 0 and 1, which the unused LUT inputs are connected to. (Our report from Appendix A, `make report`, shows the same numbers.)

nextpnr also computes how long a signal takes from an input pin to an output pin. For the slowest path, from `BTN2` to `LED2`:

```
Info: Critical path report for cross-domain path '<async>' -> '<async>':
Info:       type curr  total name
Info:     source  0.00  0.00 Source BTN2$sb_io.D_IN_0
Info:    routing  5.54  5.54 Net BTN2$SB_IO_IN (21,0) -> (18,18)
Info:                         Sink LED2_SB_LUT4_O_LC.I2
Info:                         Defined in:
Info:                              test.v:18.7-18.8
Info:      logic  1.21  6.75 Source LED2_SB_LUT4_O_LC.O
Info:    routing  4.22  10.97 Net LED2$SB_IO_OUT (18,18) -> (18,31)
Info:                         Sink LED2$sb_io.D_OUT_0
Info:                         Defined in:
Info:                              test.v:15.14-15.18
Info: 1.21 ns logic, 9.77 ns routing
```

The numbers in parentheses are positions on the chip: the pin of `BTN2` at the lower edge, the LUT in the middle, the pin of `LED2` at the upper edge. The LUT itself takes 1.21 ns, and the wires to and from it take 9.77 ns. This is typical for FPGAs: the routing is usually slower than the logic. For us it does not matter here, since nobody presses buttons within 11 ns. Once a clock is involved, it will.

Programming the board

`make prog` does everything `make synth` does and then calls `iceprog test.bin`, which writes the bitstream into the flash of the board and restarts the FPGA. A second later, the LEDs follow the buttons: `LED1` is on while the first button is pressed, `LED2` while one of the two is pressed.

Exercise 2.2. Replace `out2 = a | b;` by `out2 = a ^ b;`. Predict the value of `LUT_INIT` for the LUT of `LED2`, then check it with `make test.json` and a look into `test.json`.

2.4 Simulation

Programming the board is the final test, but it is a poor tool for finding mistakes: you see only what reaches a pin. In a *simulation*, a program computes how all signals of the design change over time, and we can look at every one of them.

We use Verilator. It translates the design into a C++ class `Vtest`, whose member variables are the ports of the module. A small C++ program, the *testbench* `tb_test.cpp`, creates such an object, sets its inputs and calls `eval()`, which computes the outputs. Here is its main loop:

01_logic_gates/tb_test.cpp

```
    std::uint64_t sim_time = 0;

    while (sim_time < MAX_SIM_TIME) {
        switch (sim_time) {
            case 5:
                dut->BTN1 = 1;
                dut->BTN2 = 0;
                break;
            case 8:
                dut->BTN1 = 1;
                dut->BTN2 = 1;
                break;
            case 11:
                dut->BTN1 = 0;
                dut->BTN2 = 1;
                break;
            default:
                dut->BTN1 = 0;
                dut->BTN2 = 0;
        }
        dut->eval();

        m_trace->dump(sim_time);
        sim_time++;
    }
```

The variable `sim_time` counts steps of the simulation; it has no unit, it is just an index. At time 5 the testbench presses the first button, at time 8 both, at time 11 the second, and at all other times none. After each `eval()`, `m_trace->dump(sim_time)` records the values of all signals in the file `waveform.vcd`, a *value change dump*.

`make sim` (or just `make`) calls Verilator, compiles the generated C++ code together with the testbench, and runs the result:

```
$ make sim

### VERILATING ###
verilator -Wall --trace --x-assign unique --x-initial unique \
        -Wno-fatal \
        -cc test.sv --top-module test \
        --exe tb_test.cpp
...

### BUILDING SIM ###
make -C obj_dir -f Vtest.mk Vtest
...

### SIMULATING ###
```

The simulation itself prints nothing. Its result is `waveform.vcd`, and `make surfer` (or `make waves` for GTKWave) shows it in a waveform viewer. Figure 2.2 shows what you see.

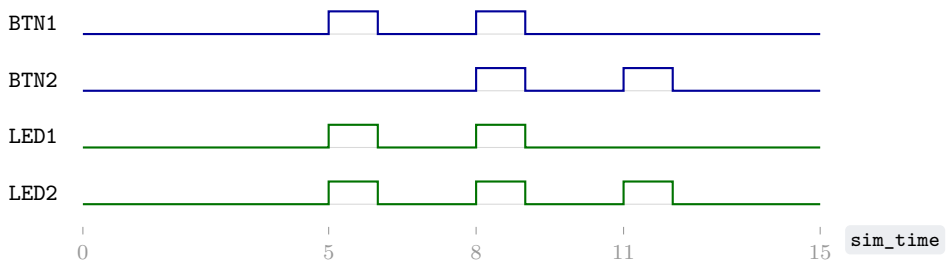


Figure 2.2: The waveforms of step 01: the inputs set by the testbench, and the outputs computed by the simulation.

In this design, the outputs follow the inputs within the same step. That is what combinational means, and in the simulation there are no wires that take time. The simulation shows the logic, not the 11 ns of the real chip.

Exercise 2.3. Change the testbench so that it also presses only the second button for a while, and check in the waveform viewer that `LED1` stays off and `LED2` goes on.

2.5 Pictures of a Design

`make svg` and `make png` produce pictures like Figure 2.1. They run yosys with only light processing (`prep` instead of `synth_ice40`) and let the tool netlistsvg draw the result. So the picture shows the logic as we wrote it, with an AND and two ORs, not the single LUT that ends up on the chip. That is what makes it useful: it is the most direct check that the text describes the circuit we had in mind.

For small designs this is the best way to see a circuit. For the larger steps the pictures become too big to read, and we turn to other views: how many logic cells each part of a design uses (`make report`), and where they end up on the chip (`make floorplan`). Appendix A describes them.

2.6 Solutions

Exercise 2.1. By absorption, $(a \wedge b) \vee a = a$: if $a = 1$ the right side is 1; if $a = 0$ both terms are 0. So `LED1` is on exactly when the first button is pressed, and `LED1` needs no gate at all. yosys finds this, and the netlist connects `LED1` directly to `BTN1`.

Exercise 2.2. The LUT selects $t_{8\cdot\texttt{BTN1}+4\cdot\texttt{BTN2}}$, and for `BTN1` $\dot\vee$ `BTN2` we need $t_0 = 0$, $t_4 = 1$, $t_8 = 1$, $t_{12} = 0$. If yosys keeps the inputs as before and again sets the unused bits like the bit they share the higher inputs with, you get `0000111111110000`. The unused bits may come out differently; what must hold is $t_0 = t_{12} = 0$ and $t_4 = t_8 = 1$. (yosys may also connect the buttons to other inputs of the LUT. Then the positions change, but the rule for reading them is the same.)

Exercise 2.3. For example, add a case to the `switch`:

```
case 14:
    dut->BTN1 = 0;
    dut->BTN2 = 1;
    break;
```

Since the `default` case releases both buttons again at the next step, the button is pressed for one step only. To keep it pressed for longer, use a range of times, for instance with an `if` instead of the `switch`.

Chapter 3

Step 02: Flip-Flops

Step 01 had no memory: the LEDs showed a function of the buttons at that very moment. Now we add state. The design of `02_flip_flops` consists of two flip-flops and the logic that computes their next values, a very small *state machine*. The new piece of SystemVerilog is `always_ff`.

3.1 The Circuit

Two D flip-flops store the bits Q_1 and Q_2. At every rising edge of the clock they take over the values

$$Q_1' = \neg Q_1 \vee Q_2, \qquad Q_2' = \neg(Q_1 \wedge Q_2),$$

which a combinational circuit computes from the current values. The outputs of the flip-flops are fed back into this circuit (Figure 3.1).

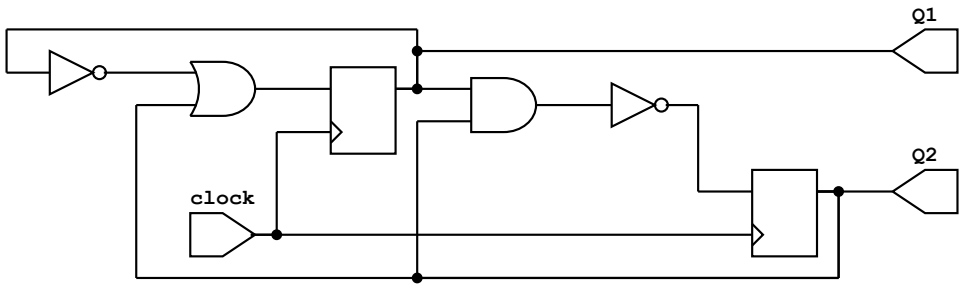


Figure 3.1: The circuit of step 02, drawn by `make svg`: two flip-flops (the boxes with the small triangle at the clock input) and the logic between their outputs and inputs.

Mathematically, the circuit is a map $\{0,1\}^2 \to \{0,1\}^2$, $(Q_1, Q_2) \mapsto (Q_1', Q_2')$, which is applied once per clock cycle. Starting from $(0,0)$ we get

$$(0,0) \mapsto (1,1) \mapsto (1,0) \mapsto (0,1) \mapsto (1,1) \mapsto (1,0) \mapsto \cdots$$

After the first step, the state runs through a cycle of length three.

3.2 The Description in SystemVerilog

02_flip_flops/test.sv

```
module test (
    input logic clock,
    output logic Q1,
    output logic Q2
);

    initial begin
        Q1 = 0;
        Q2 = 0;
```

```
    end

    logic nextQ1, nextQ2;

    always_comb begin
        nextQ1 = ~Q1 | Q2;
        nextQ2 = ~(Q1 & Q2);
    end

    always_ff @ (posedge clock) begin
        Q1 <= nextQ1;
        Q2 <= nextQ2;
    end

endmodule
```

The combinational part is an `always_comb` block as in step 01. It computes the two signals `nextQ1` and `nextQ2`, the inputs of the flip-flops.

`always_ff` describes flip-flops. The block

```
always_ff @ (posedge clock) begin
    Q1 <= nextQ1;
    Q2 <= nextQ2;
end
```

says: at every rising edge (`posedge`, positive edge) of `clock`, `Q1` takes over the value of `nextQ1`, and `Q2` that of `nextQ2`. Between two rising edges they keep their values. That is the definition of a D flip-flop with data input `nextQ1`, output `Q1`, and clock `clock`. So these three lines describe the two flip-flops of Figure 3.1.

`<=` is the non-blocking assignment. In `always_ff` we write `<=` instead of `=`. Read `Q1 <= nextQ1` as "at the clock edge, `Q1` becomes what `nextQ1` is just before the edge." All right-hand sides in the block are taken from before the edge, and only then are all flip-flops updated, at the same time. This is how real flip-flops behave: all of them sample their inputs at the same edge.

The rule for the two kinds of blocks is simple, and we follow it in all designs:

block	describes	assignment
`always_comb`	combinational logic	`=`
`always_ff @ (posedge clock)`	flip-flops, updated at the rising edge	`<=`

`initial` sets the values at power-up. The `initial` block states that `Q1` and `Q2` start as 0. On the FPGA this is the value the flip-flops have right after the configuration is loaded. Our flip-flops start with 0 anyway, so here it changes nothing on the chip, but it documents the start state, and the simulation needs it (otherwise it would not know where to start).

The same type, two kinds of hardware. `nextQ1` and `Q1` are both declared as `logic`. That `nextQ1` becomes a wire and `Q1` the output of a flip-flop follows only from the block that assigns it. In SystemVerilog, the hardware is determined by how a signal is used, not by its declaration.

Pitfall. Assign every signal in *one* block only. A signal that is assigned in an `always_comb` and also in an `always_ff` would be the output of a gate and of a flip-flop at the same time, two drivers on one wire. (The `initial` block is the one exception we allow ourselves: on the

FPGA it sets the power-up value of the flip-flop and drives nothing afterwards. Verilator's linter does not know that and warns about it, see Section 3.6.)

Exercise 3.1. Swap the two lines in the `always_ff` block. Does the circuit change? What if the block were

```
always_ff @ (posedge clock) begin
    Q1 <= ~Q1 | Q2;
    Q2 <= ~(Q1 & Q2);
end
```

without the `always_comb` block?

3.3 What yosys Makes of It

The statistics in `test.log` now list two kinds of building blocks:

```
2.60. Printing statistics.

=== test ===

        +----------Local Count, excluding submodules.
        |
        5 wires
        5 wire bits
        5 public wires
        5 public wire bits
        3 ports
        3 port bits
        4 submodules
        2   SB_DFF
        2   SB_LUT4
```

An `SB_DFF` is a D flip-flop, the one in a logic cell. Here is the netlist in short, one building block per line with the signals at its connections (`C` clock, `D` data input, `Q` output; `I0` to `I3` and `O` for the LUTs):

```
SB_DFF   C=clock, D=Q1_SB_DFF_Q_D, Q=Q1
SB_LUT4  I0=0, I1=0, I2=Q1, I3=Q2, O=Q1_SB_DFF_Q_D  LUT_INIT=1111111100001111
SB_DFF   C=clock, D=Q2_SB_DFF_Q_D, Q=Q2
SB_LUT4  I0=0, I1=0, I2=Q1, I3=Q2, O=Q2_SB_DFF_Q_D  LUT_INIT=0000111111111111
```

Each next-state function became one LUT, whose output goes to the data input of a flip-flop. Since a logic cell contains a LUT *and* a flip-flop, nextpnr puts each pair into one logic cell, and the whole design needs two of them (plus two for the constants, as in step 01). You can check the LUTs as in Section 2.3: with `Q2` on x_3 and `Q1` on x_2, the LUT of `Q1` selects $t_{8Q_2+4Q_1}$, and its `LUT_INIT` gives $t_0 = 1$, $t_4 = 0$, $t_8 = t_{12} = 1$: that is $\neg Q_1 \vee Q_2$.

3.4 How Fast May the Clock Be?

Now there are paths from flip-flops through logic to flip-flops, and nextpnr can say how long the slowest one takes. Its answer:

```
Info: Max frequency for clock 'clock$SB_IO_IN_$glb_clk': 232.23 MHz (PASS at 12.00 MHz)
```

With a clock of 232 MHz there would still be enough time for the signal from the output of a flip-flop through a LUT to the input of the next flip-flop. Our board's clock has 12 MHz, so there is plenty of room. That will change: the ULM in Chapter 10 reaches only about 15 MHz.

3.5 Where Are the Pins?

nextpnr also prints warnings this time:

```
Warning: net 'CLK' does not exist in design, ignoring clock constraint
Warning: IO 'clock' is unconstrained in PCF and will be automatically placed
Warning: IO 'Q2' is unconstrained in PCF and will be automatically placed
Warning: IO 'Q1' is unconstrained in PCF and will be automatically placed
```

The ports of this design are called `clock`, `Q1` and `Q2`, and none of these names is in `icebreaker.pcf`. The Makefile calls nextpnr with `--pcf-allow-unconstrained`, so instead of an error we get these warnings, and nextpnr connects the ports to *some* pins. Moreover, the line `set_frequency CLK 12` in the pin file refers to `CLK`, which does not exist here, so nextpnr does not know which clock to check against. (It still checks against 12 MHz, its default.)

Pitfall. Port names of the top module must be names from `icebreaker.pcf`, otherwise the port ends up on an arbitrary pin, and the design does nothing visible on the board. Read the warnings of nextpnr: "unconstrained in PCF" always means a typo or a forgotten pin.

To see something on the board, the ports need pin names. For the clock, the 12 MHz oscillator `CLK` is the obvious choice, but then the LEDs change twelve million times per second, and you see them all glowing at the same time. We need a slower clock, or a way to make something happen only once in a while. This is what the next step is about.

Exercise 3.2. Rename the ports so that the first button is the clock and the LEDs show the state: `clock` becomes `BTN1`, `Q1` becomes `LED1`, `Q2` becomes `LED2` (also in the testbench). Program the board and press the button a few times. Does the state always advance by one step per press?

3.6 Simulation

The testbench now has only one input to set, the clock. Its loop inverts it in every step:

02_flip_flops/tb_test.cpp

```
    std::uint64_t sim_time = 0;

    while (sim_time < MAX_SIM_TIME) {
        dut->clock ^= 1;
        dut->eval();

        m_trace->dump(sim_time);
        sim_time++;
    }
```

So the clock has a rising edge at every second step. Figure 3.2 shows the result of `make sim` in the waveform viewer.

The simulation starts at time 0 with `clock` already 1. This is not a rising edge, since there is no earlier value, so the state stays $(0, 0)$. The first rising edge is at time 2, and from there on the state runs through $(1, 1)$, $(1, 0)$, $(0, 1)$, $(1, 1)$, …, as computed above. Note how `nextQ1` and `nextQ2` change right after an edge, as soon as the state has changed: they are combinational, and they always show what the state will be after the next edge.

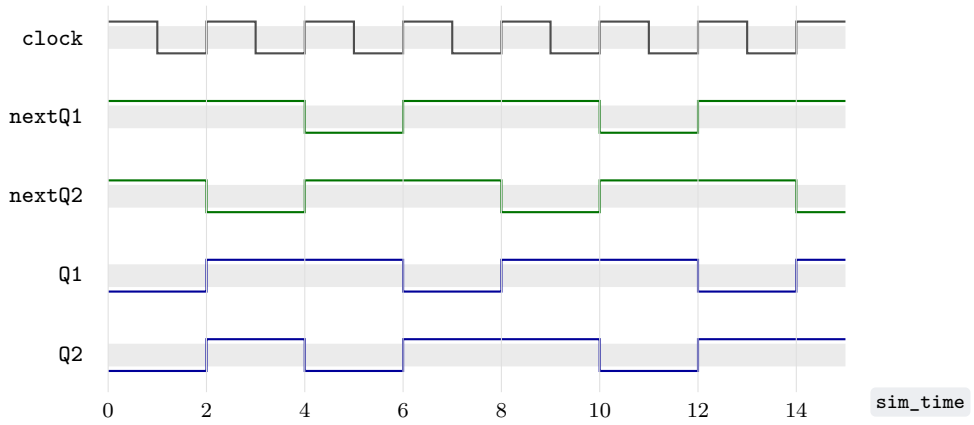


Figure 3.2: The waveforms of step 02. At every rising edge of `clock` (the gray lines), `Q1` and `Q2` take over the values that `nextQ1` and `nextQ2` had just before.

Verilator prints some warnings during `make sim`:

```
%Warning-MULTIDRIVEN: test.sv:20:9: Variable written to in always_ff also
written by other process (IEEE 1800-2023 9.2.2.4): 'Q1'
```

This is the `initial` block of the pitfall above. For a simulator, it is a second process writing `Q1`. For us, it is the power-up value of the flip-flop, and we can ignore the warning.

3.7 Solutions

Exercise 3.1. Swapping the lines changes nothing: both right-hand sides are taken from before the edge, and both flip-flops are updated together. The order of the lines in an `always_ff` block with `<=` does not matter.

The block without `always_comb` describes the same circuit as well. The expressions on the right are computed by combinational logic in any case; we only no longer give its outputs names. Here, `Q1` on the right of the second line is still the old value, since `<=` takes all right-hand sides from before the edge. (With `=` instead of `<=`, the second line would see the new value of `Q1`, and the block would describe a different circuit. This is why we never use `=` in `always_ff`.)

Exercise 3.2. After renaming, nextpnr finds all ports in the pin file, and the warnings are gone. On the board you will see the state advance, but now and then by two or more steps per press. A mechanical button does not close cleanly: its contacts bounce for a few milliseconds and produce several edges. For a clock, which must be clean, a button is a bad choice. We will instead sample buttons with the 12 MHz clock, starting in Chapter 5, and react to a press only once the contacts have settled, starting in Chapter 6.

Part II

From Counters to a Computer

Chapter 4

Step 03: A Counter on the 7-Segment Display

In step 02 the clock was too fast to see anything. Step 03 solves this with a counter: it counts the 12 million clock edges of a second and then increments a second counter, whose value appears on the two digits of the 7-segment display. For this we need many new pieces of SystemVerilog at once, but each of them describes a circuit you know: signals of several bits (vectors), constants, registers with an adder, multiplexers written as `if` and `case`, and a module used as a part of another module. Everything is in the directory `03_hexdisplay`.

4.1 The Circuit

The design consists of three parts (Figure 4.1):

- a counter `posEdgeCount` that counts the edges of the 12 MHz clock and starts again at 0 after twelve million of them;
- a one-bit register `tic`, which is 1 for exactly one clock cycle whenever `posEdgeCount` starts again, so once per second;
- an 8-bit counter `hex_val`, which is incremented in the clock cycles in which `tic` is 1. Its value goes to a second module, `dev_hex`, which drives the display.

All flip-flops in this design are clocked by the 12 MHz clock `CLK`. This is the answer to the question at the end of Chapter 3. We do not build a slower clock; we build a signal that says "now" once per second, and the register `hex_val` takes over a new value only in those clock cycles. Such a signal is called a *clock enable*. (One could also use the output of a flip-flop as the clock of other flip-flops. But then the design has several clocks whose edges are shifted against each other, and the tools can no longer check the timing between them easily. We stay with one clock.)

4.2 Vectors and Constants

Here is the top module:

03_hexdisplay/test.sv

```
module test (
    input   CLK,
    output  A0,
    output  A1,
    output  A2,
    output  A3,
    output  A4,
    output  A5,
    output  A6,
    output  A7
);

    logic [7:0] hex_pins;
    assign {A7, A6, A5, A4, A3, A2, A1, A0} = hex_pins;
```

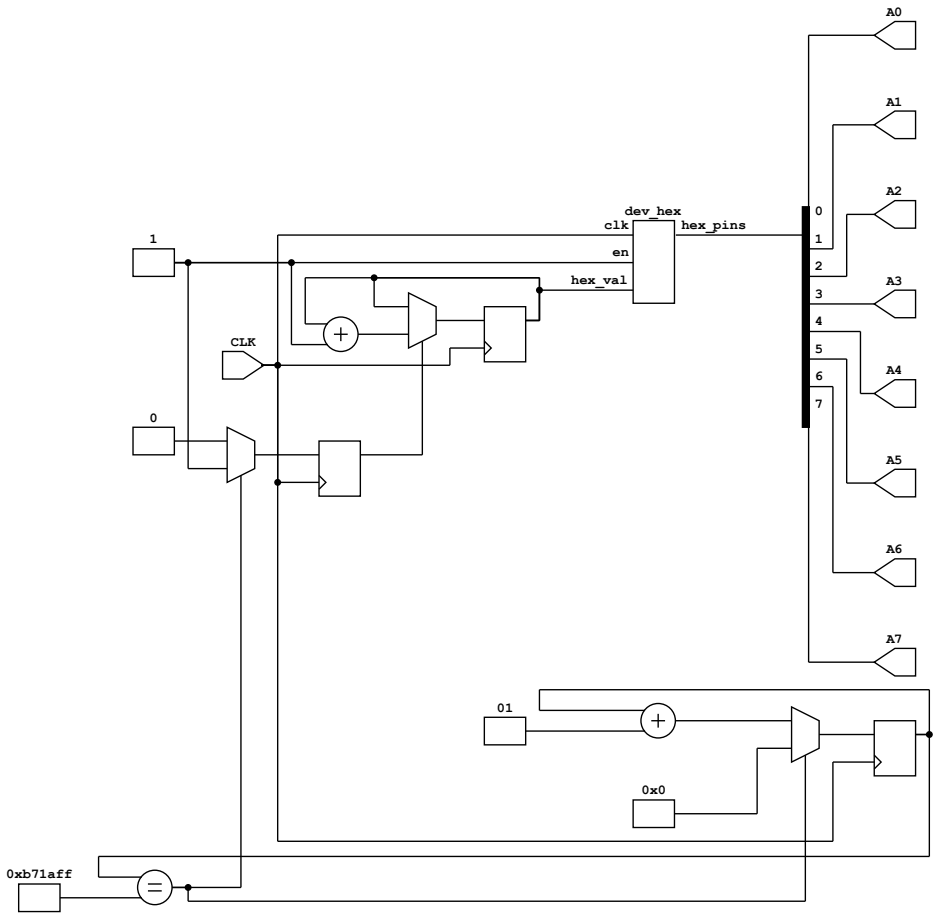


Figure 4.1: The top module of step 03, drawn by `make svg`. Bottom: `posEdgeCount` with its adder, the multiplexer that resets it and the comparison with `0xb71aff` = 11 999 999. Middle left: `tic`. Top: `hex_val` with its adder and the multiplexer that keeps the old value when `tic` is 0. The box `dev_hex` is a module of its own.

```
// clk freq
localparam CLK_FREQ = 12_000_000;
integer posEdgeCount = 0;
logic tic = 0;

always_ff @ (posedge CLK) begin
    posEdgeCount <= posEdgeCount + 1;
    tic <= 0;
    if (posEdgeCount == CLK_FREQ - 1) begin
        posEdgeCount <= 0;
        tic <= 1;
    end
end


logic [7:0] hex_val = 8'h0;

always_ff @ (posedge CLK) begin
```

```
        if (tic) begin
            hex_val <= hex_val + 1;
        end
    end

    dev_hex dev_hex0 (
        .clk(CLK),
        .en(1'b1),
        .hex_val(hex_val),
        .hex_pins(hex_pins)
    );

endmodule
```

Vectors. `logic [7:0] hex_val` declares eight signals `hex_val[7]`, …, `hex_val[0]` under one name, a *vector*, like a bus of eight wires in CircuitVerse. The range `[7:0]` says that the bits are numbered from 7 on the left down to 0 on the right. Whenever we compute with a vector, it stands for the unsigned number

$$u(hex_val) = \sum_{i=0}^{7} hex_val_i \cdot 2^i,$$

so `hex_val[7]` is the most significant bit. Parts of a vector are written `hex_val[3:0]` (the low four bits, a *nibble*) or `hex_val[7:4]` (the high nibble). In `dev_hex` you will also see `counter[2 +: 3]`: three bits, starting at bit 2 and counting upward, which is the same as `counter[4:2]`. None of this is hardware; it only names wires.

Concatenation. Braces put signals side by side into one vector, the first one on the left (most significant). Line 14,

```
assign {A7, A6, A5, A4, A3, A2, A1, A0} = hex_pins;
```

has a concatenation on the left side: it splits the vector `hex_pins` into its bits, `A7` gets `hex_pins[7]` and `A0` gets `hex_pins[0]`. The ports have to be single bits with the names from `icebreaker.pcf`, where `A0` to `A7` are the pins of PMOD 1A with the display. Inside, we prefer to work with a vector. On the right side, as in `dev_hex`, `{digit_sel, seg_pins}` joins a bit and a 7-bit vector into 8 bits.

Literals with a size. A constant can be given with its number of bits and its base: `8'h0` is the number 0 as 8 bits in hexadecimal notation, `1'b1` a single 1 bit, `7'b1000000` seven bits in binary, `4'hA` the four bits `1010`. The format is *width* `'` *base digits* with base `b`, `d` or `h`. Underscores are ignored and only help reading: `12_000_000`. A number without a size, like `1` or `0`, has 32 bits. In hardware a constant is a wire tied to 0 or 1.

`localparam` names a constant. `localparam CLK_FREQ = 12_000_000;` gives the number a name. It is not a signal and creates no hardware: wherever `CLK_FREQ` appears, the tools use the number. Even `CLK_FREQ - 1` in line 26 is computed by the tools, not by a subtractor: in Figure 4.1 it is the constant `0xb71aff` at the comparator.

`integer` is a 32-bit signed variable, an older type from Verilog. In hardware, `posEdgeCount` becomes 32 flip-flops, as we will see.

Declaration with a value. `logic tic = 0;` declares `tic` and gives it the value 0 at the start. This is the same as an `initial` block (Chapter 3): the power-up value of the flip-flops. The same holds for `posEdgeCount` and `hex_val`. Verilator again reminds us that these signals are also written in an `always_ff` block:

```
%Warning-PROCASSINIT: test.sv:21:17: Procedural assignment to declaration with
initial value: 'tic'
```

For us this is intended, as with `MULTIDRIVEN` in Chapter 3.

4.3 Counters

A counter is a register and an adder. The block of lines 35–39 says: at a rising edge of `CLK`, `hex_val` takes over `hex_val + 1` if `tic` is 1. Read it as a circuit: an 8-bit register (eight flip-flops) whose output goes through an adder that adds 1, and whose input is the output of that adder. This is the counter you built in CircuitVerse.

The adder has eight output bits, so it computes modulo 2^8:

$$u(hex_val') = \big(u(hex_val) + 1\big) \bmod 2^8,$$

and after `8'hff` comes `8'h00`. (Strictly, `hex_val + 1` has 32 bits, because the literal `1` has 32 bits. Assigning it to the 8-bit `hex_val` drops the upper 24 bits, and the result is the same.)

`if` in `always_ff` is a multiplexer. What happens if `tic` is 0? The block does not say, and for a flip-flop that means: it keeps its value. So in front of the data input of the register there is a multiplexer (Figure 4.1, top) that selects between the new value `hex_val + 1` and the old value `hex_val`, and `tic` is its select input. A register whose next value is either a new value or its old one, depending on a signal, is a register with an *enable*, and the iCE40 has a flip-flop for exactly this, `SB_DFFE`. Mathematically,

$$hex_val' = \begin{cases} hex_val + 1, & \text{if } tic = 1, \\ hex_val, & \text{otherwise.} \end{cases}$$

Several assignments: the last one wins. The block of lines 23–30 assigns `posEdgeCount` and `tic` twice:

```
posEdgeCount <= posEdgeCount + 1;
tic <= 0;
if (posEdgeCount == CLK_FREQ - 1) begin
    posEdgeCount <= 0;
    tic <= 1;
end
```

The rule is: of all assignments to a signal in a block that apply, the last one counts. So `posEdgeCount` becomes 0 if it equals `CLK_FREQ - 1`, and `posEdgeCount + 1` otherwise. And `tic` becomes 1 in that case and 0 otherwise, in short $tic' = (u(posEdgeCount) = 11\,999\,999)$. This is a description of next-state functions, not a sequence of steps: the comparison `==` is a comparator circuit, the `if` is a multiplexer behind the adder, and both flip-flops take over their new values at the same edge, as in Chapter 3.

So `posEdgeCount` runs through $0, 1, \ldots, 11\,999\,999, 0, 1, \ldots$, twelve million values, and `tic` is 1 in the clock cycle right after `posEdgeCount` was 11 999 999: once every 12 000 000 cycles, exactly once per second. A counter that produces such a pulse once every N cycles is a *clock divider*, even though no new clock comes out of it.

Exercise 4.1. An earlier version of `test.sv` compared with `CLK_FREQ` instead of `CLK_FREQ - 1`. How many clock cycles lay between two cycles in which `tic` was 1, and how long was the "second" of that design? Would you have noticed on the board?

4.4 Using a Module: dev_hex

Lines 41–46 place the display driver into our design:

```
dev_hex dev_hex0 (
    .clk(CLK),
    .en(1'b1),
    .hex_val(hex_val),
    .hex_pins(hex_pins)
);
```

This is an *instance* of the module `dev_hex`: a copy of the circuit described in `dev_hex.sv`, named `dev_hex0`, with its ports connected to our signals. In CircuitVerse terms, we place a subcircuit and wire it up. `.clk(CLK)` connects the port `clk` of the instance to our signal `CLK`. Connecting by name is safer than by position, and we always do it. The port `en` is connected to the constant 1: the display is always on. We could place several instances of the same module, each with its own name and its own flip-flops.

The Makefile passes all `.sv` files of the directory to the tools (`SV_FILES := $(wildcard *.sv)`), so `dev_hex.sv` is found without further ado; which module is the top module is given by `-top test`.

4.5 Driving the 7-Segment Display

Each digit of the display consists of seven LED segments, called a to g (Figure 4.2). The module has only eight signal lines, not $2 \cdot 7$: the seven segment lines are shared by both digits, and the eighth line selects which of the two digits is switched on. So at any moment at most one digit shows something. To show two digits, the driver alternates between them, so fast that the eye sees both at once. This is called *multiplexing* a display.

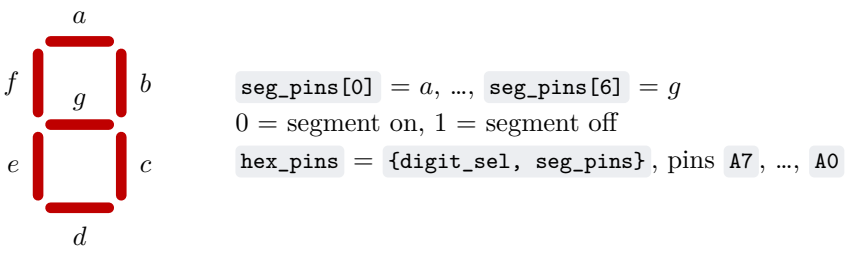


Figure 4.2: The segments of one digit, and how `dev_hex` encodes them.

The segments: a truth table

Which segments show a hexadecimal digit is a function $\{0,1\}^4 \to \{0,1\}^7$, and the last block of `dev_hex.sv` is its truth table:

03_hexdisplay/dev_hex.sv

```
    always_comb begin
        case (digit)
        4'h0: segments = 7'b1000000;
        4'h1: segments = 7'b1111001;
        4'h2: segments = 7'b0100100;
        4'h3: segments = 7'b0110000;
        4'h4: segments = 7'b0011001;
        4'h5: segments = 7'b0010010;
        4'h6: segments = 7'b0000010;
        4'h7: segments = 7'b1111000;
        4'h8: segments = 7'b0000000;
        4'h9: segments = 7'b0010000;
        4'hA: segments = 7'b0001000;
        4'hB: segments = 7'b0000011;
        4'hC: segments = 7'b1000110;
        4'hD: segments = 7'b0100001;
        4'hE: segments = 7'b0000110;
        4'hF: segments = 7'b0001110;
        endcase
    end
```

`case` **selects by value.** The `case` statement compares `digit` with each of the values on the left and applies the assignment of the matching one. In an `always_comb` block it describes combinational logic, here literally a table with 16 rows and 7 output columns. The bits are written from `segments[6]` (g) on the left to `segments[0]` (a) on the right, and a 0 switches a segment on. For example, `4'h1` gives `1111001`: only b and c are on, the two segments on the right, which form a "1". And `4'h8` gives `0000000`, all segments on.

Every segment is a Boolean function of the four bits of `digit`. A function of four bits is exactly what one LUT computes (Section 1.2), so the whole table becomes seven LUTs, one per segment.

> **Pitfall.** A `case` in `always_comb` must assign a value in *every* case. Here it does: all 16 values of a 4-bit `digit` are listed. If a case were missing, the output would have to keep its old value for that input, and a combinational circuit cannot keep anything. The tools then build a *latch*, a storage element without a clock, which is almost never what you want. Assigning a default value first, as the block that computes `digit` does with `digit = 0;`, avoids this. In `always_ff`, by contrast, a missing assignment is fine: the flip-flop keeps its value, as we saw with `hex_val`.

Multiplexing the two digits

The rest of `dev_hex` decides which digit is shown when:

03_hexdisplay/dev_hex.sv

```
// counter increments at CLK = 12 MHz.
// display refreshes at 375 KHz.
logic [29:0]     counter;
logic [2:0]      display_state;
assign display_state = counter[2 +: 3];

 logic [6:0] segments;
 logic [3:0] digit;

 always @(posedge clk) begin
     if (!en) begin
         seg_pins <= ~0;
     end
     else begin
         counter <= counter + 1;

         // Switch seg_pins_n off during digit_sel transitions
         // to prevent flicker.  Each digit has 25% duty cycle.
         case (display_state)
             0, 1:
                 seg_pins <= segments;
             2:
                 seg_pins <= ~0;
             3:
                 digit_sel <= 0;
             4, 5:
                 seg_pins <= segments;
             6:
                 seg_pins <= ~0;
             7:
                 digit_sel <= 1;
         endcase
     end
 end
```

A counter as a time base. `counter` is a 30-bit counter that is incremented at every clock edge. Only its bits 4 to 2 are used: line 17 connects them to `display_state`. They change every 4 clock cycles and run through $0, 1, \ldots, 7$ once every 32 cycles, which is the 375 kHz of the comment (12 MHz/32). yosys notices that bits 5 to 29 are never used and removes them, so only five flip-flops remain. (`always @(posedge clk)` is the older Verilog form of `always_ff @ (posedge clk)`, and describes the same flip-flops here.)

Which nibble. The next block is a multiplexer:

03_hexdisplay/dev_hex.sv

```
 always_comb begin
     digit = 0;
     case (display_state)
     0, 1:
         digit = hex_val[3:0];
     4, 5:
         digit = hex_val [7:4];
     endcase
 end
```

In states 0 and 1, `digit` is the low nibble `hex_val[3:0]`, in states 4 and 5 the high nibble, otherwise 0.

When. The `case` in the clocked block sets the registers `seg_pins` and `digit_sel` depending on the state:

`display_state`	`seg_pins`	`digit_sel`
0, 1	segments of the low nibble	(keeps 1)
2	all off (`~0`)	
3		becomes 0
4, 5	segments of the high nibble	(keeps 0)
6	all off	
7		becomes 1

(`~0` inverts the 32-bit number 0, and the lower 7 bits of the result, all ones, go into `seg_pins`.) Figure 4.3 shows one period. While `digit_sel` is 1 the display shows the low nibble, which must therefore be the right digit; while it is 0, the high nibble on the left. The select line changes only in states 3 and 7, while all segments are off. If it changed while segments were on, the other digit would light up briefly with the wrong pattern, a faint "ghost". Each digit is on for 8 of the 32 cycles, the 25 % duty cycle of the comment.

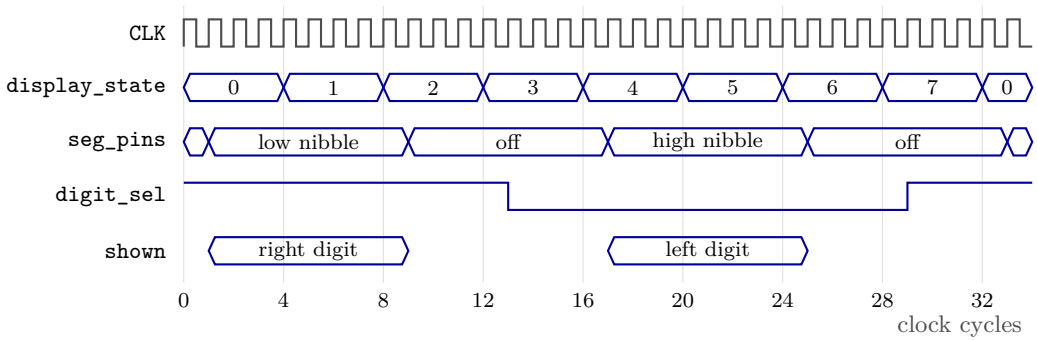


Figure 4.3: One period of `dev_hex`, 32 clock cycles. The registers `seg_pins` and `digit_sel` change one clock edge after `display_state` enters a new state. "shown" is what lights up.

Pitfall. Lines 16 and 17 declare `display_state` and connect it with `assign`. An earlier version of `dev_hex.sv` did both in one line,

```
logic [2:0] display_state = counter[2 +: 3];
```

which looks like a connection, but by the rules of SystemVerilog it is a declaration with an initial value, like `logic tic = 0;` in `test.sv`: `display_state` gets the value of `counter[4:2]` once at the start and keeps it. Verilator follows this rule: in `make sim`, `display_state` stayed 0 while `counter` counted, so the simulated display never switched digits. yosys (via sv2v) treated the line as a connection, and on the board the display worked. Simulation and hardware disagreed, and neither tool warned. A declaration with a value is a power-up value; to connect, write `assign`. (The same version also used `segments` in the clocked block before declaring it further down. The tools accepted that, but a name should be declared before it is used, as lines 19 and 20 do now.)

4.6 What yosys Makes of It

yosys flattens the design: the instance `dev_hex0` disappears as a box, and its parts appear in the netlist with names that begin with `dev_hex0.`. The statistics:

```
2.60. Printing statistics.

=== test ===

        +----------Local Count, excluding submodules.
        |
       ...
      162 submodules
       39   SB_CARRY
        7   SB_DFF
        8   SB_DFFE
        7   SB_DFFESS
       32   SB_DFFSR
       69   SB_LUT4
```

There are now four kinds of flip-flops, all of them the flip-flop of a logic cell with extra inputs: `E` (enable), `R` (reset to 0) and `S` (set to 1). Reset and set are *synchronous*, they act at the clock edge like the data input. With the cell names from `test.json` we can tell what each one is:

count	type	signals
32	`SB_DFFSR`, with synchronous reset	`posEdgeCount`
8	`SB_DFFE`, with enable	`hex_val`, enabled by `tic`
7	`SB_DFFESS`, with enable and synchronous set	`seg_pins`
7	`SB_DFF`, plain	`counter[4:0]`, `digit_sel`, `tic`

The `if` that resets `posEdgeCount` became the `R` input: when the comparator says "equal", the flip-flops become 0 at the next edge. The "all off" of `seg_pins` became the `S` input. All 32 bits of the `integer` are there, although $12\,000\,000 < 2^{24}$ and the upper 8 bits are always 0; yosys cannot know that (Exercise 4.2).

Adders and the carry chain

The 39 `SB_CARRY` belong to the three adders: 30 to the 32-bit `posEdgeCount`, 6 to the 8-bit `hex_val`, 3 to the 5 remaining bits of `counter`. Here is the netlist of the counter `hex_val`, one building block per line, with the generated net names abbreviated: `d[i]` is the data input of the flip-flop of bit i and `c[i]` the carry into bit i:

```
SB_DFFE  C=CLK, D=d[0], E=tic, Q=hex_val[0]
SB_LUT4  I0=0, I1=0, I2=0, I3=hex_val[0], O=d[0]  LUT_INIT=0000000011111111
SB_DFFE  C=CLK, D=d[1], E=tic, Q=hex_val[1]
SB_LUT4  I0=0, I1=0, I2=hex_val[1], I3=hex_val[0], O=d[1]  LUT_INIT=0110100110010110
SB_DFFE  C=CLK, D=d[2], E=tic, Q=hex_val[2]
SB_LUT4  I0=0, I1=0, I2=hex_val[2], I3=c[2], O=d[2]  LUT_INIT=0110100110010110
SB_CARRY CI=hex_val[0], CO=c[2], I0=0, I1=hex_val[1]
SB_DFFE  C=CLK, D=d[3], E=tic, Q=hex_val[3]
SB_LUT4  I0=0, I1=0, I2=hex_val[3], I3=c[3], O=d[3]  LUT_INIT=0110100110010110
SB_CARRY CI=c[2], CO=c[3], I0=0, I1=hex_val[2]
...
SB_DFFE  C=CLK, D=d[7], E=tic, Q=hex_val[7]
SB_LUT4  I0=0, I1=0, I2=hex_val[7], I3=c[7], O=d[7]  LUT_INIT=0110100110010110
SB_CARRY CI=c[6], CO=c[7], I0=0, I1=hex_val[6]
```

Read it as a ripple-carry adder for $h + 1$, with $h =$ `hex_val`. Adding 1 means: bit i flips if all bits below it are 1. With the carry c_i into bit i,

$$c_0 = 1, \qquad d_i = h_i \mathbin{\dot\vee} c_i, \qquad c_{i+1} = h_i \wedge c_i.$$

For bit 0 this is $d_0 = \neg h_0$, a LUT that inverts (`LUT_INIT` 1 for $h_0 = 0$), and the carry into bit 1 is h_0 itself. For every other bit, a LUT computes $h_i \dot{\vee} c_i$ (`0110100110010110` is the XOR of all four inputs, two of which are 0), and an `SB_CARRY` computes the carry. An `SB_CARRY` computes the carry of a full adder, $I_0 I_1 \vee (I_0 \vee I_1)\, CI$, which is $h_i \wedge c_i$ for $I_0 = 0$.

The `SB_CARRY` is not a LUT. It is a small piece of fixed logic in every logic cell, next to the LUT, with a direct wire to the logic cell above it in the same column. These wires form the *carry chain*. nextpnr places the bits of an adder in consecutive logic cells of one column, so that the carry runs up the chain without using the general routing.

4.7 What nextpnr Makes of It

```
Info: Device utilisation:
Info:            ICESTORM_LC:      75/   5280     1%
Info:                   SB_IO:       9/     39    23%
Info:                   SB_GB:       1/      8    12%
```

75 logic cells. `SB_GB` is one of the eight *global buffers*: the clock reaches all flip-flops over a special network with little delay difference between them, and nextpnr puts `CLK` on it.

The slowest path now starts at `posEdgeCount[0]` and runs along the carry chain to the top bit:

```
Info: Critical path report for clock 'CLK$SB_IO_IN_$glb_clk' (posedge -> posedge):
Info:       type curr  total name
Info:   clk-to-q  1.39  1.39 Source posEdgeCount_SB_DFFSR_Q_9_D_SB_LUT4_O_LC.O
Info:    routing  1.76  3.15 Net posEdgeCount[0] (5,1) -> (4,1)
Info:                         Sink $nextpnr_ICESTORM_LC_0.I1
Info:      logic  0.68  3.83 Source $nextpnr_ICESTORM_LC_0.COUT
Info:    routing  0.00  3.83 Net $nextpnr_ICESTORM_LC_0$O (4,1) -> (4,1)
Info:                         Sink posEdgeCount_SB_DFFSR_Q_8_D_SB_LUT4_O_LC.CIN
Info:      logic  0.28  4.10 Source posEdgeCount_SB_DFFSR_Q_8_D_SB_LUT4_O_LC.COUT
Info:    routing  0.00  4.10 Net posEdgeCount_SB_DFFSR_Q_7_D_SB_LUT4_O_I3 (4,1) -> (4,1)
Info:                         Sink posEdgeCount_SB_DFFSR_Q_7_D_SB_LUT4_O_LC.CIN
Info:      logic  0.28  4.38 Source posEdgeCount_SB_DFFSR_Q_7_D_SB_LUT4_O_LC.COUT
Info:    routing  0.00  4.38 Net posEdgeCount_SB_DFFSR_Q_6_D_SB_LUT4_O_I3 (4,1) -> (4,1)
Info:                         Sink posEdgeCount_SB_DFFSR_Q_6_D_SB_LUT4_O_LC.CIN
...
Info:      logic  0.28  13.56 Source posEdgeCount_SB_DFFSR_Q_25_D_SB_LUT4_O_LC.COUT
Info:    routing  0.00  13.56 Net posEdgeCount_SB_DFFSR_Q_26_D_SB_LUT4_O_I3 (4,4) -> (4,4)
Info:                         Sink posEdgeCount_SB_DFFSR_Q_26_D_SB_LUT4_O_LC.CIN
Info:      logic  0.28  13.83 Source posEdgeCount_SB_DFFSR_Q_26_D_SB_LUT4_O_LC.COUT
Info:    routing  0.66  14.50 Net posEdgeCount_SB_DFFSR_Q_10_D_SB_LUT4_O_I3 (4,4) -> (4,4)
Info:                         Sink posEdgeCount_SB_DFFSR_Q_10_D_SB_LUT4_O_LC.I3
Info:      setup  0.82  15.32 Source posEdgeCount_SB_DFFSR_Q_10_D_SB_LUT4_O_LC.I3
Info: 11.23 ns logic, 4.09 ns routing

Info: Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 65.27 MHz (PASS at 12.00 MHz)
```

Each step along the chain takes 0.28 ns, and the positions show the chain climbing the column $x = 4$ from tile $(4, 1)$ to tile $(4, 4)$: eight logic cells per tile, four tiles for 32 bits. 30 steps of 0.28 ns are the largest part of the 15.32 ns. The maximal clock frequency, 65 MHz, is still far above our 12 MHz. But we see a rule: the longer an adder, the slower the design. In the ULM, with its 64-bit adder, this will matter.

Exercise 4.2. Declare `posEdgeCount` as `logic [23:0]` instead of `integer`. Why are 24 bits enough? Predict the changes in the statistics of yosys, then check them, and look at the logic cells and the maximal frequency in the output of nextpnr.

4.8 Simulation

The testbench toggles the clock, 35 rising edges in all. That is enough to see `posEdgeCount` and `counter` count in the waveform viewer, and to see the display switch digits: after each rising edge, the testbench compares the output `A7`, which is `digit_sel`, with its value one edge before, and counts the changes. At the end it prints the count, and if `digit_sel` never changed, it ends with an error, so that `make sim` fails. This is exactly the check that would have caught the mistake of the pitfall above. `tic`, however, would need twelve million edges. For a simulation we make the second shorter:

> **Exercise 4.3.** Set `CLK_FREQ` to 2 and `MAX_SIM_TIME` in `tb_test.cpp` to 200, and run `make sim`. Predict at which clock edges `tic` is 1 and when `hex_val` changes, and check it in the waveform viewer. Then look at `display_state`, `digit_sel` and `seg_pins`, and compare with Figure 4.3.

(Remember to undo the change of `CLK_FREQ` before you program the board.)

4.9 What Is Next

The board now shows a number, and we know how to make something happen at a human pace with a 12 MHz clock. The display driver `dev_hex` stays with us until the last step. What is still missing is input: in step 04, the number on the display comes from the DIP switch, stored into a register at the press of a button.

4.10 Solutions

Exercise 4.1. With `== CLK_FREQ`, `posEdgeCount` ran through $0, 1, \ldots, 12\,000\,000$ and then started again at 0. These are 12 000 001 values, so `tic` was 1 every 12 000 001 cycles, and the second was one clock cycle, 83 ns, too long. On the board nobody notices: after a day, the display is 7 ms behind. A counter that starts at 0 and should produce a pulse every N cycles has to start again after $N - 1$; such off-by-one errors are found by counting in a simulation, not by watching the board.

Exercise 4.2. The counter never exceeds $11\,999\,999 < 2^{24} = 16\,777\,216$, so 24 bits hold all its values. With `logic [23:0]`, yosys reports 24 `SB_DFFSR` instead of 32, 31 `SB_CARRY` instead of 39 (8 fewer bits of the adder), and 58 `SB_LUT4` instead of 69. nextpnr then needs 64 logic cells instead of 75, and the maximal frequency rises from 65 MHz to about 76 MHz, because the carry chain is 8 steps shorter.

Exercise 4.3. With `CLK_FREQ` = 2, `posEdgeCount` runs through 0, 1, so `tic` is 1 in every second clock cycle, and `hex_val` is incremented at the edge that ends such a cycle: 1, 2, 3, … at every second edge. `display_state` changes every 4 cycles, `seg_pins` shows the segments of the low nibble, all ones, the segments of the high nibble, all ones, and `digit_sel` becomes 0 and 1 in the middle of the "off" phases, as in Figure 4.3. (Since `hex_val` now changes every two cycles, the segments shown in one phase are those of the value when the phase began, and may change during the phase.)

Chapter 5

Step 04: Storing the DIP Switch

Step 03 showed a number on the display; now we choose the number ourselves. The design of `04_dip_switch` reads eight bits from a DIP switch and stores them in an 8-bit register when the first button is pressed. The display shows the register. There is almost no new SystemVerilog in this step. What is new is that a button controls a register, and after the bouncing button at the end of Chapter 3 we look closely at what this means.

5.1 The Circuit

A DIP switch is a row of eight small switches, each one giving a constant bit for as long as we leave it alone. It sits on its own PMOD module, here on PMOD 1B, next to the display on PMOD 1A (Figure 5.1).

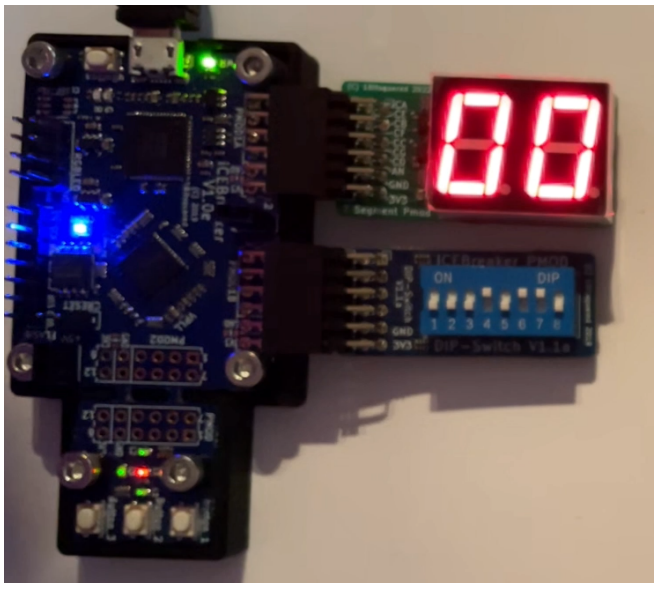


Figure 5.1: The board with the 7-segment display (PMOD 1A, top right), the DIP switch (PMOD 1B, right) and the module with buttons and LEDs (PMOD 2, bottom). The display shows `00`, the power-up value of the register.

The circuit is an 8-bit register with enable. Its data inputs are the switches, its enable is the button:

$$hex_val' = \begin{cases} hex_in, & \text{if } BTN1 = 1, \\ hex_val, & \text{otherwise,} \end{cases}$$

and `dev_hex` from Chapter 4 shows `hex_val`.

5.2 The Description in SystemVerilog

04_dip_switch/test.sv

```
module test (
    input logic CLK,
    output logic A0,
    output logic A1,
    output logic A2,
    output logic A3,
    output logic A4,
    output logic A5,
    output logic A6,
    output logic A7,
    input logic B0,
    input logic B1,
    input logic B2,
    input logic B3,
    input logic B4,
    input logic B5,
    input logic B6,
    input logic B7,
    input logic BTN1
);

    logic [7:0] hex_out;
    assign {A7, A6, A5, A4, A3, A2, A1, A0} = hex_out;

    logic [7:0] hex_in;
    assign hex_in = {B0, B1, B2, B3, B4, B5, B6, B7};

    logic [7:0] hex_val = 8'h0;

    always_ff @ (posedge CLK) begin
        if (BTN1) begin
            hex_val <= hex_in;
        end
    end

    dev_hex dev_hex0 (
        .clk(CLK),
        .en(1'b1),
        .hex_val(hex_val),
        .hex_pins(hex_out)
    );

endmodule
```

Every piece of this was introduced in Chapter 4. The file `dev_hex.sv` in this directory is a link to the one of step 03, so both steps use the same display driver.

The bit order. Line 26 collects the eight switch inputs into a vector, with `B0` on the left:

```
assign hex_in = {B0, B1, B2, B3, B4, B5, B6, B7};
```

So `B0` is `hex_in[7]`, the most significant bit, and `B7` is `hex_in[0]`. If `B0` to `B7` are the switches 1 to 8 from left to right, the switches read as the binary number from left to right,

the way we write numbers. (The concatenation makes this possible without a single gate: it only decides which wire is which bit.)

The register. The `always_ff` block in lines 30–34 is the register with enable of Section 5.1: `if (BTN1)` loads, and without an `else` the register keeps its value. This is the same construction as `hex_val` in step 03, only with `hex_in` instead of `hex_val + 1`, and with the button instead of `tic`.

5.3 What yosys Makes of It

```
2.60. Printing statistics.

=== test ===

        +----------Local Count, excluding submodules.
        |
       ...
       42 submodules
        3   SB_CARRY
        6   SB_DFF
        8   SB_DFFE
        7   SB_DFFESS
       18   SB_LUT4
```

Compared with step 03, the one-second counter is gone (no `SB_DFFSR`, 36 fewer `SB_CARRY`), and what is left besides `dev_hex` is the register. Its eight flip-flops in the netlist:

```
SB_DFFE  C=CLK, D=B0, E=BTN1, Q=hex_val[7]
SB_DFFE  C=CLK, D=B1, E=BTN1, Q=hex_val[6]
SB_DFFE  C=CLK, D=B2, E=BTN1, Q=hex_val[5]
SB_DFFE  C=CLK, D=B3, E=BTN1, Q=hex_val[4]
SB_DFFE  C=CLK, D=B4, E=BTN1, Q=hex_val[3]
SB_DFFE  C=CLK, D=B5, E=BTN1, Q=hex_val[2]
SB_DFFE  C=CLK, D=B6, E=BTN1, Q=hex_val[1]
SB_DFFE  C=CLK, D=B7, E=BTN1, Q=hex_val[0]
```

This is as direct as it gets. Each switch goes straight into the data input `D` of a flip-flop, and the button straight into all eight enable inputs `E`. There is no LUT between them: the multiplexer of the `if` is the enable of the `SB_DFFE`. (The yosys option `-dffe_min_ce_use 4` in the Makefile says to use the enable input only if at least four flip-flops share the same enable signal; for a single flip-flop like `digit_sel` in `dev_hex`, yosys builds the multiplexer with a LUT instead.)

The flip-flops are clocked by `CLK`, not by the button. This is the difference to the experiment of Exercise 3.2, where the button *was* the clock.

5.4 The Button

In Exercise 3.2, the state sometimes advanced by two or more steps per press, because the contacts of the button bounce: for a few milliseconds after we press it (and again when we release it), the signal jumps between 0 and 1 several times. Each of these jumps was a clock edge.

Here, `BTN1` is not a clock. At every edge of the 12 MHz clock, the register looks at the button. Pressing it for a tenth of a second means that `BTN1` is 1 at about 1.2 million clock edges, and at each of them the register loads the switches. During the bouncing, the enable goes on and off a few times, and the register loads in some cycles and not in others.

None of this matters, because loading is *idempotent*: loading the same value twice gives the same result as loading it once. Let L_s be the map "load s", $L_s(h) = s$ for every register value h. Then $L_s \circ L_s = L_s$, and whether the register loads once, three times or a million times during a press, it ends up with s, the setting of the switches. In step 02 the map applied per edge was the next-state map of the state machine, and applying it twice is not the same as applying it once. That is why bouncing mattered there and does not matter here.

Is the button synchronised? No. The button changes whenever we press it, not in step with the clock, and its signal goes straight into the enable inputs of eight flip-flops. If it changes just before a clock edge, it may reach some of the eight flip-flops in time and others not, and in rare cases a flip-flop may even need a little longer than usual to decide between 0 and 1 (*metastability*). Usually one samples such an input with two flip-flops in a row first, a *synchroniser*, and uses only the output of the second one. Here we can do without: if some flip-flops load at the edge where the press begins and others do not, they all load at the next edge, 83 ns later. And when the button is released, all eight bits already hold the setting of the switches, so it does not matter which of them load once more.

Exercise 5.1. Press the button, and while you keep it pressed, change a switch. What does the display do? What happens if you change a switch while the button is not pressed?

Exercise 5.2. Suppose we want to count the presses of `BTN1` instead: `hex_val` is incremented by one per press. Why does `if (BTN1) hex_val <= hex_val + 1;` not work? Describe a circuit that produces a pulse of one clock cycle when the (synchronised) button changes from 0 to 1, write it in SystemVerilog, and say what bouncing does to it.

5.5 What nextpnr Makes of It

```
Info: Device utilisation:
Info:            ICESTORM_LC:     29/  5280     0%
Info:                   SB_IO:    18/    39    46%
Info:                   SB_GB:     1/     8    12%
```

29 logic cells: the eight flip-flops of the register and `dev_hex`. 18 `SB_IO`: the clock, the button, eight switches, eight display pins. nextpnr finds all of them in `icebreaker.pcf`, which has a section for the DIP switch:

```
# for DIP switch
set_io -nowarn B0         43
set_io -nowarn B1         38
...
set_io -nowarn B7         28
```

These are the same pins as `P1B1` to `P1B4` and `P1B7` to `P1B10` further down in the file, the signal pins of PMOD 1B. (A PMOD connector has twelve pins in two rows; in each row, one pin is ground and one is 3.3 V, which leaves eight signals, numbered 1–4 and 7–10.) So, as in Section 3.5, there is no warning about unconstrained pins.

5.6 Simulation

The testbench changes the inputs at the falling edges of the clock, half a cycle before the rising edge that samples them:

04_dip_switch/tb_test.cpp

```
    while (sim_time < MAX_SIM_TIME) {
        dut->CLK ^= 1;

        if (dut->CLK == 1) {
            posedge_cnt++;
        } else {
            // Change the inputs at the falling edge, half a clock cycle
            // before the rising edge that samples them.
            dut->B0 = 0;
            dut->B1 = 0;
            dut->B2 = 0;
            dut->B3 = 0;
            dut->B4 = 0;
            dut->B5 = 0;
            dut->B6 = 0;
            dut->B7 = 0;
            switch (posedge_cnt + 1) {
                case 5:
                    dut->B0 = 1;
                    dut->B1 = 1;
                    dut->BTN1 = 1;
                    break;
                case 8:
                    dut->B1 = 1;
                    dut->BTN1 = 1;
                    break;
                case 11:
                    dut->B0 = 1;
                    dut->B7 = 1;
                    dut->BTN1 = 0;
                    break;
                default:
                    dut->BTN1 = 0;
            }
        }
        dut->eval();
```

`posedge_cnt + 1` is the number of the next rising edge. For the 5th rising edge it sets `B0` and `B1` and presses the button, for the 8th it sets only `B1` with the button, for the 11th it sets `B0` and `B7` without the button. Figure 5.2 shows the result of `make sim`.

The inputs change at time 7, and `hex_val` becomes `c0` = `11000000` at the rising edge at time 8: `B0` and `B1` are the two most significant bits. It becomes `40` at time 14, and at time 20 nothing happens, since the button is not pressed.

Pitfall. An earlier version of the testbench set the inputs at the rising edges, in the same step as the clock, before `eval()`. Then an input and the clock change at the same moment, and whether the flip-flops see the old or the new input at that edge is a race. Verilator let them see the new inputs, so `hex_val` changed at the very edge at which the inputs changed.

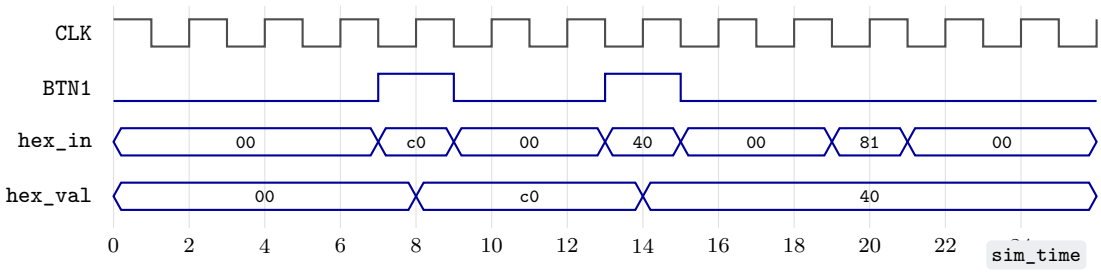


Figure 5.2: The waveforms of step 04 (values in hexadecimal). The inputs change at falling edges. `hex_val` takes over `hex_in` at the rising edges at which `BTN1` is 1, and ignores `hex_in` otherwise.

On a real chip, an input that changes together with the clock edge violates the *setup time*, the short time before the edge during which the input of a flip-flop must be stable. A testbench should change its inputs away from the active edge, for example at the falling edge, as this one does now.

5.7 What Is Next

We now have input (the switches), storage (a register), and output (the display). Step 05 adds the missing piece of a computer, arithmetic: four registers, an adder/subtractor and status flags, controlled by instructions that we set on the DIP switch.

5.8 Solutions

Exercise 5.1. While the button is pressed, the register loads the switches at every clock edge, so the display follows the switches at once: the register behaves like a wire from the switches to the display. When you release the button, the register keeps the last setting. Without the button, the switches have no effect at all.

Exercise 5.2. `if (BTN1) hex_val <= hex_val + 1;` increments at every clock edge while the button is pressed, about 1.2 million times for a press of a tenth of a second. We must react to the *change* of the button, not to its level. For this we remember the value of the button from the cycle before: a press begins in the cycle in which the button is 1 now and was 0 one cycle before. As a circuit: a synchroniser of two flip-flops, a third flip-flop that delays by one cycle, and an AND gate with an inverted input:

```
logic btn_s1 = 0, btn_s2 = 0, btn_prev = 0;

always_ff @ (posedge CLK) begin
    btn_s1   <= BTN1;       // synchroniser, first stage
    btn_s2   <= btn_s1;     // synchroniser, second stage
    btn_prev <= btn_s2;     // value one clock earlier
end

logic press;
assign press = btn_s2 & ~btn_prev;

always_ff @ (posedge CLK) begin
    if (press) begin
        hex_val <= hex_val + 1;
    end
end
```

`press` is 1 for exactly one cycle per rising edge of `btn_s2`. But bouncing produces several rising edges per press, and each of them is counted: we are back at the problem of Exercise 3.2, because incrementing is not idempotent. The remedy is a *debouncer*: accept a new value of the button only after it has been stable for some milliseconds, for example with a counter that counts the cycles in which the synchronised button differs from the accepted value, is reset to 0 as soon as they agree again, and copies the new value when it reaches 2^{17} (about 11 ms at 12 MHz). Step 05 uses exactly this circuit for its button (Section 6.3).

Chapter 6

Step 05: A Mini-ALU

With step 05 the board starts to compute. The design of `05_mini_alu` is the Mini-ALU that we build in CircuitVerse in the course HPC0 (worksheets 4 and 6): four 8-bit registers, one of them the zero register, an adder/subtractor, four status flags and a decoder. It executes 8-bit instructions that we set on the DIP switch, one per press of a button. The new pieces of SystemVerilog are the array, four registers under one name selected by a signal, and the replication `{8{sub}}`. And for the first time we react to a button press rather than to the level of the button, with the synchroniser and debouncer of Exercise 5.2.

6.1 The Mini-ALU from the Course

Let us first recall what we built in CircuitVerse, so that we can compare it with the text later.

The instruction format. An instruction is a bit pattern $(instr_7, \ldots, instr_0)$, split into four fields of two bits each:

$instr_7\, instr_6$	$instr_5\, instr_4$	$instr_3\, instr_2$	$instr_1\, instr_0$
Op	X	Y	Z

Op is the operation code, X, Y and Z name one of the four registers `%0` to `%3`, and XY is the 4-bit pattern $(instr_5, \ldots, instr_2)$. With $u(\cdot)$ for the unsigned integer that a bit pattern represents, worksheet 4 defines

Op	operation
00	$u(\mathrm{XY}) \to u(\%Z)$
01	$(u(\mathrm{XY}) \cdot 2^4 + u(\%Z)) \bmod 2^8 \to u(\%Z)$
10	$(u(\%X) + u(\%Y)) \bmod 2^8 \to u(\%Z)$
11	$(u(\%X) - u(\%Y)) \bmod 2^8 \to u(\%Z)$

The first two operations together build any 8-bit value in a register: Op = 00 sets the low nibble and clears the high one, Op = 01 then adds the high nibble.

The parts. The CircuitVerse Mini-ALU consists of

- a *register file* with four 8-bit registers, one write port (`data_in`, `addr_in`) and two read ports (`addr1_out`, `addr2_out`). A demultiplexer selects the register that receives the clock edge, a multiplexer per read port selects the register that is read. `%0` is a *zero register*: it has no flip-flops at all, reading it selects a constant 0, and a value written to it is lost;
- an 8-bit *adder/subtractor*: one adder whose second operand b is passed through XOR gates, so that `add/sub` = 1 inverts it and also feeds the carry-in (the XOR trick: $b \dot\vee 0 = b$, $b \dot\vee 1 = \neg b$);
- the *status flags*, computed from the operands a, b, the result s and the carry-out, and stored in four flip-flops: ZF = 1 if $s = 0$; CF, the carry-out for an addition and the inverted carry-out for a subtraction; SF = s_7; and OF = 1 if a and the added operand (b, or $\neg b$ for a subtraction) have the same sign and s has the other one;

- modules `ZeroExtend` and `LeftShift` for the immediate values of Op = 00 and Op = 01;
- the *ALU*, which connects them: the first operand of the adder is either the immediate value `data_in` or the register `data1_out`, chosen by `use_data1_out`; the second operand is always `data2_out`; the result goes back to the register file;
- the *decoder*, which splits the instruction with splitters and produces the control signals.

The only inputs left at the end are the instruction and the clock. The decoder of worksheet 6 connects `addr_in` to Z and `addr1_out` to X, and chooses the rest by Op:

Op	`data_in`	`addr2_out`	`add/sub`	`use_data1_out`
00	`ZeroExtend`(XY)	00, so $b = u(\%0) = 0$	0	0
01	`LeftShift`(XY)	Z	0	0
10	(not used)	Y	0	1
11	(not used)	Y	1	1

So every instruction is an addition or a subtraction $a \pm b$, even the two that load a value. (The table of operations in the decoder step of worksheet 6 writes Op = 11 as $(-u(\%X) + u(\%Y)) \bmod 2^8$. The circuit, with a from X and b from Y, computes $u(\%X) - u(\%Y)$, as worksheet 4 and the final table of worksheet 6 say, and so does our code.)

6.2 The Description in SystemVerilog

The file `test.sv` describes exactly this circuit (Figure 6.1), except for the clock, which we discuss in Section 6.3. We go through it in pieces.

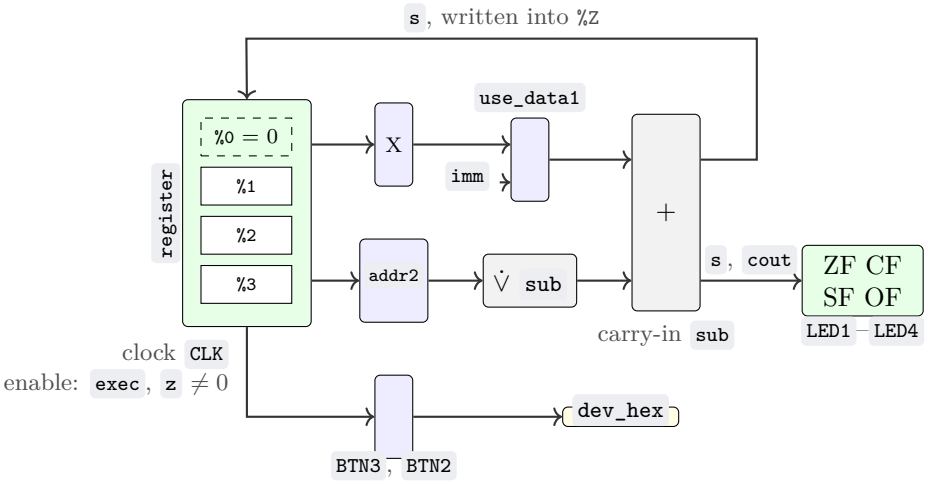


Figure 6.1: The circuit described by `05_mini_alu/test.sv`. The decoder (not drawn) computes `imm`, `addr2`, `sub` and `use_data1` from the instruction. Every read multiplexer gives 0 for address 0. At a rising edge of `CLK` in which `exec` is 1, the sum `s` is written into register Z (unless Z is `%0`) and the flags are stored.

05_mini_alu/test.sv

```
19      input logic BTN1,
20      input logic BTN2,
21      input logic BTN3,
22      output logic LED1,
23      output logic LED2,
24      output logic LED3,
25      output logic LED4
26  );
27
28      logic [7:0] hex_pins;
29      assign {A7, A6, A5, A4, A3, A2, A1, A0} = hex_pins;
30
31      logic [7:0] instr;
32      assign instr = {B0, B1, B2, B3, B4, B5, B6, B7};
33
34      logic [1:0] op, x, y, z;
35      assign {op, x, y, z} = instr;
```

The ports. `A0` to `A7` go to the 7-segment display, `B0` to `B7` come from the DIP switch, as in Chapter 5. `BTN1` executes the instruction, `BTN2` and `BTN3` select the register that the display shows. New are the outputs `LED1` to `LED4` for the flags; they are four of the five LEDs on PMOD 2, next to the buttons.

Decoding the instruction. Line 32 collects the switches into `instr`, with `B0` as the most significant bit `instr[7]`. So the instruction reads in the order `B0`, …, `B7`, in the order in which we write it down. Line 35 then splits it. A concatenation on the *left* side of an assignment (Chapter 4) distributes the bits of the right side, from the left: `op` gets `instr[7:6]`, `x` gets `instr[5:4]`, `y` gets `instr[3:2]`, `z` gets `instr[1:0]`. This is exactly the `Instruction` module

of worksheet 4: a splitter, nothing but wires. The field XY needs no name of its own; where we need it, we write `{x, y}`.

The register file

05_mini_alu/test.sv

```
59      // Register file. %0 is the zero register: every read port gives 0 for
60      // address 0, and writes to %0 are ignored. So register[0] is never used.
61      logic [7:0] register[4];
```

An array of registers. Line 61 declares an *array*: four elements `register[0]` to `register[3]`, each a vector of 8 bits. The range in front of the name, `[7:0]`, gives the bits of one element; the size after the name, `[4]`, gives the number of elements and is short for `[0:3]`. (sv2v writes it like that: `reg [7:0] register [0:3];`.) `register[2]` is an 8-bit vector, and `register[2][3:0]` is its low nibble. Like every `logic`, the array says nothing yet about hardware. That follows from its use, and it is used in two ways.

Reading with a signal as index is a multiplexer. In `register[x]` the index is not a constant but the 2-bit signal `x`. The expression stands for whichever of the four registers `x` selects, that is, a 4:1 multiplexer for 8-bit words with select input `x`. The design has three of these *read ports*: `register[x]` for the first operand, `register[addr2]` for the second, and `register[{BTN3, BTN2}]` for the display. The register file of the course has two, plus the outputs `%0` to `%3` for its four displays.

The zero register. Each read port is followed by an `if` that replaces the value by 0 when the address is 0, for example

```
b = register[addr2];
if (addr2 == 2'b00) begin
    b = 8'h00;
end
```

(lines 103–106, the last assignment wins, Chapter 4). This is the multiplexer of the course with a constant 0 at its input 0. Since no read port ever delivers `register[0]`, and the write below skips it, nothing in the design depends on `register[0]`, and yosys removes its flip-flops: as in the course, `%0` has no storage.

Writing with a signal as index selects the register. In the `always_ff` block further down, `register[z] <= s` says: at the edge, the register with number `z` takes the new value, and the other registers keep theirs. In hardware, all flip-flops share one clock. Each register gets an *enable* input: at the edge, a flip-flop with enable takes over its data input only if the enable is 1, otherwise it keeps its value. A decoder computes the enables from `z`: register i is enabled exactly when $u(\texttt{z}) = i$ (and, as we will see, `exec` is 1). The demultiplexer of worksheet 6 does the same job by passing the clock edge only to the selected register. On an FPGA we do not switch clocks with logic, since a clock should arrive at all flip-flops at the same moment; the enable achieves the same result with a clean clock.

The decoder

05_mini_alu/test.sv

```
63      // Decoder: immediate operand, second read address, add/sub, first operand
64      logic [7:0] imm;
65      logic [1:0] addr2;
66      logic sub, use_data1;
67
68      always_comb begin
69          case (op)
70              2'b00: begin
71                  imm = {4'h0, x, y};     // u(XY)
72                  addr2 = 2'b00;          // + u(%0) = 0
73              end
74              2'b01: begin
75                  imm = {x, y, 4'h0};     // u(XY) * 2^4
76                  addr2 = z;              // + u(%Z)
77              end
78              default: begin
79                  imm = 8'h00;            // not used
80                  addr2 = y;              // u(%X) +/- u(%Y)
81              end
82          endcase
83      end
84
85      assign sub = (op == 2'b11);
86      assign use_data1 = op[1];
```

This is the table of Section 6.1, written as a `case` in `always_comb` (combinational logic, Chapter 4). `imm` is the `data_in` of the course: `{4'h0, x, y}` is `ZeroExtend`, a concatenation with four zero bits on the left, and `{x, y, 4'h0}` is `LeftShift`, four zero bits on the right, so $u(\{\mathtt{x},\mathtt{y},4'\mathtt{h0}\}) = u(\mathrm{XY}) \cdot 2^4$. Neither needs a single gate. `addr2` is the `addr2_out` of the course, `sub` is `add/sub`, and `use_data1` is `use_data1_out`. The `default` branch covers Op = 10 and 11, which differ only in `sub`. Every branch assigns both outputs, so no latch is needed (Chapter 4).

The adder/subtractor

05_mini_alu/test.sv

```
88      // Adder/subtractor: s = a + b or s = a - b = a + ~b + 1
89      logic [7:0] a, b, b_xor, s;
90      logic cout;
91
92      always_comb begin
93          a = imm;
94          if (use_data1) begin
95              a = register[x];
96              if (x == 2'b00) begin
97                  a = 8'h00;
98              end
99          end
100     end
101
102     always_comb begin
103         b = register[addr2];
104         if (addr2 == 2'b00) begin
105             b = 8'h00;
106         end
107     end
108
109     assign b_xor = b ^ {8{sub}};        // b, or ~b for a subtraction
110     assign {cout, s} = {1'b0, a} + {1'b0, b_xor} + {8'h00, sub};
```

Lines 92–107 are the two operand multiplexers of Figure 6.1, with the zero register built into the read ports. Lines 109 and 110 are the adder/subtractor of worksheet 4.

Replication. `{8{sub}}` is the bit `sub` repeated eight times, a *replication*: eight wires connected to the same signal. So `b ^ {8{sub}}` is a row of eight XOR gates, each with `sub` at one input, and `b_xor` is b for an addition and $\neg b$ for a subtraction.

The carry-out. The adder in line 110 has 9-bit operands: `{1'b0, a}` puts a zero bit in front of `a`, and `{8'h00, sub}` is the carry-in as a 9-bit number. The 9-bit sum is split into its top bit `cout` and the 8 bits `s`. So

$$2^8 \cdot cout + u(s) = u(a) + u(b \dot{\vee} sub \cdots sub) + sub,$$

which for a subtraction is $u(a) + (2^8 - 1 - u(b)) + 1 = 2^8 + u(a) - u(b)$. Thus $u(s) = (u(a) - u(b)) \bmod 2^8$, the two's-complement subtraction of the course, and $cout = 1$ exactly when $u(a) \geq u(b)$. The `+` operator describes the whole carry chain; we do not write out the full adders.

The status flags

05_mini_alu/test.sv

```
112     // Status flags
113     logic ZF = 0, CF = 0, SF = 0, OF = 0;
114
115     always_ff @ (posedge CLK) begin
116         if (exec) begin
117             if (z != 2'b00) begin
118                 register[z] <= s;
119             end
120             ZF <= (s == 8'h00);
121             CF <= cout ^ sub;
122             SF <= s[7];
123             OF <= (a[7] & b_xor[7] & ~s[7]) | (~a[7] & ~b_xor[7] & s[7]);
124         end
125     end
126
127     assign {LED1, LED2, LED3, LED4} = {ZF, CF, SF, OF};
```

The flip-flops `ZF`, `CF`, `SF`, `OF` are the `Status-Flags-FF` of worksheet 6, and the expressions are those of worksheet 4:

- `s == 8'h00` is a comparator with a constant, the NOR of all eight bits of s;
- `cout ^ sub` is CF. For an addition the carry-out says that the sum does not fit into 8 bits. For a subtraction we just saw that the carry-out is 1 exactly when there is *no* borrow, $u(a) \geq u(b)$, so it has to be inverted, which is the XOR trick once more;
- `s[7]` is SF;
- for OF, `b_xor[7]` is the sign bit of the operand that is actually added, b or $\neg b$. Signed overflow happens exactly when a and this operand are both negative and s is not, or both non-negative and s is negative.

Line 127 shows the flags on the LEDs: `LED1` is ZF, `LED2` CF, `LED3` SF and `LED4` OF.

Two details follow the course. First, the flags are stored for every executed instruction, also for Op = 00 and Op = 01: the adder is used for them too. After `00001101`, which computes $3 + u(\%0) = 3 + 0$, all four flags are 0. Second, the flags do not depend on the destination. The `if (z != 2'b00)` only guards the write into the register file; the flags are assigned outside of it. An instruction with destination `%0` computes s as always, stores the flags, and discards s. This is the answer to the question of worksheet 6, and it follows from the circuit, not from a special rule.

The display shows `register[{BTN3, BTN2}]`, with 0 for `%0`: with no button pressed `%0`, with `BTN2` `%1`, with `BTN3` `%2`, with both `%3`. The module `dev_hex` is the one of Chapter 4.

Exercise 6.1. Which instruction computes $(u(\%2) - u(\%1)) \bmod 2^8 \to u(\%3)$? Write down the eight switch positions `B0`, …, `B7`. If `%1` = 3 and `%2` = 2, what does the display show for `%3` afterwards, and which LEDs are on?

Exercise 6.2. Write a program that puts `80` (hexadecimal) into `%1`, and then computes $(u(\%1) + u(\%1)) \bmod 2^8 \to u(\%1)$. Predict the register and the four flags after each instruction.

6.3 The Button

In the course, pressing the button produces the rising clock edge at which the Mini-ALU executes the instruction. On the board, the 12 MHz clock `CLK` is the only clock (Section 4.1): all flip-flops of the design, those of `dev_hex`, of the register file and of the flags, form one *clock domain*, the flip-flops driven by the same clock. The button produces a signal `exec` instead, which is 1 for exactly one clock cycle per press, and the register file and the flags are enabled by it (line 115). For the Mini-ALU, a press is then exactly one rising edge of the course.

Why not simply use `BTN1` as the clock? In the solution of Exercise 3.2 we saw that a button is a bad clock: its contacts bounce and produce several edges per press. For the Mini-ALU this would mean that an instruction is sometimes executed several times. Executing `00001101` twice puts 3 into `%1` twice, which does no harm, but `10011001`, $(u(\%1) + u(\%2)) \bmod 2^8 \to u(\%1)$, would add `%2` two or three times. The same happens if we only synchronise the button and look for its rising edge, as Exercise 5.2 showed: each bounce is a rising edge. We need a debouncer.

05_mini_alu/test.sv

```
37    // BTN1: synchroniser, debouncer, and a pulse of one clock cycle per press
38    logic btn_s1 = 0, btn_s2 = 0;   // synchroniser
39    logic btn = 0;                  // debounced button
40    logic [16:0] btn_cnt = 0;       // cycles in which btn_s2 differs from btn
41    logic exec = 0;                 // 1 for one cycle: execute the instruction
42
43    always_ff @ (posedge CLK) begin
44        btn_s1 <= BTN1;
45        btn_s2 <= btn_s1;
46        exec <= 0;
47        if (btn_s2 == btn) begin
48            btn_cnt <= 0;
49        end
50        else begin
51            btn_cnt <= btn_cnt + 1;
52            if (btn_cnt == 17'h1ffff) begin
53                btn <= btn_s2;
54                exec <= btn_s2;
55            end
56        end
57    end
```

This is the circuit of the solution of Exercise 5.2:

- `btn_s1` and `btn_s2` are the synchroniser (Section 5.4). Only `btn_s2` is used.
- `btn` is the debounced button, the value we have accepted. `btn_cnt` is a 17-bit counter, which counts the cycles in which the synchronised button differs from `btn`, and is reset to 0 as soon as both agree again. Only when they have differed for 2^{17} cycles in a row, that is, when `btn_cnt` reaches `17'h1ffff` and `btn_s2` still differs, is the new value accepted. 2^{17} cycles at 12 MHz are about 11 ms, longer than the bouncing of a button, and short enough that we do not notice the delay. During the bouncing, the button keeps returning to its old value for a moment, which resets the counter; so the counter only reaches its end once the button is stable.
- `exec` is set in the same cycle in which `btn` changes, to the new value. So it becomes 1 when a press is accepted, and not when a release is accepted. The default assignment `exec <= 0` sets it back to 0 one cycle later.

As next-state functions, with $d = 1$ if `btn_s2` $\neq$ `btn` and $e = 1$ if additionally $u(btn_cnt) = 2^{17} - 1$:

$$btn_cnt' = d \cdot \big((u(btn_cnt) + 1) \bmod 2^{17}\big), \qquad btn' = btn \mathbin{\dot\vee} e, \qquad exec' = e \wedge btn_s2.$$

Loading a value and keeping it, as in step 04, needed none of this, because loading is idempotent (Section 5.4). Executing an instruction is not.

Exercise 6.3. How long after the button has stopped bouncing is the instruction executed? What happens if you press the button only for 5 ms? And if you press it again 5 ms after releasing it?

6.4 What yosys Makes of It

yosys first recognises the array as a *memory*, an array that is read and written with a signal as index. It then decides how to build it:

```
2.35. Executing MEMORY_LIBMAP pass (mapping memories to cells).
using FF mapping for memory test.register
...
2.39. Executing MEMORY_MAP pass (converting memories to logic and flip-flops).
...
Mapping memory \register in module \test:
  created 4 $dff cells and 0 static cells of width 8.
  read interface: 0 $dff and 9 $mux cells.
  write interface: 4 write mux blocks.
```

"FF mapping" means flip-flops, as opposed to one of the block RAMs of the FPGA that we will meet in Chapter 8. A block RAM would be far too large for 32 bits, and above all it cannot do what we need: it has one read port, and it delivers the data only after a clock edge. Our register file has three read ports that deliver their values at once. So yosys builds four 8-bit registers ("4 $dff cells of width 8"), multiplexers for the read ports, and the write logic. Later it notices that nothing depends on `register[0]` and removes it. After mapping to the building blocks of the iCE40:

```
2.60. Printing statistics.

=== test ===

        +----------Local Count, excluding submodules.
        |
      158 wires
      371 wire bits
      158 public wires
      371 public wire bits
       24 ports
       24 port bits
        1 cells
        1   $scopeinfo
      220 submodules
       26   SB_CARRY
        9   SB_DFF
       28   SB_DFFE
        7   SB_DFFESS
       18   SB_DFFSR
      132   SB_LUT4
```

(`$scopeinfo` is bookkeeping about the module hierarchy, not hardware.) There are 62 flip-flops. The 28 `SB_DFFE`, with enable, are the registers `%1` to `%3` (24) and the four flags, all enabled by `exec`. The 18 `SB_DFFSR`, with synchronous reset, are `btn_cnt`, reset when the button agrees

with `btn`, and `exec`. Of the 9 `SB_DFF`, three are `btn_s1`, `btn_s2` and `btn`, and six belong to `dev_hex`, as do the 7 `SB_DFFESS` (Section 4.6). Of the 26 `SB_CARRY`, 3 belong to the counter of `dev_hex`, 15 to `btn_cnt`, and 8 to our adder/subtractor: one carry chain for all four operations, plus one for the carry-out `cout`.

nextpnr reports:

```
Info: Device utilisation:
Info:            ICESTORM_LC:     165/   5280     3%
Info:                   SB_IO:      24/     39    61%
Info:                   SB_GB:       1/      8    12%
Info: Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 47.05 MHz (PASS at 12.00 MHz)
```

One global buffer, since there is one clock. The slowest path runs from a register through a read multiplexer and the operand logic into the carry chain of the adder, and from its end back into a register: 21 ns, so 47 MHz, far above our 12 MHz. The switches are not synchronised; they are read only in the cycle in which `exec` is 1, and they do not change while we press the button.

The DIP switch is on PMOD 1B, and `icebreaker.pcf` names its eight pins `B0` to `B7`; the LEDs and buttons of PMOD 2 are `LED1` to `LED5` and `BTN1` to `BTN3`. So all 24 ports of `test` have pins, and nextpnr places them without warnings (Section 3.5).

6.5 Simulation

The testbench `tb_test.cpp` executes a program and checks every result. Each instruction comes with the predicted registers and flags:

05_mini_alu/tb_test.cpp

```
// One instruction of the program, with the predicted registers %0 to %3
// (-1: not written yet, not checked) and flags ZF, CF, SF, OF.
struct Step {
    std::uint8_t instr;
    int reg[4];
    const char *flags;
    const char *comment;
};

const Step program[] = {
    {0b00001101, {0x00, 0x03,   -1,   -1}, "0000", "u(0011) -> %1"},
    {0b00001010, {0x00, 0x03, 0x02,   -1}, "0000", "u(0010) -> %2"},
    {0b10011011, {0x00, 0x03, 0x02, 0x05}, "0000", "%1 + %2 -> %3"},
    {0b11011011, {0x00, 0x03, 0x02, 0x01}, "0000", "%1 - %2 -> %3"},
    {0b01000101, {0x00, 0x13, 0x02, 0x01}, "0000", "u(0001)*2^4 + %1 -> %1"},
    {0b11100100, {0x00, 0x13, 0x02, 0x01}, "0110", "%2 - %1 -> %0"},
    {0b01011110, {0x00, 0x13, 0x72, 0x01}, "0000", "u(0111)*2^4 + %2 -> %2"},
    {0b10101011, {0x00, 0x13, 0x72, 0xE4}, "0011", "%2 + %2 -> %3"},
    {0b01111110, {0x00, 0x13, 0x62, 0xE4}, "0100", "u(1111)*2^4 + %2 -> %2"},
    {0b11010111, {0x00, 0x13, 0x62, 0x00}, "1000", "%1 - %1 -> %3"},
    {0b00111100, {0x00, 0x13, 0x62, 0x00}, "0000", "u(1111) -> %0"},
};
```

The first four instructions are the program of worksheet 4, the fifth is from worksheet 6, and the rest test the flags: the sixth is a subtraction with destination `%0`, the eighth an addition with a signed overflow, $114 + 114 = 228$, which as a signed 8-bit number is -28.

For each instruction, the testbench sets the switches and presses and releases `BTN1` like a real button: the signal bounces four times within 2 ms, and then stays for $2^{17} + 100$ clock cycles. As in Chapter 5, the inputs change half a cycle before a rising edge. The testbench counts the cycles in which `exec` is 1, which must be exactly one per instruction. Then it reads the four registers the way we do on the board: it sets `BTN2` and `BTN3` and looks at the value `hex_val` that `dev_hex` displays. To look at signals inside the design, the testbench includes the header `Vtest___024root.h`; then `dut->rootp->test__DOT__exec` is the signal `exec` of the module `test`, and `test__DOT__dev_hex0__DOT__hex_val` is `hex_val` in the instance `dev_hex0`. The flags it reads from the outputs `LED1` to `LED4`.

```
$ make sim

### VERILATING ###
verilator -Wall --trace --x-assign unique --x-initial unique \
        -Wno-fatal \
        -cc dev_hex.sv test.sv --top-module test \
        --exe tb_test.cpp
...

### BUILDING SIM ###
make -C obj_dir -f Vtest.mk Vtest
...

### SIMULATING ###
00001101: %0=00 %1=03 %2=2F %3=2F  ZF=0 CF=0 SF=0 OF=0  ok     u(0011) -> %1
00001010: %0=00 %1=03 %2=02 %3=2F  ZF=0 CF=0 SF=0 OF=0  ok     u(0010) -> %2
10011011: %0=00 %1=03 %2=02 %3=05  ZF=0 CF=0 SF=0 OF=0  ok     %1 + %2 -> %3
11011011: %0=00 %1=03 %2=02 %3=01  ZF=0 CF=0 SF=0 OF=0  ok     %1 - %2 -> %3
01000101: %0=00 %1=13 %2=02 %3=01  ZF=0 CF=0 SF=0 OF=0  ok     u(0001)*2^4 + %1 -> %1
11100100: %0=00 %1=13 %2=02 %3=01  ZF=0 CF=1 SF=1 OF=0  ok     %2 - %1 -> %0
01011110: %0=00 %1=13 %2=72 %3=01  ZF=0 CF=0 SF=0 OF=0  ok     u(0111)*2^4 + %2 -> %2
10101011: %0=00 %1=13 %2=72 %3=E4  ZF=0 CF=0 SF=1 OF=1  ok     %2 + %2 -> %3
01111110: %0=00 %1=13 %2=62 %3=E4  ZF=0 CF=1 SF=0 OF=0  ok     u(1111)*2^4 + %2 -> %2
11010111: %0=00 %1=13 %2=62 %3=00  ZF=1 CF=0 SF=0 OF=0  ok     %1 - %1 -> %3
00111100: %0=00 %1=13 %2=62 %3=00  ZF=0 CF=0 SF=0 OF=0  ok     u(1111) -> %0
0 errors
```

Before an instruction writes them, `%2` and `%3` show `2F`: the simulation gives flip-flops without an initial value a random value; on the board they start with 0. Those values are not checked. (The random values need the line `Verilated::commandArgs(argc, argv);` in the testbench, see Appendix A.) The simulation runs for about three million clock cycles, a quarter of a second of the board, and takes less than a second. To keep the waveform file small, the testbench writes only the first 20 cycles of each press and each release, and the 20 cycles after each `exec`, into `waveform.vcd`.

Exercise 6.4. In the program above, `11100100` has destination `%0` and still sets CF and SF. How would you change `test.sv` so that an instruction with destination `%0` leaves the flags alone? Why is it useful that it does not? Hint: think of comparing two registers.

6.6 What Is Still Missing

The Mini-ALU executes one instruction per button press, and we have to set every instruction by hand. A computer fetches its instructions from a memory, and to get a program into that memory, we need a way to talk to the board from our computer. The next step does this: it connects the FPGA to the serial line of the USB chip. The register file comes back in Chapter 10, with sixteen registers of 64 bits instead of four of eight, and again with a zero register.

6.7 Solutions

Exercise 6.1. We need Op = 11, X = 10, Y = 01 and Z = 11, so the instruction is `11100111`: `B0` = 1, `B1` = 1, `B2` = 1, `B3` = 0, `B4` = 0, `B5` = 1, `B6` = 1, `B7` = 1. With `%1` = 3 and `%2` = 2 it computes $(2-3) \bmod 256 = 255$, so the display shows `FF` for `%3`. ZF = 0. The adder computes $2 + 252 + 1 = 255$ without carry-out, so there is a borrow and CF = 1. SF = 1. OF = 0: $a = 2$ is non-negative and the added operand $\neg b =$ `11111100` is negative, and the sum of two numbers with different signs never overflows. So `LED2` and `LED3` are on.

Exercise 6.2. `00000001` loads 0 into `%1` (ZF = 1, since the sum is 0, and the other flags 0), `01100001` adds $u(1000) \cdot 2^4 = 128$, so `%1` = `80` (ZF = 0, CF = 0, SF = 1, OF = 0: $0 + (-128)$ does not overflow). Then `10010101` computes $128 + 128 = 256$, so `%1` = `00`, with ZF = 1, CF = 1 (unsigned overflow), SF = 0 and OF = 1: two negative operands, $-128 + (-128) = -256$, give a non-negative result. (If `%1` was 0 already, the first instruction is not needed.) Putting the three instructions with these predictions into the table of the testbench confirms them.

Exercise 6.3. `btn` changes when the synchronised button has differed from it for 2^{17} cycles in a row, so the instruction is executed about 2^{17} cycles, 11 ms, after the last bounce. (A few cycles more: two for the synchroniser, and `exec` is a register itself. In the simulation, `exec` is 1 exactly $2^{17} + 2$ cycles after the last change of `BTN1`.) A press of 5 ms is ignored: the counter is reset when the button is released, before it reaches its end. A press 5 ms after a release is also ignored: the release has not been accepted yet, `btn` is still 1, and the new press only means that the button agrees with `btn` again. On the board, both are far shorter than a normal press.

Exercise 6.4. Move the four flag assignments into the `if (z != 2'b00)` block. But an instruction that computes flags and discards its result is useful: `11011000`, for example, computes $u(\%1) - u(\%2)$ and only keeps the flags. Afterwards ZF says whether the registers are equal and CF whether $u(\%1) < u(\%2)$, without destroying a register. Many processors have such a *compare* instruction, and with a zero register we get it for free, as worksheet 6 points out.

Chapter 7

Step 06: The Serial Line

So far the board has talked to us through LEDs and a display, and we have talked to it through buttons and switches. To load programs, we need a real connection to the computer. Step 06 uses the serial line that the USB chip provides (Section 1.4). The design of `06_io_mirror` receives every character we type in a terminal, shows its code on the display, and sends it back. So what we see in the terminal is what the FPGA sent. On the way we meet a lot of new SystemVerilog: modules with `parameter`s, state machines with `typedef enum`, a `function`, and a trap that the synthesis tool does not let us fall into.

7.1 Sending Bits One After the Other

The serial line has two wires, one for each direction. In `icebreaker.pcf` they are called `RX` (pin 6, from the computer to the FPGA) and `TX` (pin 9, from the FPGA to the computer), named from the FPGA's point of view. The circuit that sends and receives on such a line is called a *UART*, a *universal asynchronous receiver/transmitter*.

Asynchronous means: there is no clock wire. Sender and receiver only agree on the *baud rate*, the number of bits per second. Ours is 9600 baud, so one bit lasts $1/9600\,\text{s} \approx 104.2\,\mu\text{s}$, which is $12\,000\,000/9600 = 1250$ cycles of our 12 MHz clock. Each side measures this time with its own clock.

One byte is sent as a *frame* (Figure 7.1):

- When nothing is sent, the line is *idle* and stays at 1.
- A *start bit*, a 0, announces a frame. Its falling edge is the moment from which the receiver measures time.
- Then come the 8 data bits, the *least* significant bit first.
- A *stop bit*, a 1, ends the frame. Our transmitter sends two of them; a receiver needs only one, the second simply looks like idle line.

A frame with one stop bit takes 10 bit times, ours 11. The receiver samples each bit in its middle, where it is furthest from both of its edges. Since the two clocks are never exactly equal, the sample points drift; but if the baud rates of both sides differ by a few percent at most, the last sample of a frame, about 9.5 bit times after the falling edge, is still well inside its bit. The next frame starts with a new falling edge, so the error does not accumulate from frame to frame.

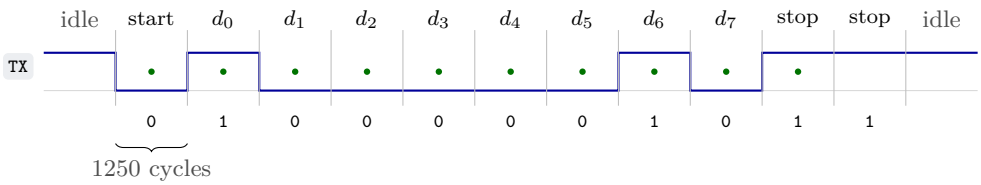


Figure 7.1: The frame for the character `A`, code $41_{16} = 01000001_2$, as `uart_tx` sends it: start bit, the data bits d_0 to d_7 starting with the least significant, two stop bits. The dots mark where a receiver samples.

7.2 The Top Module

The top module `test.sv` connects a receiver `uart_rx`, a transmitter `uart_tx` and `dev_hex`, and controls them with a small state machine. Its first part declares the signals between them:

06_io_mirror/test.sv

```
1  module test (
2      input logic CLK,
3      input logic RX,
4      output logic TX,
5      output logic A0,
6      output logic A1,
7      output logic A2,
8      output logic A3,
9      output logic A4,
10     output logic A5,
11     output logic A6,
12     output logic A7
13 );
14
15     logic rx_ready;
16     logic [7:0] rx_data_out;
17     /* verilator lint_off UNUSEDSIGNAL */
18     logic rx_idle;
19     logic rx_eop;
20     /* verilator lint_on UNUSEDSIGNAL */
21
22     logic tx_start;
23     logic [7:0] tx_data_in;
24     logic tx_busy;
25
26     typedef enum {
27         IO_READY_TO_RECEIVE,
28         IO_BUFFERING_DATA,
29         IO_INIT_TRANSMISSION,
30         IO_STARTED_TRANSMISSION
31     } state_t;
32
33     state_t state = IO_READY_TO_RECEIVE, next_state;
```

The comments `/* verilator lint_off UNUSEDSIGNAL */` and `lint_on` tell Verilator not to warn that `rx_idle` and `rx_eop` are never read. The receiver provides them, and we do not need them.

`typedef enum` **names the states.** Lines 26 to 31 define a new type `state_t` whose values

have names: `IO_READY_TO_RECEIVE`, `IO_BUFFERING_DATA`, and so on. The names stand for the numbers 0, 1, 2, 3, in this order. Line 33 declares two signals of this type, `state` with the power-up value `IO_READY_TO_RECEIVE` and `next_state`. As with `logic`, the type says nothing about the hardware; that `state` becomes a register follows from the `always_ff` block that assigns it. What the `enum` gives us is readability: we write the name of a state instead of a number, and a simulator shows the names in the waveform.

How many bits does `state` have? Without further information, an `enum` is based on the type `int`, which has 32 bits. sv2v writes `reg [31:0] state`. yosys finds that the upper 30 bits are always 0 and removes their flip-flops, but we will see in Section 7.7 that the 32 bits still matter.

A state machine

A *state machine* is a register, the *state*, together with combinational logic that computes the next state from the current state and the inputs. We have seen the smallest example in Chapter 3. Here the description is split into three blocks:

06_io_mirror/test.sv

```
35      //
36      // how states are connected
37      //
38      always_comb begin
39          case (state)
40              IO_READY_TO_RECEIVE:
41                  next_state = IO_BUFFERING_DATA;
42              IO_BUFFERING_DATA:
43                  next_state = IO_INIT_TRANSMISSION;
44              IO_INIT_TRANSMISSION:
45                  next_state = IO_STARTED_TRANSMISSION;
46              IO_STARTED_TRANSMISSION:
47                  next_state = IO_READY_TO_RECEIVE;
48              default:
49                  next_state = IO_READY_TO_RECEIVE;
50          endcase
51      end
52
53      //
54      // rules for state transitions
55      //
56      always_ff @ (posedge CLK) begin
57          case (state)
58              IO_READY_TO_RECEIVE:
59                  if (rx_ready) begin
60                      state <= next_state;
61                  end
62              IO_BUFFERING_DATA:
63                  if (!tx_busy) begin
64                      state <= next_state;
65                  end
66              IO_INIT_TRANSMISSION:
67                  state <= next_state;
68              IO_STARTED_TRANSMISSION:
69                  state <= next_state;
70          endcase
71      end
```

The `always_comb` block says which state follows which: a combinational circuit from `state` to `next_state`, here simply the successor in a cycle of four. The `always_ff` block is the state register. In some states it takes over `next_state` only under a condition, and if the condition is false it keeps its value. As a circuit, this is a register with enable, where the enable is 1 in `IO_READY_TO_RECEIVE` if `rx_ready` is 1, in `IO_BUFFERING_DATA` if `tx_busy` is 0, and in the other two states always. Figure 7.2 shows the result as a state diagram.

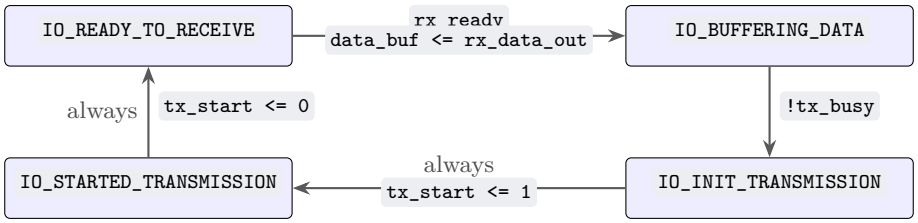


Figure 7.2: The state machine of `06_io_mirror/test.sv`. An arrow is taken at a rising edge of `CLK` if its condition holds; otherwise the state stays. The assignments below the arrows are made at the same edge.

The third block describes what happens in each state:

06_io_mirror/test.sv

```
73      //
74      // action in each state
75      //
76      logic [7:0] data_buf;
77
78      always_ff @ (posedge CLK) begin
79          case (state)
80              IO_READY_TO_RECEIVE:
81                  if (rx_ready) begin
82                      data_buf <= rx_data_out;
83                  end
84              IO_BUFFERING_DATA:
85                  ;
86              IO_INIT_TRANSMISSION:
87                  begin
88                      tx_start <= 1;
89                      tx_data_in <= data_buf;
90                  end
91              IO_STARTED_TRANSMISSION:
92                  tx_start <= 0;
93          endcase
94      end
```

It describes three registers with enable: `data_buf` loads the received byte when `rx_ready` is 1 in the first state; `tx_data_in` and `tx_start` change in the last two states. A state that is not mentioned, or the empty statement `;`, means: the register keeps its value.

Together: the receiver signals a byte with `rx_ready`, which is 1 for exactly one clock cycle. At that edge, the byte goes into `data_buf`, and the machine moves to `IO_BUFFERING_DATA`. There it waits until the transmitter is free. In `IO_INIT_TRANSMISSION` it hands the byte to the transmitter and sets `tx_start`, which is 1 during the one cycle in `IO_STARTED_TRANSMISSION` and is cleared at the edge back to the first state. So the transmitter sees a start pulse of one cycle. The display shows `data_buf`, the last byte received.

Exercise 7.1. Change one line of `test.sv` so that the board answers each character with the next one in the code table: `a` with `b`, `A` with `B`.

7.3 Parameters

The last part of `test.sv` creates the receiver and the transmitter:

06_io_mirror/test.sv

```
96      localparam CLK_FREQ = 12_000_000;
97      localparam BAUD = 9_600;
98
99      uart_rx #(
100         .clk_freq(CLK_FREQ),
101         .baud(BAUD)
102     ) uart_rx0 (
103         .clk(CLK),
104         .rx(RX),
105         .rx_ready(rx_ready),
106         .rx_data(rx_data_out),
107         .rx_idle(rx_idle),
108         .rx_eop(rx_eop)
109     );
110
111     uart_tx #(
112         .clk_freq(CLK_FREQ),
113         .baud(BAUD)
114     ) uart_tx0 (
115         .clk(CLK),
116         .tx_start(tx_start),
117         .tx_data(tx_data_in),
118         .tx(TX),
119         .tx_busy(tx_busy)
120     );
```

Between the module name and the instance name stands a list `#(…)`: values for the *parameters* of the module. In `uart_rx.sv` they are declared as

```
parameter clk_freq = 12000000;
parameter baud = 115200;
parameter oversampling = 8;
```

A `parameter` is a constant of the module, like a `localparam` (Chapter 4), with one difference: each instance may replace its value. The numbers after `=` are only defaults. Here `.baud(BAUD)` sets `baud` to 9600 for both instances, `oversampling` keeps its default. Parameters can also be given by position, in the order of their declaration; `uart_rx` does that when it creates its tick generator:

```
baud_tick_gen #(clk_freq, baud, oversampling) tickgen(.clk(clk), .enable(1'b1), .tick(os_tick));
```

A parameter is not a wire. It is a number that is fixed when the design is synthesised, and it changes the circuit itself: other values give other widths and other constants, so each set of parameter values describes a different circuit. yosys builds a separate module for each set, with a name derived from the values. The per-module statistics of `make report` (Appendix A) list them:

```
$ grep "^===" test_report_stat.txt
=== $paramod$2b706d08ef05df23db92f52fab0b950e0ab44c8f\baud_tick_gen ===
=== $paramod$b8fadbcf2318bd0d6773d4349c254f0af7dca0b4\baud_tick_gen ===
=== $paramod$b8fadbcf2318bd0d6773d4349c254f0af7dca0b4\uart_rx ===
=== $paramod$b8fadbcf2318bd0d6773d4349c254f0af7dca0b4\uart_tx ===
=== dev_hex ===
=== test ===
=== design hierarchy ===
```

There are two versions of `baud_tick_gen`, one with oversampling 8 for the receiver and one with oversampling 1 for the transmitter.

A note on style. `uart_rx.sv`, `uart_tx.sv` and `baud_tick_gen.sv` come from the iCEBreaker examples of Piotr Esden-Tempski and are written in the older Verilog style. `reg` and `wire` are the predecessors of `logic`, and `always @(posedge clk)` with `<=` is what we write as `always_ff`, as in `dev_hex` (Chapter 4). Ports without a type, such as `input clk`, are single wires. It describes the same kind of hardware; only the tools no longer check that an `always` block really describes flip-flops.

7.4 Baud Ticks from 12 MHz

Both the transmitter and the receiver need a signal that is 1 for one clock cycle, at the baud rate (transmitter) or at eight times the baud rate (receiver). For 9600 baud a counter to 1250 would do, but for eight times 9600 we would need $12\,000\,000/76\,800 = 156.25$ cycles, and for 115200 baud 104.17. `baud_tick_gen` solves this for all rates at once:

06_io_mirror/baud_tick_gen.sv

```
23 module baud_tick_gen(
24         input clk, enable,
25         output tick);
26 parameter clk_freq = 12000000;
27 parameter baud = 115200;
28 parameter oversampling = 1;
29
30 /* verilator lint_off WIDTH */
31 function integer log2_(input integer v);
32 begin
33     log2_=0; while(v >> log2_) log2_ = log2_ + 1;
34 end endfunction
35 /* verilator lint_on WIDTH */
36
37 localparam acc_width = log2_(clk_freq / baud) + 8; // +/- 2% max timing error over a byte
38
39 reg [acc_width:0] acc = 0;
40
41 localparam shiftlimiter = log2_((baud * oversampling) >> (31 - acc_width)); // this makes sure
       inc calculation doesn't overflow (verilog uses 32bit variables internally)
42 localparam inc = ((baud * oversampling << (acc_width - shiftlimiter)) + (clk_freq >> (
       shiftlimiter + 1))) / (clk_freq >> shiftlimiter); // Calculate accumulate increment
43 //initial $display("acc_width %d, shiftlimit %d, inc %d", acc_width, shiftlimiter, inc);
44
45 always @(posedge clk) if (enable) acc <= acc[acc_width-1:0] + inc[acc_width:0]; else acc <= inc[
       acc_width:0];
46 assign tick = acc[acc_width];
47
48 endmodule
```

The circuit is in lines 39, 45 and 46: a register `acc` of $w+1$ bits, where w is `acc_width`, and an adder. In each cycle, the low w bits of `acc` plus the constant `inc` are stored back; the top bit `acc[w]` receives the carry-out of this w-bit addition, and it is the output `tick`. Mathematically, with a_n the value of the low w bits after n cycles,

$$a_{n+1} = (a_n + inc) \bmod 2^w, \qquad tick_{n+1} = 1 \iff a_n + inc \geq 2^w.$$

So $a_n = n \cdot inc \bmod 2^w$ (starting from 0), and the number of ticks in the first n cycles is $\lfloor n \cdot inc / 2^w \rfloor$. At a clock of f Hz this gives on average

$$f \cdot \frac{inc}{2^w}$$

ticks per second. Since $inc < 2^w$, at most one tick comes per cycle, and the gap between two ticks is either $\lfloor 2^w/inc \rfloor$ or $\lceil 2^w/inc \rceil$ cycles. Such a circuit is called a *phase accumulator*, or a fractional divider: it divides the clock by the fraction $2^w/inc$.

The constants. To get $o \cdot b$ ticks per second for baud rate b and oversampling o, we need $inc \approx o \cdot b \cdot 2^w/f$, rounded. The larger w, the smaller the rounding error. Line 37 sets w to 8 more than the number of binary digits of f/b. Line 42 computes the rounded value of $o \cdot b \cdot 2^w/f$ in integer arithmetic: adding half the divisor, `clk_freq >> (shiftlimiter + 1)`, before the division turns truncation into rounding, and `shiftlimiter` shifts numerator and denominator so that the intermediate values stay below 2^{31}. If we uncomment the `$display` in line 43, the simulation prints the values for our two instances:

```
acc_width         19, shiftlimit          2, inc        419
acc_width         19, shiftlimit          5, inc       3355
```

For the transmitter, $w = 19$ and $inc = 419$, so

$$12\,000\,000 \cdot \frac{419}{2^{19}} \approx 9590.1 \text{ ticks per second,}$$

about 0.1 % slower than 9600; a bit lasts 1251 or 1252 cycles instead of 1250. For the receiver, $inc = 3355$ gives 76 789.9 instead of 76 800, an error of 0.013 %. Both are far inside the tolerance of a UART.

A function computed by the tools. `log2_` in lines 31 to 34 is a `function`: it takes an integer and returns one. Here it is only used in `localparam`s, so it is evaluated by the tools, during synthesis, and it describes *no* circuit at all. Its `while` loop is a loop in yosys, not in hardware. (A function applied to signals would describe combinational logic; we will not need that.) Despite its name, `log2_(v)` is not $\log_2 v$: it counts the binary digits of v, that is, $\lfloor \log_2 v \rfloor + 1$ for $v \geq 1$. For example, $f/b = 1250$ has 11 binary digits, so $w = 19$.

The enable. If `enable` is 0, line 45 sets `acc` to `inc`. The transmitter connects `enable` to `tx_busy`, so its tick generator starts afresh with every frame, and the first tick comes one bit time after the transmission starts. The receiver's tick generator always runs.

Exercise 7.2. With the default parameters (12 MHz, 115200 baud, no oversampling), compute `acc_width`, `shiftlimiter` and `inc`, and the actual tick rate. How large is the relative error?

7.5 The Transmitter

06_io_mirror/uart_tx.sv

```
parameter clk_freq = 12000000;
parameter baud = 115200;

wire bit_tick;
baud_tick_gen #(clk_freq, baud) tickgen(.clk(clk), .enable(tx_busy), .tick(bit_tick));

localparam
        IDLE      = 4'b0000, // tx = high
        BIT_START = 4'b0100, // tx = low
        BIT0      = 4'b1000, // tx = data bit 0
        BIT1      = 4'b1001, // tx = data bit 1
        BIT2      = 4'b1010, // tx = data bit 2
        BIT3      = 4'b1011, // tx = data bit 3
        BIT4      = 4'b1100, // tx = data bit 4
        BIT5      = 4'b1101, // tx = data bit 5
        BIT6      = 4'b1110, // tx = data bit 6
        BIT7      = 4'b1111, // tx = data bit 7
        BIT_STOP1 = 4'b0010, // tx = high
        BIT_STOP2 = 4'b0011; // tx = high

reg [3:0] tx_state = IDLE;
wire tx_ready = (tx_state == IDLE);
assign tx_busy = ~tx_ready;
```

The states of the transmitter are not an `enum` but `localparam`s with codes chosen by hand. The choice is clever: the eight data states have bit 3 set and the number of the data bit in bits 2 to 0, the states in which the line is 1 (`IDLE`, `BIT_STOP1`, `BIT_STOP2`) are the codes below 4, and `BIT_START` is 4. This makes the output logic small:

06_io_mirror/uart_tx.sv

```
reg [7:0] tx_shift = 0;
always @(posedge clk)
begin
        if (tx_ready & tx_start)
                tx_shift <= tx_data;
        else
        if (tx_state[3] & bit_tick)
                tx_shift <= (tx_shift >> 1);

        case (tx_state)
                IDLE:      if(tx_start) tx_state <= BIT_START;
                BIT_START: if(bit_tick) tx_state <= BIT0;
                BIT0:      if(bit_tick) tx_state <= BIT1;
                BIT1:      if(bit_tick) tx_state <= BIT2;
                BIT2:      if(bit_tick) tx_state <= BIT3;
                BIT3:      if(bit_tick) tx_state <= BIT4;
                BIT4:      if(bit_tick) tx_state <= BIT5;
                BIT5:      if(bit_tick) tx_state <= BIT6;
                BIT6:      if(bit_tick) tx_state <= BIT7;
                BIT7:      if(bit_tick) tx_state <= BIT_STOP1;
                BIT_STOP1: if(bit_tick) tx_state <= BIT_STOP2;
                BIT_STOP2: if(bit_tick) tx_state <= IDLE;
```

```
73                 default:   if(bit_tick) tx_state <= IDLE;
74         endcase
75
76 end
77
78 //           high if state START, STOP1, STOP2
79 //           |               high if transmitting bits and bit is 1
80 //           |               |
81 //           V               V
82 assign tx = (tx_state < 4) | (tx_state[3] & tx_shift[0]);
```

`tx_shift` is a *shift register*: when the transmitter is idle and `tx_start` is 1, it loads the byte; in each data state it shifts right by one at every `bit_tick`. So `tx_shift[0]` is always the bit to be sent, first d_0, then d_1, and so on: least significant bit first. The state moves on at every `bit_tick`. In line 82, `tx_state < 4` is 1 in idle and in the stop bits, `tx_state[3] & tx_shift[0]` is the data bit in the data states, and in `BIT_START` both are 0. The codes 1, 5, 6 and 7 are not used, and the `default` branch leads from them back to `IDLE`.

7.6 The Receiver

Receiving is harder: the receiver has to find the start of a frame and the middle of each bit by itself. It does that by looking at the line eight times per bit, at each `os_tick` of its tick generator. This is called *oversampling*.

06_io_mirror/uart_rx.sv

```
55 wire os_tick;
56 baud_tick_gen #(clk_freq, baud, oversampling) tickgen(.clk(clk), .enable(1'b1), .tick(os_tick));
57
58 // sync rx to our clock domain
59 reg [1:0] rx_sync = 2'b11;
60 always @(posedge clk) if(os_tick) rx_sync <= {rx_sync[0], rx};
61
62 // filter/debounce rx
63 reg [1:0] filter_cnt = 2'b11;
64 reg rx_bit = 1'b1;
65 always @(posedge clk)
66         if(os_tick) begin
67                 // increment/decrement filter_cnt
68                 // the filter_cnt counts up when rx_sync is high until it reaches 2'b11
69                 // the filter_cnt counts down when rx_sync is low until it reaches 2'b00
70                 // this filter will ignore glitches of length up to 2 x oversampling period
71                 // the filter does not provide hold hysteresis
72                 if((rx_sync[1] == 1'b1) && (filter_cnt != 2'b11)) filter_cnt <= filter_cnt + 1'
    d1;
73                 if((rx_sync[1] == 1'b0) && (filter_cnt != 2'b00)) filter_cnt <= filter_cnt - 1'
    d1;
74
75                 // set rx_bit output high when filter_cnt reaches max and low when min
76                 if(filter_cnt == 2'b11) rx_bit <= 1'b1;
77                 else
78                 if(filter_cnt == 2'b00) rx_bit <= 1'b0;
79                 // XXX: we probably should set rx_bit to some defined value when filter_cnt is
    not 11 nor 00.
80         end
```

Synchronising. `RX` changes whenever the computer sends, without any relation to our clock. A flip-flop whose input changes just at the clock edge may need unusually long to settle on 0 or 1. The two flip-flops `rx_sync` in series give it a whole sampling period to settle before the value is used. This is the standard way to bring an outside signal into a clocked design.

Filtering. `filter_cnt` is a 2-bit counter that counts up while the line is 1 and down while it is 0, stopping at 3 and 0. `rx_bit` only changes when the counter reaches 3 or 0, that is, when the line had the new value for three samples in a row. Short glitches are ignored.

06_io_mirror/uart_rx.sv

```
localparam l2o = log2(oversampling);
reg [l2o-2:0] os_cnt = 0;
/* verilator lint_off WIDTH */
always @(posedge clk)
    if(os_tick)
        os_cnt <= (rx_state == IDLE) ? 1'd0 : os_cnt + 1'd1;

wire sample_now = os_tick && (os_cnt == ((oversampling / 2) - 1));
/* verilator lint_on WIDTH */

// rx data finite state machine
always @(posedge clk)
        case(rx_state)
                IDLE:      if(~rx_bit) rx_state <= BIT_START;
                BIT_START: if(sample_now) rx_state <= BIT0;
                BIT0:      if(sample_now) rx_state <= BIT1;
                BIT1:      if(sample_now) rx_state <= BIT2;
                BIT2:      if(sample_now) rx_state <= BIT3;
                BIT3:      if(sample_now) rx_state <= BIT4;
                BIT4:      if(sample_now) rx_state <= BIT5;
                BIT5:      if(sample_now) rx_state <= BIT6;
                BIT6:      if(sample_now) rx_state <= BIT7;
                BIT7:      if(sample_now) rx_state <= BIT_STOP;
                BIT_STOP:  if(sample_now) rx_state <= IDLE;
                default:   rx_state <= IDLE;
        endcase

// accumulate rx data into the rx_data shift register
// rx_state[3] -> this bit is only high when the state machine is in state BIT0:7
always @(posedge clk) begin
        if(sample_now && rx_state[3]) begin
                rx_data <= {rx_bit, rx_data[7:1]};
        end
end
```

Finding the middle of a bit. `os_cnt` has `l2o-1` bits; `log2(8)` counts the binary digits of 8, which gives 4, so `os_cnt` has 3 bits and counts the oversampling ticks modulo 8. In `IDLE` it is held at 0: `c ? a : b` is the *conditional operator*, a 2:1 multiplexer that selects `a` if `c` is 1 and `b` otherwise. When the filtered line falls to 0, the start bit has begun, and the state machine goes to `BIT_START`. Now `os_cnt` counts, and `sample_now` is 1 at the tick on which `os_cnt` is 3: about half a bit after the falling edge, and from then on every 8 ticks, one bit later. Each `sample_now` advances the state, and in the eight data states (bit 3 of the state set) the filtered bit is shifted into `rx_data` from the left. After eight shifts, the first bit received, d_0, has arrived at position 0. The delay of the synchroniser and the filter does not matter: it delays the falling edge and all data bits alike.

06_io_mirror/uart_rx.sv

```
128 always @(posedge clk)
129 begin
130         rx_ready <= (sample_now && (rx_state == BIT_STOP) && rx_bit); // high for one cycle only
131         //rx_error <= (sample_now && (rx_state == BIT_STOP) && ~rx_bit);
132 end
```

At the sample point of the stop bit, `rx_ready` becomes 1 for one cycle if the stop bit is 1; a frame without a valid stop bit is dropped. Since `sample_now` is 1 for one cycle, so is `rx_ready`, as the comment says. The rest of `uart_rx.sv` measures pauses between bytes for `rx_idle` and `rx_eop`, which we do not use.

7.7 A Latch from an Incomplete case

An earlier version of `test.sv` had no `default` branch in the `always_comb` block of the state machine. With this version, yosys stops:

```
$ make test.json
sv2v -v --define=SYNTHESIS --write=test.v baud_tick_gen.sv dev_hex.sv test.sv uart_rx.sv uart_tx.sv
yosys -ql test.log -p 'synth_ice40 -abc9 -dffe_min_ce_use 4 -device u -dsp -top test -json test.json' test.v
ERROR: Latch inferred for signal `\test.\next_state [1:0]' from process `\test.$proc$test.v:241$380':
    $auto$proc_dlatch.cc:559:proc_dlatch$652
make: *** [Makefile:40: test.json] Error 1
```

What is a latch doing here? An `always_comb` block must say, for every possible value of its inputs, what its outputs are. If there is an input value for which `next_state` is not assigned, then `next_state` must keep its previous value for that input. A circuit without clock that keeps a value is a *latch*: level-sensitive storage, the D latch of the course. On an FPGA it has to be built from LUTs with feedback, its timing is hard to analyse, and it is almost never what we wanted. `synth_ice40` therefore aborts by default (its option `-latches` has the default `error`); with `-latches warn` it would only warn and build the latch from LUTs.

But the `case` lists all four states. Why is it incomplete? Because `state` has 32 bits, as we saw: the `case` covers four of its 2^{32} values. That only these four occur is a property of our state machine, not of the circuit that computes `next_state`, and yosys builds that circuit for all inputs. (Verilator, which knows that an `enum` variable holds one of its values, does not complain. Simulator and synthesis tool look at the same text differently.) The `default` branch makes the assignment complete.

Pitfall. In `always_comb`, assign every output in every case, for example with a `default` branch, or with a value assigned before the `case`, as `dev_hex` does with `digit = 0;`. In `always_ff`, a missing assignment is no problem: a flip-flop that keeps its value is a flip-flop with enable. That is why the `case` blocks in `always_ff` above need no `default`.

Exercise 7.3. Remove the `default` branch again, and give the `enum` a base type of two bits: write `typedef enum logic [1:0]` instead of `typedef enum` in line 26. Does yosys still find a latch? Why?

7.8 What yosys and nextpnr Make of It

```
2.60. Printing statistics.

=== test ===

       +----------Local Count, excluding submodules.
       |
      168 wires
      328 wire bits
      168 public wires
      328 public wire bits
       11 ports
       11 port bits
        5 cells
        5   $scopeinfo
      243 submodules
       40   SB_CARRY
       33   SB_DFF
       33   SB_DFFE
       12   SB_DFFESR
        7   SB_DFFESS
       15   SB_DFFSR
        5   SB_DFFSS
       98   SB_LUT4
```

Most of the 40 `SB_CARRY` belong to the two accumulators of the tick generators: 18 each, for adders of 19 bits. The flip-flops come in more kinds than before; the letters S and R stand for set and reset inputs, which yosys uses for assignments of constants, such as `acc <= inc` when `enable` is 0. nextpnr places it all on 133 logic cells:

```
Info: Device utilisation:
Info:            ICESTORM_LC:     133/   5280     2%
Info:                   SB_IO:      11/     39    28%
Info:                   SB_GB:       1/      8    12%
Info: Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 66.13 MHz (PASS at 12.00 MHz)
```

7.9 Talking to the Board

After `make prog`, the board waits for characters. On the computer, the USB chip appears as a serial port, and any terminal program can use it. The repository contains a script for C-Kermit:

06_io_mirror/kermit-serial

```
#!/usr/bin/env kermit +
;
; Based on: https://gist.github.com/ajc/789214
;
; copy this to ~/bin/kermit-serial and make executable!

set carrier-watch off
set line /dev/tty.usbserial-ibj0a3QC1
set terminal cr-display crlf
set terminal lf-display crlf
set speed 9600
set stop-bits 2

connect
```

`set line` names the serial port. The USB chip has two channels, one for programming the flash and one for the serial line. On a Mac the serial line appears as `/dev/tty.usbserial-`

followed by an identifier of the chip; the name in the script is that of the author's board. Look with `ls /dev/tty.usbserial-*` which name yours has, and change the line. (On Linux the port is usually `/dev/ttyUSB1`.) `set speed 9600` is our baud rate, `set stop-bits 2` matches the two stop bits of our transmitter. `carrier-watch off` tells Kermit not to wait for modem signals, which our line does not have, and the two `set terminal` lines make the Return key, which sends a carriage return (code `0D`), also start a new line on the screen. `connect` starts the terminal; `Ctrl-\` followed by `c` returns to the Kermit prompt, where `exit` ends it.

Copy the script to a directory in your `PATH`, make it executable and run it. Each key you press is sent to the FPGA, comes back and appears on the screen, and the display shows its code: `41` for `A`, `61` for `a`. Kermit itself does not show what you type, so if nothing appears, the FPGA did not answer.

The design has a buffer of only one byte. While it waits in `IO_BUFFERING_DATA` for the transmitter, it ignores `rx_ready`, and a byte that arrives in this time is lost. When you type, this does not happen; when a program sends many bytes in a row, it can. The next step puts FIFOs between the UART and the rest of the design.

7.10 Simulation

To see the UART at work, the testbench has to play the computer: it sends a frame on `RX` and watches what comes back on `TX`. The level of `RX` in each clock cycle is a function of the cycle number:

06_io_mirror/tb_test.cpp

```
constexpr std::uint64_t BIT = 1250;     // clock cycles per bit: 12 MHz / 9600
constexpr std::uint64_t T0 = 1000;      // the frame on RX starts here
constexpr std::uint8_t DATA = 'A';      // the byte sent on RX

// Level of the RX line at clock cycle c: a frame with the byte DATA
// (start bit, 8 data bits LSB first, stop bit) starting at cycle T0.
static int
rx_level(std::uint64_t c)
{
    if (c < T0) {
        return 1;                       // idle
    }
    std::uint64_t k = (c - T0) / BIT;
    if (k == 0) {
        return 0;                       // start bit
    }
    if (k <= 8) {
        return DATA >> (k - 1) & 1;     // data bit k - 1
    }
    return 1;                           // stop bit, then idle
}
```

The main loop drives the clock itself, one cycle per iteration, prints every change of `TX` with its cycle, and samples the echo in the middle of each data bit:

06_io_mirror/tb_test.cpp

```
    for (std::uint64_t c = 0; c < 30000; ++c) {
        dut->RX = rx_level(c);
        dut->CLK = 1;
        dut->eval();
        m_trace->dump(sim_time++);
        dut->CLK = 0;
        dut->eval();
        m_trace->dump(sim_time++);

        if (dut->TX != tx) {
            tx = dut->TX;
            std::printf("cycle %5llu: TX = %d\n", (unsigned long long)c, tx);
            if (tx == 0 && tx_start == 0) {
                tx_start = c;
                echo = 0;
            }
        }
        // sample the data bits of the echo in the middle of each bit
        if (tx_start && c > tx_start) {
            std::uint64_t t = c - tx_start;
            std::uint64_t k = t / BIT;
            if (t % BIT == BIT / 2 && k >= 1 && k <= 8) {
                echo |= dut->TX << (k - 1);
            }
        }
    }

    std::printf("sent %02x, received %02x: %s\n", DATA, echo,
                echo == DATA ? "PASSED" : "FAILED");
```

The output of `make sim`:

```
$ make sim
...

### SIMULATING ###
cycle 13757: TX = 0
cycle 15009: TX = 1
cycle 16260: TX = 0
cycle 22516: TX = 1
cycle 23768: TX = 0
cycle 25019: TX = 1
sent 41, received 41: PASSED
```

The middle of the stop bit of the frame on `RX` is at cycle $1000 + 9.5 \cdot 1250 = 12\,875$. The receiver reports the byte somewhat later, since the synchroniser and the filter delay the line by a few oversampling ticks, and the echo starts at cycle 13 757 with the start bit. Then, with $41_{16} = 01000001_2$ sent LSB first: $d_0 = 1$ from 15 009, d_1 to $d_5 = 0$ from 16 260, $d_6 = 1$ from 22 516, $d_7 = 0$ from 23 768, and from 25 019 on the line stays 1 for the stop bits and idle. The bits last 1251 or 1252 cycles, as computed in Section 7.4; five bits from 16 260 to 22 516 are 6256 cycles. The testbench decodes the echo and compares it with the byte it sent; if they differ, it prints `FAILED` and `make sim` ends with an error.

Exercise 7.4. Change `rx_level` so that the testbench sends the 26 letters `A` to `Z` back to back, each frame directly after the stop bit of the one before, and print the bytes that come

back. Which letters are missing, and why? (Hint: compare the length of a frame on `RX` and on `TX`.)

7.11 What Is Next

We can now exchange bytes with the computer, one at a time. Step 07 turns this into an I/O interface for the rest of the design: FIFOs that buffer bytes in both directions, built from block RAM, and the first SystemVerilog `interface`s to connect the parts.

7.12 Solutions

Exercise 7.1. In `IO_INIT_TRANSMISSION`, send the next code instead of the received one:

```
tx_data_in <= data_buf + 1;
```

The display still shows the received byte, since it shows `data_buf`. (For `z`, the answer is `{`, code `7B`.)

Exercise 7.2. With $f = 12\,000\,000$, $b = 115\,200$, $o = 1$: $f/b = 104$ (integer division), which has 7 binary digits, so `acc_width` $= 15$. Then `shiftlimiter` = `log2_(115200 >> 16)` = `log2_(1)` $= 1$, and

$$inc = \left\lfloor \frac{115\,200 \cdot 2^{14} + 3\,000\,000}{6\,000\,000} \right\rfloor = \lfloor 315.07 \rfloor = 315.$$

The simulation with `$display` agrees (`acc_width 15, shiftlimit 1, inc 315`). The tick rate is $12\,000\,000 \cdot 315/2^{15} \approx 115\,356$ per second, 0.14 % too fast. Without rounding, $115\,200 \cdot 2^{15}/12\,000\,000 = 314.57$; the error comes from rounding to an integer, and a larger w would make it smaller.

Exercise 7.3. With `logic [1:0]` as base type, `state` has 2 bits, and the four branches cover all four values. The `case` is complete, `next_state` is assigned for every input, and there is no latch. yosys synthesises the design without error, and the statistics are the same as with the `default` branch.

Exercise 7.4. In `rx_level`, the frame number is $f = \lfloor (c - \texttt{T0})/(10 \cdot \texttt{BIT}) \rfloor$ and the bit within the frame is $k = \lfloor (c - \texttt{T0})/\texttt{BIT} \rfloor \bmod 10$; for $f < 26$ the function returns the start bit, a data bit of letter number f, or the stop bit, as before, and 1 after the last frame. The echo then misses `L` and `W`: 24 letters come back. A frame on `RX` takes 10 bit times, 12 500 cycles, but our transmitter sends two stop bits, 11 bit times or about 13 768 cycles. So every character, the echo falls behind by about 1268 cycles. The state machine accepts a byte only in `IO_READY_TO_RECEIVE`; with a byte waiting in `IO_BUFFERING_DATA` for the transmitter, it ignores `rx_ready`. After ten characters the backlog is a whole frame: the eleventh letter, `L`, arrives while `K` is still waiting, and is lost. Then the backlog starts again from a smaller value, and eleven letters later `W` is lost. The FIFOs of step 07 make the buffer much longer, but the rate problem remains: in the long run a sender that is faster than the transmitter fills any buffer.

Chapter 8

Step 07: An I/O Device with Buffers

In step 06 every character that arrived on the serial line was sent back at once, and then it was gone. A computer needs more: characters that arrive while the program is busy must wait somewhere until the program asks for them, and characters that the program writes must wait until the slow serial line can take them. So we build an I/O device with an input buffer and an output buffer. A buffer of this kind is a *FIFO*, a queue, and its content lives in a memory block of the FPGA. On the way we meet three new pieces of SystemVerilog: `interface` and `modport`, which bundle wires, and `package`, which bundles declarations. Everything is in the directory `07_io_interface`; the UART modules come from step 06 (Chapter 7), the display from step 03.

8.1 What the Design Does

Figure 8.1 shows the parts. The module `dev_io` contains the UART receiver and transmitter and two FIFOs. Every character that arrives on `RX` goes two ways: into the input buffer, and into the output buffer, from where it is sent back on `TX`, so that you see what you type. The other side of `dev_io`, the one a computer would use, has two operations:

getc reads the oldest character of the input buffer and removes it ("pop" it);

putc appends a character to the output buffer ("push" it).

In this step there is no computer yet. The top module `test` plays its role with buttons: the display shows the oldest character in the input buffer as a hexadecimal number, `BTN1` pops it, and `BTN2` pushes the byte that is set at the inputs `PMOD_1B` into the output buffer. `LED1` is on while the input buffer is empty, `LED2` when it is full, and `LED3` while `BTN1` is pressed although there is nothing to pop.

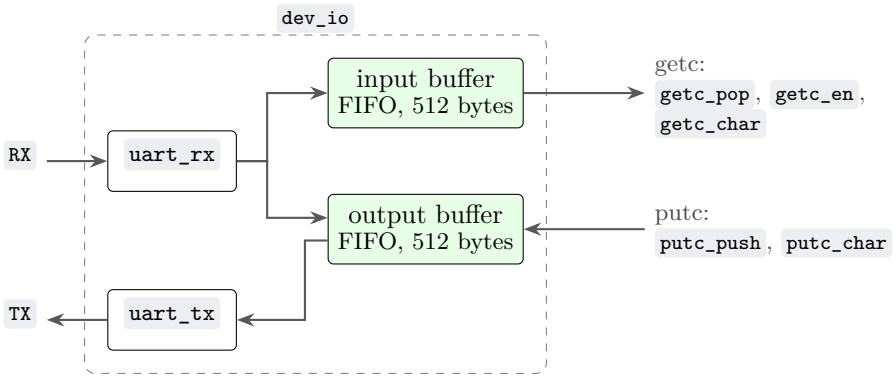


Figure 8.1: The I/O device of step 07. The signals on the right are the side a computer will use; in this step, buttons and the display take its place.

The new top module is short, because the work is done in `dev_io`:

07_io_interface/test.sv

```
    localparam CLK_FREQ = 12_000_000;
    localparam BAUD = 9_600;

    logic rst = 0;

    if_io if_io();

    dev_io #(
        .CLK_FREQ(CLK_FREQ),
        .BAUD(BAUD)
    ) io0(
        .clk(CLK),
        .rst(rst),
        .rx(RX),
        .tx(TX),
        .io(if_io.server)
    );
```

Two things are new here: the line `if_io if_io();` and the port connection `.io(if_io.server)`. They are the subject of the next section.

The ports of the display and of the input byte are vectors this time, `PMOD_1A` and `PMOD_1B`, each of 8 bits. The pin file gives every bit its own pin, with the index in brackets:

```
set_io -nowarn PMOD_1A[0]   4
set_io -nowarn PMOD_1A[1]   2
...
set_io -nowarn PMOD_1B[7]  28
```

These are the same pins as `A0` to `A7` and `B0` to `B7`: the display on PMOD 1A and the DIP switch on PMOD 1B, now addressed as two buses.

8.2 Interfaces: Named Bundles of Wires

Between `test` and `dev_io` there are seven signals: three go into `dev_io`, four come out. We could list them as seven ports of `dev_io` and connect them one by one, and in step 08 we would do the same again in the loader, and in step 09 in the processor. Instead we give the bundle a name:

07_io_interface/if_io.sv

```
`include "pkg_ram.sv"
import pkg_ram::RAM_BYTE_SIZE;

interface if_io;
    logic getc_pop;
    logic putc_push;
    logic [RAM_BYTE_SIZE-1:0] putc_char;

    logic inbuf_full;
    logic getc_en;
    logic [RAM_BYTE_SIZE-1:0] getc_char;
    /* verilator lint_off UNUSEDSIGNAL */
    logic putc_push_done;
    /* verilator lint_on UNUSEDSIGNAL */
```

```
    modport server(
        input getc_pop,
        input putc_push,
        input putc_char,
        output inbuf_full,
        output getc_en,
        output getc_char,
        output putc_push_done
    );

    modport client(
        output getc_pop,
        output putc_push,
        output putc_char,
        input inbuf_full,
        input getc_en,
        input getc_char,
        input putc_push_done
    );

endinterface
```

(The first two lines import a constant from a package; we come to them in the next section. `RAM_BYTE_SIZE` is 8.)

An `interface` is a named bundle of wires. Between `interface if_io;` and `endinterface` we declare signals, exactly as in a module. The line `if_io if_io();` in `test` creates one *instance* of the bundle, called `if_io` as well: seven wires (or groups of wires, `getc_char` has eight) that exist once in the circuit. Think of a ribbon cable with seven named strands. An interface contains no logic of its own here; it is wiring.

A `modport` gives the direction of each wire, seen from one side. A ribbon cable has two ends. At the end that plugs into `dev_io`, `getc_pop` is an input and `getc_char` an output; at the other end it is the other way round. The modport `server` lists the directions as `dev_io` sees them, the modport `client` as the user of `dev_io` sees them (Figure 8.2). A module that takes the bundle as a port names the modport:

```
module dev_io #( ... ) (
    ...
    if_io.server io
);
```

Inside `dev_io`, the strands of the bundle are called `io.getc_pop`, `io.getc_char`, and so on, and the modport tells the tools which of them `dev_io` may drive. When we instantiate `dev_io`, we plug the `server` end of our bundle into this port with `.io(if_io.server)`. The top module `test` uses the signals of the bundle directly, as `if_io.getc_pop` and so on; it takes the role of the client.

What the tools make of it. An interface is a convenience for us; in the circuit there are only wires. sv2v translates interfaces into plain Verilog by taking them apart. Here is an excerpt of `test.v`:

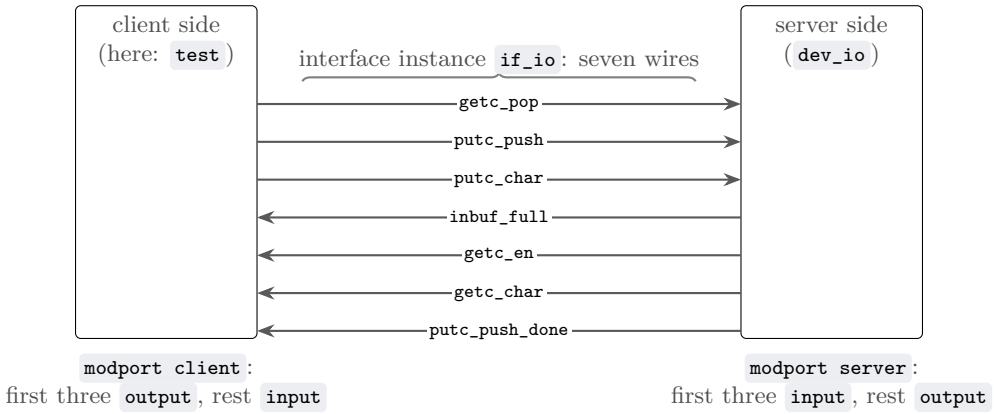


Figure 8.2: The interface `if_io` as a bundle of wires between two modules. The two modports describe the same wires, with the directions as seen from either end.

```
// removed module with interface ports: fifo
// removed interface: if_fifo
// removed package "pkg_ram"
// removed ["import pkg_ram::RAM_BYTE_SIZE;"]
// removed interface: if_io
        // expanded interface instance: if_io
        generate
                if (1) begin : if_io
                        reg getc_pop;
                        reg putc_push;
                        // removed localparam type pkg_ram_data_size_t
                        wire [7:0] putc_char;
                        wire inbuf_full;
                        wire getc_en;
                        wire [7:0] getc_char;
                        reg putc_push_done;
                end
        endgenerate
```

The interface instance became a group of ordinary signals inside `test`. And every module that has an interface port, here `dev_io` and `fifo`, was removed as a module: sv2v copied its contents into the module that instantiates it, with the interface signals connected directly. The circuit is the same, but the hierarchy is gone. This is why `make report` cannot list `dev_io` and `fifo` separately (Section 8.8); the report says so in its header.

8.3 Packages and Include Files

Several modules need the same constants, for example the width of a byte or, from step 08 on, the number of address bits of the memory. We collect them in a *package*:

07_io_interface/pkg_ram.sv

```
`ifndef PKG_RAM
`define PKG_RAM

package pkg_ram;

typedef enum {
    SPRAM_ADDRW = 14, // address space for words in a single spram block
    SPRAM_WIDTH = 16, // 1 word = 2 bytes
    RAM_ADDRW = 17    // address space for quad words stored in 4 blocks
} addr_t;
```

```
typedef enum {
    RAM_BYTE,
    RAM_WORD,
    RAM_LONG,
    RAM_QUAD
} data_type_t;

typedef enum {
    RAM_NIBBLE_SIZE = 4,
    RAM_BYTE_SIZE = 8,
    RAM_WORD_SIZE = 16,
    RAM_LONG_SIZE = 32,
    RAM_QUAD_SIZE = 64
} data_size_t;

typedef enum {
    RAM_NOP,
    RAM_FETCH,
    RAM_STORE
} op_t;

endpackage // pkg_ram

`endif // PKG_RAM
```

A package is a named collection of declarations. It describes no hardware, it only gives names to types and constants. Inside it, the names are declared with `typedef enum`, which we know from step 06: there it named the states of a state machine. Here the enumerations are used with explicit values as a list of named integer constants: `RAM_BYTE_SIZE` is 8, `RAM_QUAD_SIZE` is 64, `RAM_ADDRW` is 17, and so on. The enumeration `data_type_t` and `op_t` will be real types in step 08.

There are two ways to use a name from a package. The full name `pkg_ram::RAM_BYTE_SIZE` works everywhere. Or we *import* it once, `import pkg_ram::RAM_BYTE_SIZE;`, and then write just `RAM_BYTE_SIZE`. In `if_io.sv` the import stands outside any module or interface; for the tools we use, it then applies to everything that follows in the same run.

`` `include `` copies text. A package must be known before it is used. The Makefile passes all `*.sv` files to sv2v and Verilator in alphabetical order, so `if_io.sv` comes before `pkg_ram.sv`. Therefore `if_io.sv` starts with `` `include "pkg_ram.sv" ``. Like `#include` in C, this is done by a preprocessor before anything else: the line is replaced by the contents of the file. Now the package is defined before `if_io` uses it.

But it would be defined twice: once through the include, and once more when the tools reach `pkg_ram.sv` in the file list (and in step 08 many files include it). A second definition of the same package is an error. The first and the last two lines of `pkg_ram.sv` prevent this; they are an *include guard*. `` `ifndef PKG_RAM `` means "only if the macro `PKG_RAM` is not defined, keep the text up to `` `endif ``". The first copy defines `PKG_RAM`, so every later copy is dropped by the preprocessor. Just as with interfaces, sv2v resolves all of this; `test.v` contains no package at all, only the constants substituted where they are used.

8.4 A FIFO

A FIFO (first in, first out) is a queue: we *push* elements at the back and *pop* them at the front, and they come out in the order in which they went in. In hardware we build it from a memory with N entries and two registers, the *read pointer* r and the *write pointer* w. Push stores the new element at address w and increments w; pop takes the element at address r and increments r. When a pointer reaches the end of the memory it starts again at address 0, so the memory is used as a ring (Figure 8.3).

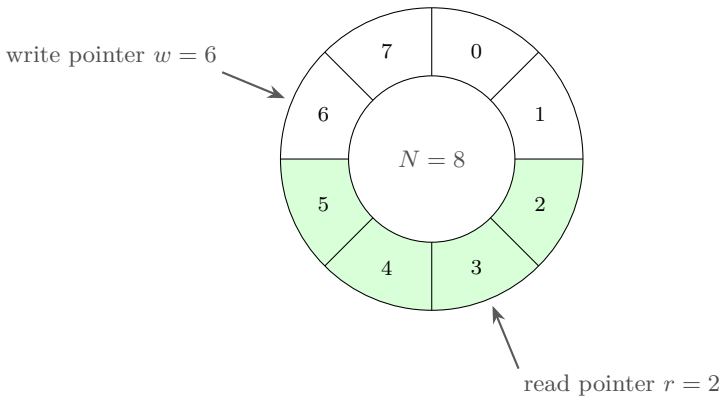


Figure 8.3: A FIFO with $N = 8$ entries as a ring. Entries 2 to 5 (shaded) hold the queue; the next pop reads entry 2, the next push writes entry 6.

Counting with pointers, modulo something

How many elements are in the queue? If neither pointer had wrapped around, it would be $w - r$. With wrapping, the pointers are numbers modulo N, and the count is $(w - r) \bmod N$. Here is the catch: the count can be anything from 0 to N, which are $N + 1$ values, but $(w - r) \bmod N$ has only N. An empty FIFO has $w = r$; but after N pushes into an empty FIFO, w has gone once around the ring, and again $w = r$. With pointers modulo N we cannot tell empty from full.

The usual solution, and the one used here, is to give each pointer one bit more than the address needs. With $N = 2^k$ entries, an address has k bits, and the pointers have $k + 1$ bits, so they count modulo $2N$. The memory address is the pointer modulo N, which is simply its lower k bits. The number of elements is

$$n = (w - r) \bmod 2N,$$

and as long as $0 \le n \le N$ (we never push into a full FIFO and never pop from an empty one), this is the true count: empty and full are now different values of $w - r$. In terms of the bits:

- empty, $n = 0$: $w = r$, all $k + 1$ bits equal;
- full, $n = N$: $w - r \equiv N \pmod{2N}$, that is, the lower k bits are equal (same address) and the top bits differ (one pointer is a lap ahead).

Our FIFO has $N = 512$ entries of one byte, so $k = 9$ and the pointers have 10 bits. An example: $r = 1000$ and $w = 6$ means $n = (6 - 1000) \bmod 1024 = 30$. The read pointer is at address $1000 \bmod 512 = 488$, the write pointer at address 6: the queue occupies addresses 488 to 511 and 0 to 5.

The FIFO in SystemVerilog

The FIFO has its own interface:

07_io_interface/if_fifo.sv

```
interface if_fifo #(
    localparam WIDTH = 8                // each entry is one byte
);
    logic empty;
    logic full;
    logic pop_front;
    logic [WIDTH-1:0] data_out;
    logic push_back;
    logic [WIDTH-1:0] data_in;

    modport fifo(
        input pop_front,
        input push_back,
        input data_in,
        output empty,
        output full,
        output data_out
    );
endinterface
```

Like a module (Chapter 7), an interface can have a parameter list; here it fixes `WIDTH = 8`. The FIFO itself:

07_io_interface/fifo.sv

```
/*
 * FIFO
 *
 * Code written after reading the paper
 * http://www.sunburst-design.com/papers/CummingsSNUG2002SJ_FIFO1.pdf
 *
 * push_back == 1 --> data_in is stored in *next* cycle and an element
 *                    appended to the back in the *next* cycle
 * pop_front == 1 --> data_out is valid in *this* cycle and front element
 *                    removed in *this* cycle
 *
 * Side effect: If FIFO is not empty data_out is always valid and contains the
 *              front element.
 *
 * Requirements:
 * (1) "push_back <= 1" is illegal if full == 1  *or* push_back == 1
 * (2) "pop_front <= 1" is illegal if empty == 1 *or* pop_front == 1
 *
 * About (1): data_in has to stay valid until next cycle
 * About (2): pop_front removes an element in the current cycle. But it
 *            takes an extra cycle until data_out is updated with the
 *            new front element.
 */


module fifo #(
    localparam SIZE = 4096,             // size of one EBR (Embedded Block RAM)
    localparam WIDTH = 8,               // each entry is one byte
```

```
    localparam DEPTH = SIZE / WIDTH,    // number of addressable bytes ...
    localparam ADDRW = $clog2(DEPTH)    // Number of bits in an address
) (
    input logic clk,
    input logic rst,
    if_fifo.fifo fifo
);
    // read/write pointers
    logic [ADDRW:0] read_ptr = 0;
    logic [ADDRW:0] write_ptr = 0;

    always_ff @ (posedge clk) begin
        if (rst) begin
            read_ptr <= 0;
            write_ptr <= 0;
        end else begin
            case ({fifo.push_back, fifo.pop_front})
                2'b00: ;
                2'b10: write_ptr <= write_ptr + 1;
                2'b01: read_ptr <= read_ptr + 1;
                2'b11:
                    begin
                        write_ptr <= write_ptr + 1;
                        read_ptr <= read_ptr + 1;
                    end
            endcase
        end
    end

    always_comb begin
        fifo.full = read_ptr[ADDRW] !=  write_ptr[ADDRW]
                 && read_ptr[ADDRW-1:0] == write_ptr[ADDRW-1:0];
        fifo.empty = read_ptr == write_ptr;
    end

    dp_bram4096 #(WIDTH) ram_inst(
        .clk_in(clk),
        .en_in(fifo.push_back),
        .addr_in(write_ptr[ADDRW-1:0]),
        .data_in(fifo.data_in),
        .addr_out(read_ptr[ADDRW-1:0]),
        .data_out(fifo.data_out)
    );

endmodule
```

The parameters compute the sizes: one memory block of 4096 bits, entries of 8 bits, so `DEPTH` $= 512$ entries and `ADDRW` $= \lceil \log_2 512 \rceil = 9$ address bits. `$clog2` is evaluated by the tools, not by the circuit: it computes a constant. All four are `localparam`s, which cannot be changed from outside: the FIFO always fills exactly one memory block, and `dp_bram4096` below computes the same depth from the same width. The pointers are declared as `logic [ADDRW:0]`, that is with `ADDRW` $+ 1 = 10$ bits, exactly as in our analysis.

The `always_ff` block describes the two pointer registers and their next-state logic: each is a counter (register plus adder) whose increment is enabled by `push_back` or `pop_front`. The `case` over the concatenation `{fifo.push_back, fifo.pop_front}` lists all four combinations;

with both, both pointers advance and the count stays the same. Wrapping around costs nothing: a 10-bit counter computes modulo 2^{10} by itself, because the carry out of the top bit is dropped.

The `always_comb` block computes `full` and `empty` with the comparisons from above. `read_ptr[ADDRW]` is the top bit, `[ADDRW-1:0]` the address bits.

Finally, the memory `dp_bram4096` is instantiated with the write pointer's address bits as write address and the read pointer's as read address. Its write enable is `push_back` itself: at the clock edge at which the pointer advances, the byte is written into the memory.

The comment at the top states the rules for using the FIFO. The important one for us: `data_out` is the memory's output, and this output, as we see in a moment, is a register that is loaded at every clock edge from the address that the read pointer had *before* the edge. So after a pointer changes, it takes one more clock cycle until `data_out` shows the new front element.

8.5 Memory: Block RAM

A FIFO of 512 bytes needs 4096 bits of storage. As flip-flops, that would be 4096 of our 5280 logic cells, and the multiplexer that selects one of 512 bytes would need a few thousand LUTs more. This is what the memory blocks mentioned in Section 1.1 are for. The iCE40UP5K has 30 *block RAMs* (also called EBR, embedded block RAM) of 4 kbit each, called `SB_RAM40_4K` in the netlist. Each one can be configured as 256×16, 512×8, 1024×4 or 2048×2 bits, and it has two ports: one for writing and one for reading, each with its own address, both working at the clock edge.

We do not instantiate `SB_RAM40_4K` ourselves. Instead we describe a memory in a way that matches what the block can do, and yosys recognises it:

07_io_interface/dp_bram4096.sv

```
module dp_bram4096 #(
    parameter WIDTH = 8,
    localparam SIZE = 4096, // total bits
    localparam DEPTH = SIZE / WIDTH,
    localparam ADDRW = $clog2(DEPTH)
) (
    input logic clk_in,
    input logic en_in,
    input logic [ADDRW-1:0] addr_in,
    input logic [WIDTH-1:0] data_in,
    input logic [ADDRW-1:0] addr_out,
    output logic [WIDTH-1:0] data_out
);

    // no_rw_check: no extra logic for a read and a write of the same address
    // at the same edge (a FIFO does this only while it is empty)
    (* no_rw_check *)
    logic [WIDTH-1:0] mem [DEPTH];

    initial begin
        mem[0] = 255;
    end

    always @ (posedge clk_in) begin
        if (en_in) begin
            mem[addr_in] <= data_in;
```

```
        end
        data_out <= mem[addr_out];
    end

endmodule
```

An array of vectors is a memory. `logic [WIDTH-1:0] mem [DEPTH];` declares `DEPTH` = 512 elements `mem[0]` to `mem[511]`, each a vector of 8 bits. The dimension after the name, `[DEPTH]`, is short for `[0:DEPTH-1]`. `mem[addr_in]` selects an element by an address that is a signal, so this is a memory with an address decoder, not a fixed wire.

The pattern that becomes a block RAM. In the `always` block,

- one element is written at the clock edge, at one address, if an enable is 1: this is the write port;
- `data_out` is a register that takes, at the clock edge, the element at another address: this is the read port. The read is *synchronous*: the data appears after the clock edge, not combinationally.

This is exactly the behaviour of `SB_RAM40_4K`, whose read port has an output register. (The block uses plain `always` where we would write `always_ff`; it describes the same flip-flops.) yosys reports in `test.log` what it found:

```
2.35. Executing MEMORY_LIBMAP pass (mapping memories to cells).
using FF mapping for memory test.$flatten\dev_hex0.$auto$proc_rom.cc:155:do_switch$540
mapping memory test.io0.output_buffer.ram_inst.mem via $__ICE40_RAM4K_
mapping memory test.io0.input_buffer.ram_inst.mem via $__ICE40_RAM4K_
```

Both FIFO memories became block RAMs (`$__ICE40_RAM4K_` is yosys's name for `SB_RAM40_4K` at this stage). The ROM of 16 entries that `dev_hex` contains, the table of digit patterns, is so small that yosys builds it from logic cells instead. The statistics at the end of the log:

```
2.60. Printing statistics.

=== test ===

        +----------Local Count, excluding submodules.
        |
       11 cells
        9   $scopeinfo
        2   SB_RAM40_4K
      538 submodules
      102   SB_CARRY
       44   SB_DFF
       89   SB_DFFE
       12   SB_DFFESR
        7   SB_DFFESS
       53   SB_DFFSR
        5   SB_DFFSS
      226   SB_LUT4
```

Two `SB_RAM40_4K` for the two FIFOs. (The `$scopeinfo` entries are not hardware; yosys keeps them to remember the names of the parts of the design after flattening. The flip-flops come in variants: `E` has an enable input, `SR` and `SS` a synchronous reset or set input. All of them are the flip-flop of a logic cell.)

Reading and writing the same address. What should `data_out` be if, at one clock edge, we write address 5 and read address 5? With `<=`, all right-hand sides are taken from before the edge (Chapter 3), so the description says: the old content (*read-first*). The block RAM does not guarantee this; for such a collision its output is undefined. Left to itself, yosys therefore

adds logic around the block that detects a collision and delivers the old value. The line before the declaration of `mem`,

```
(* no_rw_check *)
```

tells it not to. Text between `(*` and `*)` is an *attribute*: a note for the tools, attached to the declaration that follows. It describes no hardware itself; `no_rw_check` promises yosys that we never rely on the value read in a collision, so it may use the block RAM as it is. sv2v copies attributes into `test.v`. Whether the promise holds for our FIFOs is Exercise 8.3.

How much depends on the pattern? We synthesised `dp_bram4096` alone, as sv2v translates it, and in two modified versions: one without the attribute, and one where the read is combinational, `assign data_out = mem[addr_out];`:

```
== without_attribute.v ==
      47 cells
      27   SB_DFF
      19   SB_LUT4
       1   SB_RAM40_4K
== dp_bram4096.v ==
       1 cells
       1   SB_RAM40_4K
== async_read.v ==
    8303 cells
    4096   SB_DFFE
    4207   SB_LUT4
```

Without the attribute, 27 flip-flops and 19 LUTs appear next to the block RAM: the collision logic (yosys's log calls it "emulate read-first behavior"), once per FIFO. The last version shows what happens if we leave the pattern: the block RAM cannot read combinationally, so yosys builds the memory from 4096 flip-flops and the read multiplexer from 4207 LUTs, more than the whole chip has.

> **Pitfall.** Synthesis directives are specific to a tool. An earlier version of `dp_bram4096.sv` had the comment `/* synthesis syn_ramstyle = "no_rw_check" */` after the declaration of `mem`. That is the syntax of another vendor's synthesis tool; yosys ignores comments (and sv2v drops them), so the design silently carried the 46 extra cells per FIFO. If a directive matters, check in the statistics that it has an effect.

Initial contents. The `initial` block sets `mem[0]` to 255. For a block RAM this is possible: its initial contents are part of the bitstream, and all other entries start as 0. In our design this has a visible effect: before the first character arrives, the read pointer is 0, the output register shows `mem[0]`, and the display shows `FF`.

8.6 The I/O Device

Now we can read `dev_io`. It instantiates two interfaces of type `if_fifo` and two FIFOs, each connected through the modport `fifo`:

```
if_fifo inbuf(), outbuf();

fifo input_buffer(
    .clk(clk),
    .rst(rst),
    .fifo(inbuf.fifo)
);
```

Note that `dev_io` has no wires of its own for the FIFO signals: `inbuf` *is* the bundle of wires between `dev_io` and the input buffer, and `dev_io` drives `inbuf.push_back` and reads `inbuf.data_out`.

From the receiver into the buffers

07_io_interface/dev_io.sv

```
    logic rx_ready /* verilator public */;
    logic [7:0] rx_data_out /* verilator public */;
    /* verilator lint_off UNUSEDSIGNAL */
    logic rx_idle;
    logic rx_eop;
    /* verilator lint_on UNUSEDSIGNAL */

    logic rx_inbuf_push = 0;
    logic rx_outbuf_push = 0;
    logic [7:0] rx_data_buf;


    // buffer received data
    always_ff @ (posedge clk) begin
        if (rx_ready) begin
            rx_data_buf <= rx_data_out;
        end
    end

    // received data needs to go into the input buffer
    always_ff @ (posedge clk) begin
        if (rst) begin
            rx_inbuf_push <= 0;
        end
        if (rx_ready) begin
            rx_inbuf_push <= 1;
        end
        inbuf.push_back <= 0;
        if (!inbuf.full && !inbuf.push_back) begin
            if (rx_inbuf_push) begin
                inbuf.push_back <= 1;
                inbuf.data_in <= rx_data_buf;
                rx_inbuf_push <= 0;
            end
        end
    end
```

The receiver from step 06 reports a byte by setting `rx_ready` for one clock cycle. The first block stores the byte in the register `rx_data_buf`. The second block remembers in the flip-flop `rx_inbuf_push` that this byte still has to go into the input buffer, and pushes it when the FIFO is not full and has not been pushed in the previous cycle (rule (1) in the comment of `fifo.sv`). `inbuf.push_back` is a register that is 1 for exactly one cycle per byte. A third block of the same structure (lines 103–125) pushes into the output buffer; besides the received bytes it accepts the bytes of `putc`, and it acknowledges them with `putc_push_done`.

Exercise 8.1. In the input buffer, the read pointer is $r = 700$ and the write pointer is $w = 188$ (both in decimal). How many characters are waiting? Is the buffer full? At which

memory addresses are the characters? What would happen if `dev_io` pushed 512 more characters without looking at `full`?

When is a character available?

07_io_interface/dev_io.sv

```
logic inbuf_empty_r = 1;
logic getc_pop_r = 0;

always_ff @ (posedge clk) begin
    inbuf_empty_r <= inbuf.empty;
    getc_pop_r <= io.getc_pop;
end

// !inbuf_empty_r and !getc_pop_r indicate that io.getc_char is valid
// !inbuf.empty indicates that io.getc_pop is allowed
assign io.getc_en = !inbuf_empty_r && !getc_pop_r && !inbuf.empty;

//------------------------------------------------------------------------
//  Pop front from input buffer
//------------------------------------------------------------------------

assign inbuf.pop_front = io.getc_pop;
assign io.getc_char = inbuf.data_out;
```

Why not simply `getc_en = !inbuf.empty`? Figure 8.4 shows the reason; it is taken from the simulation of this step, where a character `'Z'` (`0x5a`) arrives. At the edge at time 14, the byte is written into the memory and the write pointer advances, so `empty` becomes 0. At the same edge, the output register of the memory loads the entry at the read pointer, and since the write happens at the same edge, it gets the old content, `0xff`. Only at the next edge (time 16) does `data_out` show `0x5a`. The register `inbuf_empty_r` is `empty` delayed by one cycle, and `getc_en` waits for both: it becomes 1 exactly when `getc_char` is valid.

The same delay occurs after a pop. In the clock cycle in which `getc_pop` is 1, `getc_char` is the front character, and the pop takes effect at the end of this cycle. In the cycle after it, the read pointer has moved, but the memory output still shows the character just popped; only one edge later does it show the next one. The register `getc_pop_r`, `getc_pop` delayed by one cycle, covers this cycle: `getc_en` is 0 in it, even if more characters are waiting. So a user of `dev_io` may rely on one simple rule: whenever `getc_en` is 1, `getc_char` is the front character and a pop is allowed. It takes the character in the cycle in which it pops (as the comment in `fifo.sv` says).

Pitfall. A signal that says "the data is valid" must account for every reason why the data can be stale. An earlier version of `dev_io` had only `inbuf_empty_r`, so `getc_en` was 1 in the cycle after a pop while `getc_char` still showed the popped character. On the board nobody noticed: at 9600 baud a character takes more than 12 000 clock cycles, and the user had taken each one long before the next arrived. In a simulation in which characters arrive every two clock cycles, the loader of Chapter 9 read characters twice and skipped others.

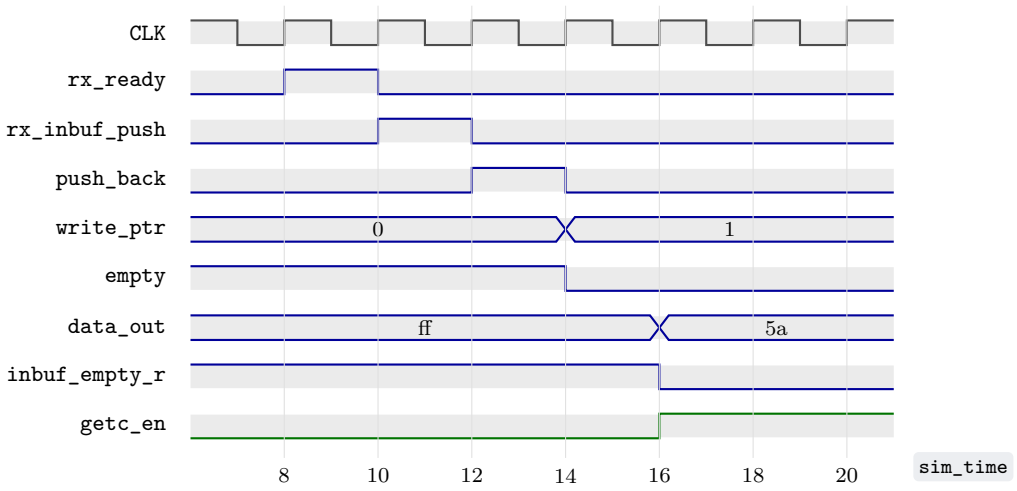


Figure 8.4: A character arrives in the input buffer (simulation of step 07; the gray lines are rising clock edges). `empty` becomes 0 one cycle before the memory output shows the new character; `getc_en` waits for it.

From the output buffer to the transmitter

07_io_interface/dev_io.sv

```
    logic tx_start;
    logic [7:0] tx_data_in;
    logic tx_busy;

    always_ff @ (posedge clk) begin
        tx_start <= 0;
        outbuf.pop_front <= 0;
        if (!tx_start && !tx_busy) begin
            if (!outbuf.empty && !outbuf.pop_front) begin
                outbuf.pop_front <= 1;
            end
        end
        if (outbuf.pop_front) begin
            tx_start <= 1;
            tx_data_in <= outbuf.data_out;
        end
    end
```

When the transmitter is idle and the output buffer is not empty, the block sets `pop_front` for one cycle. In this cycle `data_out` is the front byte, and at its end the byte is copied to `tx_data_in` and `tx_start` is set: the transmitter from step 06 starts sending. Here, too, the byte is taken in the cycle of the pop.

The top module

07_io_interface/test.sv

```
    assign LED1 = !if_io.getc_en;         // inbuf is empty
    assign LED2 = if_io.inbuf_full;       // inbuf is full
    assign LED3 = BTN1 && !if_io.getc_en; // error: can't pop from empty inbuf

    // ---------

    initial begin
        if_io.getc_pop = 0;
    end

    logic btn1_press;

    debounce debounce1(
        .clk(CLK),
        .btn(BTN1),
        .press(btn1_press)
    );

    always_ff @ (posedge CLK) begin
        if_io.getc_pop <= 0;
        if (btn1_press) begin
            if_io.getc_pop <= if_io.getc_en;
        end
    end
```

A button bounces (Chapter 3): for a few milliseconds after it is pressed or released, the contacts open and close several times. Each pop must remove exactly one character, so `test` does not look at `BTN1` directly. The module `debounce` turns every press into a pulse `btn1_press` that is 1 for exactly one clock cycle, and in that cycle `getc_pop` is set to `getc_en`: we pop only if there is something to pop. Pushing with `BTN2` works the same way, with a second instance of `debounce`. The `initial` block sets the power-up value of the flip-flop `getc_pop`, which lives in the interface.

07_io_interface/debounce.sv

```
/*
 * Debouncer for a push button
 *
 * btn is synchronised with two flip-flops. The debounced level takes the
 * value of the synchronised button only after it has been stable for 2^W
 * cycles (W = 17: about 11 ms at 12 MHz). press is 1 for exactly one cycle
 * when the debounced level changes from 0 to 1.
 */

module debounce #(
    parameter W = 17
) (
    input logic clk,
    input logic btn,
    output logic press
);

    // synchroniser
```

```
    logic btn_s1 = 0, btn_s2 = 0;

    always_ff @ (posedge clk) begin
        btn_s1 <= btn;
        btn_s2 <= btn_s1;
    end

    // debounced level: takes the value of btn_s2 after btn_s2 differed from
    // it for 2^W cycles in a row
    logic level = 0;
    logic [W-1:0] cnt = 0;

    always_ff @ (posedge clk) begin
        if (btn_s2 == level) begin
            cnt <= 0;
        end
        else if (&cnt) begin
            level <= btn_s2;
            cnt <= 0;
        end
        else begin
            cnt <= cnt + 1;
        end
    end

    // edge detector: press is 1 in the first cycle in which level is 1
    logic level_r = 0;

    always_ff @ (posedge clk) begin
        level_r <= level;
    end

    assign press = level && !level_r;

endmodule
```

The debouncer is the one suggested in Chapter 5, in three parts:

- a *synchroniser* of two flip-flops, `btn_s1` and `btn_s2`, because the button changes without regard to the clock;
- the debounced `level`, a flip-flop that follows `btn_s2` only slowly. The counter `cnt` counts the cycles in which `btn_s2` differs from `level`, and is cleared in every cycle in which they agree. Only if they differ in $2^W = 2^{17} = 131\,072$ cycles in a row, about 11 ms at 12 MHz, does `level` take the new value. A bounce shorter than that clears the counter again and has no effect. `&cnt` is a *reduction*: the AND of all bits of `cnt`, so it is 1 exactly when `cnt` is $2^W - 1$, all ones.
- an edge detector, `level_r` one cycle behind `level`, so `press` is 1 in the first cycle in which `level` is 1.

The price is a delay of 11 ms between pressing the button and the pop, which nobody notices, and about 60 cells per button (Section 8.8).

Exercise 8.2. In `test.sv`, instantiate both debouncers with `debounce #(.W(2))`, so that a level counts after four stable cycles. The testbench of the next section presses each button

once, but lets it bounce: four short pulses of 300 cycles before it stays pressed, and three short interruptions when it is released. Predict what the testbench prints, and check with `make sim`.

8.7 Simulation

At 9600 baud a single bit lasts $12\,000\,000/9600 = 1250$ clock cycles, and a character more than ten times as long. A simulation that sends real characters through `uart_rx` would mostly show waiting. The testbench therefore bypasses the receiver and sets its outputs directly. It drives the clock in a function `tick`, one cycle per call, and at three rising edges it delivers the characters `'Z'`, `'1'` and `'2'`:

07_io_interface/tb_test.cpp

```
void
tick()
{
    dut->CLK = 1;
    dut->eval();
    ++posedge_cnt;

    decode_tx();

    dut->test->io0->uart_rx0->rx_ready = 0;
    switch (posedge_cnt) {
        case 5:
            dut->test->io0->uart_rx0->rx_ready = 1;
            dut->test->io0->uart_rx0->rx_data = 'Z';
            break;
        case 15:
            dut->test->io0->uart_rx0->rx_ready = 1;
            dut->test->io0->uart_rx0->rx_data = '1';
            break;
        case 25:
            dut->test->io0->uart_rx0->rx_ready = 1;
            dut->test->io0->uart_rx0->rx_data = '2';
            break;
    }
```

`dut->test->io0->uart_rx0->rx_ready` is the signal `rx_ready` inside the instance `uart_rx0` inside `io0` inside `test`. Verilator makes such internal signals accessible from C++ only if they are marked with the comment `/* verilator public */`, which is why the copies of the UART modules in this directory differ from those of step 06 in exactly this point. In the same way, `test.sv` marks `hex_val`, the front character that the display shows, so that the testbench can read it.

The rest of the testbench plays the user. It presses `BTN1` once to pop `'Z'`, and then `BTN2` once to push the byte `0x41`, `'A'`, set at `PMOD_1B`. To test the debouncers, each press bounces:

07_io_interface/tb_test.cpp

```
// Press and release a button. The press bounces: the contact closes and
// opens a few times before it stays closed, and again when it is released.
void
press(unsigned char &btn)
{
    for (int i = 0; i < 4; ++i) {
        btn = 1;
        for (int k = 0; k < 300; ++k) tick();
        btn = 0;
        for (int k = 0; k < 200; ++k) tick();
    }
    btn = 1;
    for (std::uint64_t k = 0; k < HOLD; ++k) tick();
    for (int i = 0; i < 3; ++i) {
        btn = 0;
        for (int k = 0; k < 200; ++k) tick();
        btn = 1;
        for (int k = 0; k < 300; ++k) tick();
    }
    btn = 0;
    for (std::uint64_t k = 0; k < HOLD; ++k) tick();
}
```

`HOLD` is 140 000 cycles, a little more than the 2^{17} that the debouncer waits. Meanwhile, a small decoder in `tick` samples `TX` in the middle of each bit and collects the bytes that the transmitter sends: the echo of the three received characters, and the pushed `'A'`. At the end the testbench prints what it saw and compares it with what it expects:

```
$ make sim
...

### SIMULATING ###
front of input buffer: 5a
after BTN1:            31
TX: 5a 31 32 41
PASSED
```

One press, one pop, one push. The whole run is about 600 000 clock cycles; only the first 100 go into `waveform.vcd`, which keeps the file small. Figure 8.4 is the beginning of it.

8.8 Resources

```
$ make report
...
Resource report for test in 07_io_interface

== Device utilisation (nextpnr, iCE40 UP5K) ==
   ICESTORM_LC = logic cell (4-input LUT + flip-flop)

    ICESTORM_LC:       285/   5280    5%
    ICESTORM_RAM:        2/     30    6%
    SB_IO:              24/     39   61%
    SB_GB:               1/      8   12%

Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 52.80 MHz (PASS at 12.00 MHz)

== Cells per module (yosys) ==
   first line: the whole design; below it: the cells of each submodule
   itself, without its own submodules (indented under it)
   cells = LUTs, flip-flops, carry cells, RAMs, ... (before packing into logic cells)
   modules with interface ports are merged into their parent by sv2v

      585 test
       65 uart_rx
       58   baud_tick_gen
       29 uart_tx
       58   baud_tick_gen
        1 dp_bram4096
       62 debounce
       47 dev_hex
```

The design fits easily: 285 logic cells and 2 of the 30 block RAMs. The list of modules shows the effect of sv2v from Section 8.2: `dev_io` and both FIFOs have interface ports and were merged into `test`, so their cells are counted there. The two memories appear once as `dp_bram4096`, with the single block RAM cell we saw above, and the two debouncers once as `debounce` with 62 cells (the list shows each module once, not once per instance).

Exercise 8.3. The attribute `(* no_rw_check *)` promises yosys that a read and a write of the same address at the same edge never matter. Argue that the promise holds for the I/O device: when can a write and a read of the same address happen at the same edge, and is `data_out` used in the following cycle? Then delete the attribute and check with `make report` what it saves.

8.9 What Is Next

We have a device that buffers input and output, and a defined way to read and write characters through `if_io`. What is missing for a computer is memory for the program and its data: 512 bytes of block RAM would not get us far. The next step uses the much larger memory blocks of the iCE40UP5K and writes a program into them, typed in as hexadecimal digits through `dev_io`.

8.10 Solutions

Exercise 8.1. $n = (188 - 700) \bmod 1024 = 512$, so 512 characters are waiting and the buffer is full: the address bits are equal ($700 \bmod 512 = 188$) and the top bits differ ($700 \geq 512$, $188 < 512$). The characters occupy all addresses, starting at the front at address 188, up to 511, then 0 to 187. Pushing into a full buffer is forbidden by the rules of the FIFO, and `dev_io` checks `full` before every push. If 512 pushes happened anyway, they would overwrite all waiting characters, and w would become $188 + 512 = 700 = r$: the FIFO would report `empty`, although 512 characters were pushed since the last pop.

Exercise 8.2. With $W = 2$, a level counts after $2^2 = 4$ stable cycles. The bounces last 200 and 300 cycles, far more than four, so each of them is a separate press for the debouncer: four presses before the button stays down, the real one, and three more at the release, eight rising edges of `level` per button. The eight pops of `BTN1` remove all three characters (pops on an empty buffer are suppressed by `getc_en`), and the display shows `00`, the never-written entry 3 of the memory. The eight pushes of `BTN2` send eight `'A'`:

```
$ make sim
...

### SIMULATING ###
front of input buffer: 5a
after BTN1:            00
TX: 5a 31 32 41 41 41 41 41 41 41 41
FAILED
```

The debouncer must wait longer than the contacts bounce; with $W = 17$ it waits about 11 ms.

Exercise 8.3. A write to the address at the read pointer happens only when the FIFO is empty (otherwise the write pointer is ahead of the read pointer, and when it is a whole lap ahead, the FIFO is full and no push is allowed). After this edge, `data_out` may be wrong for one cycle. For the input buffer, `getc_en` stays 0 in this cycle because `inbuf_empty_r` is still 1. For the output buffer, the pop is decided at the end of this cycle and happens in the next, when `data_out` has been loaded again. So the device works without the extra logic. Without the attribute, `make report` shows 47 cells for `dp_bram4096` instead of 1: the attribute saves 46 cells per FIFO.

Chapter 9

Step 08: A Program Loader

A computer needs memory, and the 512 bytes of a block RAM are not much. The iCE40UP5K has four larger memory blocks, together 128 KiB, and in this step we put them to use: as the 64-bit memory of the ULM, which can be read and written in bytes, words (2 bytes), longs (4 bytes) and quads (8 bytes). To get something into the memory, we build a *loader*: it reads hexadecimal digits from the I/O device of step 07, combines two of them into a byte, and stores the bytes one after the other, until the character `0x04` ends the input. A small debugger lets us look at the result with the buttons. New are the SPRAM blocks of the FPGA and how we instantiate them, a simulation model that the synthesis must not see, the shift and mask logic for accesses of different sizes, and `for` loops. Everything is in `08_dev_loader`; the I/O device and the FIFOs come from step 07 (Chapter 8).

9.1 What the Design Does

In a terminal connected to the board you type a program as hexadecimal digits, for example `32 10`, and then Ctrl-D, which sends the character `0x04`. The loader stores `0x32` at address 0 and `0x10` at address 1. Characters that are not hexadecimal digits, like the space, are skipped. While you type, the display shows the byte being assembled, and `LED4` is on while half of a byte (one digit) has been typed. After `0x04`, `LED3` goes on and the memory belongs to the debugger: `BTN1` reads the byte at address 0, each press of `BTN2` the byte at the next address, and the display shows the low byte of the address, or the byte read while `BTN3` is held. Pressing `BTN1` and `BTN3` together resets the loader.

Figure 9.1 shows how the parts are connected. Three interfaces of type `if_ram` connect the loader and the debugger to a switch, and the switch to the memory `dev_ram`. The switch is controlled by the loader's output `done`.

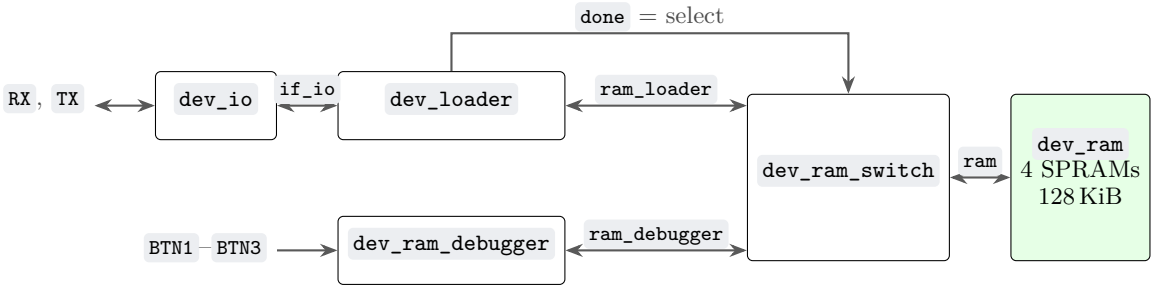


Figure 9.1: The parts of step 08. Until the loader is done, the switch connects it to the memory; afterwards, the debugger.

9.2 The Single-Port RAMs of the iCE40UP5K

Besides the 30 block RAMs of Section 8.5, the iCE40UP5K has four *single-port RAMs*, SPRAM for short. Each has 256 kbit, organised as 16384 words of 16 bits, so together they hold $4 \cdot 256\,\text{kbit} = 1\,\text{Mbit} = 128\,\text{KiB}$, more than eight times as much as all block RAMs ($30 \cdot 4\,\text{kbit} = 15\,\text{KiB}$). The price is in the name: a single port means one address per clock cycle, used either for reading or for writing.

Instead of describing a memory and hoping that yosys recognises it, as in step 07, we instantiate the hard block directly, like a module. Its name in the netlist is `SB_SPRAM256KA`, and `spram.sv` wraps it:

08_dev_loader/spram.sv

```
// From: https://projectf.io/posts/spram-ice40-fpga/
module spram #(
    localparam WIDTH=16,     // fixed data width 16-bits
    localparam DEPTH=16384,  // fixed depth: 16K
    localparam ADDRW=$clog2(DEPTH)
) (
    input logic clk,
    input logic [3:0] we,
    input logic [ADDRW-1:0] addr,
    input logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);

    SB_SPRAM256KA spram_inst (
        .ADDRESS(addr),
        .DATAIN(data_in),
        .MASKWREN(we),
        .WREN(|we),
        .CHIPSELECT(1'b1),
        .CLOCK(clk),
        .STANDBY(1'b0),
        .SLEEP(1'b0),
        .POWEROFF(1'b1),
        .DATAOUT(data_out)
    );
endmodule
```

There is no module `SB_SPRAM256KA` in our design that describes how it works. yosys knows it as one of the building blocks of the iCE40, like `SB_LUT4`, and puts it into the netlist as it is; nextpnr places it on one of the four SPRAM sites. Its ports:

`ADDRESS`, `DATAIN`, `DATAOUT`: 14 address bits ($2^{14} = 16384$ words) and 16 data bits in each direction.

`WREN`, `MASKWREN`: `WREN` = 1 writes, `WREN` = 0 reads. `MASKWREN` has one bit per 4-bit group (*nibble*) of the word, and only the nibbles whose bit is 1 are written. So we can write a single byte of a word by setting two of the four bits. `spram.sv` uses one signal `we` for both: write if any mask bit is set.

`CLOCK`: both reading and writing happen at the rising edge. A read delivers the word at `DATAOUT` after the edge, like the block RAM of step 07.

`CHIPSELECT`, `STANDBY`, `SLEEP`, `POWEROFF`: for power saving. We keep the block always selected and awake. Note that `POWEROFF` is active low: 1 means "power on".

A model for the simulation, hidden from the synthesis

Verilator does not know `SB_SPRAM256KA`: it is not a SystemVerilog module but a piece of silicon. For the simulation we need a description of what it does, a *simulation model*. The file `SB_SPRAM256KA.sv` contains one, taken from yosys's own models of the iCE40 cells:

08_dev_loader/SB_SPRAM256KA.sv

```
module SB_SPRAM256KA (
        input [13:0] ADDRESS,
        input [15:0] DATAIN,
        input [3:0] MASKWREN,
        input WREN, CHIPSELECT, CLOCK, STANDBY, SLEEP, POWEROFF,
        output reg [15:0] DATAOUT
);
`ifndef SYNTHESIS
        reg [15:0] mem [0:16383];
        wire off = SLEEP || !POWEROFF;
        integer i;

        // Kept as in the yosys model. Only runs on a falling POWEROFF edge,
        // which never happens here (spram.sv ties POWEROFF to 1).
        /* verilator lint_off BLKSEQ */
        always @(negedge POWEROFF) begin
            for (i = 0; i <= 16383; i = i+1)
                mem[i] = 16'bx;
        end
        /* verilator lint_on BLKSEQ */

        always @(posedge CLOCK, posedge off) begin
            if (off) begin
                DATAOUT <= 0;
            end else
            if (STANDBY) begin
                DATAOUT <= 16'bx;
            end else
            if (CHIPSELECT) begin
                if (!WREN) begin
                        DATAOUT <= mem[ADDRESS];
                end
                else begin
                    if (MASKWREN[0]) mem[ADDRESS][ 3: 0] <= DATAIN[ 3: 0];
                    if (MASKWREN[1]) mem[ADDRESS][ 7: 4] <= DATAIN[ 7: 4];
                    if (MASKWREN[2]) mem[ADDRESS][11: 8] <= DATAIN[11: 8];
                    if (MASKWREN[3]) mem[ADDRESS][15:12] <= DATAIN[15:12];
                    DATAOUT <= 16'bx;
                end
            end
        end
`endif // SYNTHESIS
endmodule
```

The model is a memory `mem` of 16384 words, here declared with an explicit range `[0:16383]` instead of the size `[DEPTH]` of step 07; both mean the same. At the rising clock edge it either reads into `DATAOUT` or writes the selected nibbles, and while writing `DATAOUT` becomes undefined (`16'bx`). This is how we learn the behaviour of the hard block without the board.

The body of the model is enclosed in `` `ifndef SYNTHESIS `` … `` `endif ``. As with the include guard

of Section 8.3, the preprocessor drops the text if the macro `SYNTHESIS` is defined. The Makefile calls sv2v with `--define=SYNTHESIS` (Section 2.3), Verilator without. So Verilator simulates the model, while the file that yosys reads contains only the ports:

```
module SB_SPRAM256KA (
        ADDRESS,
        DATAIN,
        MASKWREN,
        WREN,
        CHIPSELECT,
        CLOCK,
        STANDBY,
        SLEEP,
        POWEROFF,
        DATAOUT
);
        input [13:0] ADDRESS;
        input [15:0] DATAIN;
        input [3:0] MASKWREN;
        input WREN;
        input CHIPSELECT;
        input CLOCK;
        input STANDBY;
        input SLEEP;
        input POWEROFF;
        output reg [15:0] DATAOUT;
endmodule
```

An empty module like this is a *black box*: it says which connections the block has, but nothing about its inside, which is exactly right for a hard block. Without the guard, a tool that does not know `SB_SPRAM256KA` would have to build a memory of 256 kbit from logic cells, and the whole chip does not have that many flip-flops. (yosys's `synth_ice40` happens to be safe anyway: it loads its own definition of the cell, and we checked that it produces the same statistics without `--define=SYNTHESIS`. The guard does not rely on that.)

9.3 for Loops Are Replicated Hardware

The model contains our first `for` loop:

```
always @(negedge POWEROFF) begin
    for (i = 0; i <= 16383; i = i+1)
        mem[i] = 16'bx;
end
```

In the simulation this is what it looks like: a loop that the simulator runs, setting all words to "undefined" when the power goes off. It is part of the model, which the synthesis never sees.

In a description of hardware, a `for` loop means something else. There is no time in which a loop could run; the tools *unroll* it, that is, they write the body down once for each value of the loop variable, and each copy becomes its own piece of circuit. The number of iterations must be a constant. An example: the module `quad_lshift` below (Section 9.4) selects one of eight shifts with a `case`. The same can be written with a loop:

```
always_comb begin
    data_out = data_in;
    for (int k = 1; k < 8; k++)
        if (offset == k)
            data_out = data_in << 8*k;
end
```

Unrolled, this is seven `if` statements one after the other, and each `if` is a comparator and a 64-bit multiplexer that either passes on the value from the copy before or replaces it: a chain of seven multiplexers. In `always_comb`, a later assignment to the same signal wins, so the chain is ordered, with the last copy closest to the output. We synthesised the repository's version, this loop, and a third version with a single shift, `data_out = data_in << {offset, 3'b000};` (the comment in `quad_lshift.sv`):

```
== quad_lshift, case version ==
      209 cells
      209   SB_LUT4
== quad_lshift, for version ==
      276 cells
      276   SB_LUT4
== quad_lshift, shift version ==
      182 cells
      182   SB_LUT4
```

All three compute the same function. The loop is a chain of seven 2-to-1 multiplexers and costs the most; the `case` is one 8-to-1 multiplexer per bit; the single shift by $8 \cdot$ `offset` lets yosys build a *barrel shifter*: three stages that shift by 8, 16 and 32 bits or not, one stage per bit of `offset`. yosys simplifies a lot, but not everything; how we describe a function still shows in the result.

9.4 A 64-bit Memory from Four 16-bit Ones

The ULM has 64-bit registers and reads and writes bytes, words, longs and quads. We place the four SPRAMs side by side, so that each address of the SPRAMs holds a quad of 64 bits (Figure 9.2). The memory is addressed in bytes: a byte address a has 17 bits, since $2^{17} = 128\,\text{Ki}$, and we split it as

$$a = 8t + o, \qquad t = \lfloor a/8 \rfloor \in \{0, \ldots, 16383\}, \quad o = a \bmod 8 \in \{0, \ldots, 7\},$$

where the *tag* t (the upper 14 bits of a) is the address of the quad in all four SPRAMs and the *offset* o (the lower 3 bits) selects the byte within the quad. The byte with offset 0 is the most significant byte of the quad: the bytes are stored in big-endian order, as on the ULM.

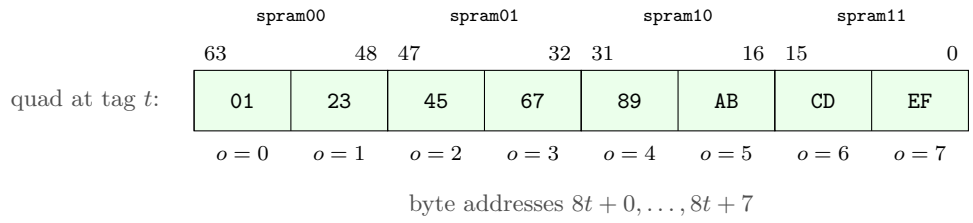


Figure 9.2: One quad of the memory, here with the value `0x0123456789ABCDEF`. Each SPRAM holds 16 of its 64 bits, all four at the same address t. The byte at address $8t + o$ is bits $63 - 8o$ down to $56 - 8o$.

The types of an access

The package `pkg_ram` from Section 8.3 defines the kinds of access:

08_dev_loader/pkg_ram.sv

```
typedef enum {
    RAM_BYTE,
    RAM_WORD,
    RAM_LONG,
    RAM_QUAD
} data_type_t;

typedef enum {
    RAM_NIBBLE_SIZE = 4,
    RAM_BYTE_SIZE = 8,
    RAM_WORD_SIZE = 16,
    RAM_LONG_SIZE = 32,
    RAM_QUAD_SIZE = 64
} data_size_t;

typedef enum {
    RAM_NOP,
    RAM_FETCH,
    RAM_STORE
} op_t;
```

and the interface to the memory uses them as types:

08_dev_loader/if_ram.sv

```
`include "pkg_ram.sv"

interface if_ram;
    pkg_ram::op_t op;
    pkg_ram::data_type_t data_type;
    logic [pkg_ram::RAM_ADDRW-1:0] addr;
    logic [pkg_ram::RAM_QUAD_SIZE-1:0] data_in;
    logic [pkg_ram::RAM_QUAD_SIZE-1:0] data_out;

    modport server(
        input op,
        input data_type,
        input addr,
        input data_in,
        output data_out
    );

    modport client(
        output op,
        output data_type,
        output addr,
        output data_in,
        input data_out
    );

endinterface
```

A user sets `op` to `RAM_FETCH` or `RAM_STORE` for one clock cycle, together with `data_type`, the byte address `addr` and, for a store, `data_in`. A value of 8, 16 or 32 bits is always passed in the low bits of the 64-bit `data_in` and `data_out`.

Reading: shift left, then extend

08_dev_loader/dev_ram.sv

```
    logic fetch, store;
    assign fetch = ram.op == RAM_FETCH;
    assign store = ram.op == RAM_STORE;

    //
    // update addr and size if there's a fetch or store operation
    //
    logic [RAM_ADDRW-1:0] addr, addr_r = 0;
    data_type_t data_type, data_type_r = RAM_QUAD;

    always_ff @ (posedge clk) begin
        if (fetch || store) begin
            addr_r <= ram.addr;
            data_type_r <= ram.data_type;
        end
    end

    always_comb begin
        addr = addr_r;
        data_type = data_type_r;
        if (fetch || store) begin
            addr = ram.addr;
            data_type = ram.data_type;
        end
    end

    //
    // split addr into tag and offset
    //
    logic [2:0] addr_offset;
    logic [SPRAM_ADDRW-1:0] addr_tag;

    always_comb begin
        addr_offset = addr[2:0];
        addr_tag = addr[RAM_ADDRW-1:3];
    end
```

The SPRAM delivers its data one clock cycle after the address. By then the user has usually taken `op` back to `RAM_NOP` and may have changed `addr`. So `dev_ram` stores the address and data type of every access in the registers `addr_r` and `data_type_r`, and the `always_comb` block selects: during an access the new values, otherwise the stored ones. Thus the offset that the shifters see belongs to the data that the SPRAMs deliver. (In cycles without an access, `we` is 0, and the SPRAMs keep reading the quad at the stored address.)

The rest of `dev_ram` instantiates the four SPRAMs with the tag as address, and between them and the interface the units that move the bytes. The read side:

08_dev_loader/dev_ram.sv

```
    //
    // left shift quad from spram into quad_out
    //
    logic [SPRAM_WIDTH-1:0] w00_out, w01_out, w10_out, w11_out;
    logic [RAM_QUAD_SIZE-1:0] quad_out;

    quad_lshift quad_lshift0(
        .data_in({w00_out, w01_out, w10_out, w11_out}),
        .offset(addr_offset),
        .data_out(quad_out)
    );

    //
    // (zero) extend data from quad_out into ram.data_out
    //
    quad_ext_high quad_ext_high(
        .data_in(quad_out),
        .data_type(data_type),
        .data_out(ram.data_out)
    );
```

To read the value at offset o, `quad_lshift` shifts the quad left by $8o$ bits, so that the wanted bytes become the most significant ones:

08_dev_loader/quad_lshift.sv

```
`include "pkg_ram.sv"
import pkg_ram::RAM_QUAD_SIZE;
import pkg_ram::RAM_LONG;
import pkg_ram::RAM_WORD;
import pkg_ram::RAM_BYTE;

module quad_lshift (
    input logic [RAM_QUAD_SIZE-1:0] data_in,
    input logic [2:0] offset,
    output logic [RAM_QUAD_SIZE-1:0] data_out
);

    always_comb begin
        //data_out = data_in << {offset, 3'b000};
        case (offset)
            default:
                data_out = data_in;
            3'b001: data_out = data_in << 8;
            3'b010: data_out = data_in << 16;
            3'b011: data_out = data_in << 24;
            3'b100: data_out = data_in << 32;
            3'b101: data_out = data_in << 40;
            3'b110: data_out = data_in << 48;
            3'b111: data_out = data_in << 56;
        endcase
    end

endmodule
```

(`<<` shifts left and fills with zeros; the bits that move out at the top are lost. The imports of

`RAM_LONG` and so on are not used here.) Then `quad_ext_high` takes the top 8, 16, 32 or 64 bits and puts them into the low bits of the result, filling the rest with zeros:

08_dev_loader/quad_ext_high.sv

```
    always_comb begin
        case (data_type)
            default:
                data_out = data_in;
            RAM_LONG:
                data_out = {
                    {RAM_QUAD_SIZE-RAM_LONG_SIZE{1'b0}},
                    data_in[RAM_QUAD_SIZE-1:RAM_QUAD_SIZE-RAM_LONG_SIZE]
                };
            RAM_WORD:
                data_out = {
                    {RAM_QUAD_SIZE-RAM_WORD_SIZE{1'b0}},
                    data_in[RAM_QUAD_SIZE-1:RAM_QUAD_SIZE-RAM_WORD_SIZE]
                };
            RAM_BYTE:
                data_out = {
                    {RAM_QUAD_SIZE-RAM_BYTE_SIZE{1'b0}},
                    data_in[RAM_QUAD_SIZE-1:RAM_QUAD_SIZE-RAM_BYTE_SIZE]
                };
        endcase
    end
```

`{RAM_QUAD_SIZE-RAM_BYTE_SIZE{1'b0}}` is a replication (Chapter 6): 56 copies of the bit 0, concatenated. So, for a byte, `data_out` is 56 zeros followed by bits 63 to 56 of the shifted quad. The `default` case, here for `RAM_QUAD`, passes everything.

With the quad of Figure 9.2, a byte fetch at offset 3:

from the SPRAMs	`0x0123456789ABCDEF`	
shifted left by $8 \cdot 3 = 24$	`0x6789ABCDEF000000`	(`quad_lshift`)
top 8 bits, zero-extended	`0x0000000000000067`	(`quad_ext_high`)

and a long fetch at offset 4 gives `0x89ABCDEF00000000` after the shift and `0x0000000089ABCDEF` at the end.

Writing: extend, shift right, mask

A store runs the other way. `quad_ext_low` moves the low 8, 16 or 32 bits of `data_in` to the top of a quad, with zeros below (it is the mirror image of `quad_ext_high`), and `quad_rshift` shifts right by $8o$, which moves them to their place. Then all four SPRAMs get 16 bits each, but only the nibbles that belong to the value may be written. This is the job of `quad_we_mask`:

08_dev_loader/quad_we_mask.sv

```
always_comb begin
    {we00, we01, we10, we11} = 16'h0000;
    if (store) begin
        if (data_type == RAM_QUAD)
            {we00, we01, we10, we11} = 16'hFFFF;
        else if (data_type == RAM_LONG)
            case (offset)
                3'b000: {we00, we01, we10, we11} = 16'hFF00;
                3'b100: {we00, we01, we10, we11} = 16'h00FF;
                default: ;
            endcase
        else if (data_type == RAM_WORD)
            case (offset)
                3'b000: {we00, we01, we10, we11} = 16'hF000;
                3'b010: {we00, we01, we10, we11} = 16'h0F00;
                3'b100: {we00, we01, we10, we11} = 16'h00F0;
                3'b110: {we00, we01, we10, we11} = 16'h000F;
                default: ;
            endcase
        else if (data_type == RAM_BYTE)
            case (offset)
                3'b000: {we00, we01, we10, we11} = 16'hC000;
                3'b001: {we00, we01, we10, we11} = 16'h3000;
                3'b010: {we00, we01, we10, we11} = 16'h0C00;
                3'b011: {we00, we01, we10, we11} = 16'h0300;
                3'b100: {we00, we01, we10, we11} = 16'h00C0;
                3'b101: {we00, we01, we10, we11} = 16'h0030;
                3'b110: {we00, we01, we10, we11} = 16'h000C;
                3'b111: {we00, we01, we10, we11} = 16'h0003;
            endcase
    end
end
```

The 16 mask bits `{we00, we01, we10, we11}` correspond to the 16 nibbles of the quad, the first bit to bits 63–60. A byte is two nibbles, so a byte store at offset o sets bits $2o$ and $2o+1$ counted from the left: `16'hC000` for offset 0, `16'h3000` for offset 1, and so on. Here is a byte store of `0xAB` at offset 5:

`data_in`	`0x00000000000000AB`	
to the top	`0xAB00000000000000`	(`quad_ext_low`)
shifted right by $8 \cdot 5 = 40$	`0x0000000000AB0000`	(`quad_rshift`)
mask	`0x0030` $= 0000\ 0000\ 0011\ 0000_2$	(`quad_we_mask`)

The mask enables two nibbles of `spram10`, and there the 16 bits are `0x00AB`: exactly the byte `0xAB` lands in bits 23 to 16 of the quad, and the other seven bytes in memory stay as they were.

For words and longs, the mask is set only if the offset is a multiple of the size. A store to an unaligned address, say a word at offset 1, writes nothing.

Exercise 9.1. The quad at tag t is `0x0123456789ABCDEF`. What does a word fetch from address $8t+6$ deliver? And a word fetch from $8t+5$? What happens with a word store to $8t+5$?

Exercise 9.2. A long `0xDEADBEEF` is stored at byte address 12. Which tag and offset is that, what are `we00` to `we11`, and which 16 bits does each SPRAM get at `DATAIN`?

9.5 What the Shifters Cost

Two shifters, two extenders and a mask generator: how much hardware is that?

```
$ make report
...
Resource report for test in 08_dev_loader

== Device utilisation (nextpnr, iCE40 UP5K) ==
   ICESTORM_LC = logic cell (4-input LUT + flip-flop)

    ICESTORM_LC:       847/   5280   16%
    ICESTORM_RAM:        2/     30    6%
    SB_IO:              18/     39   46%
    SB_GB:               2/      8   25%
    ICESTORM_SPRAM:      4/      4  100%

Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 27.24 MHz (PASS at 12.00 MHz)

== Cells per module (yosys) ==
   first line: the whole design; below it: the cells of each submodule
   itself, without its own submodules (indented under it)
   cells = LUTs, flip-flops, carry cells, RAMs, ... (before packing into logic cells)
   modules with interface ports are merged into their parent by sv2v

     1644 test
       65 uart_rx
       58   baud_tick_gen
       29 uart_tx
       58   baud_tick_gen
        1 dp_bram4096
       62 debounce
       47 dev_hex
      129 quad_ext_high
      129 quad_ext_low
      209 quad_lshift
      209 quad_rshift
       53 quad_we_mask
        2 spram
```

The modules without interface ports appear in the list: the five `quad_*` modules together have $129 + 129 + 209 + 209 + 53 = 729$ cells, all of them LUTs. That is more than the whole design of step 07, for logic that does nothing but move bytes around. Each output bit of a shifter is a multiplexer that chooses among up to eight input bits, controlled by three offset bits; such a function has up to eleven inputs and needs several 4-input LUTs, and there are 64 output bits.

These per-module numbers come from synthesising every module on its own, for arbitrary inputs. In the real design, yosys flattens everything and removes logic whose inputs are constant or whose outputs are unused. In this step the loader and the debugger only access bytes, only the low byte of `data_in` is ever set, and only the low byte of `data_out` is used. So the whole design has fewer LUTs than the five modules alone:

```
2.60. Printing statistics.

=== test ===

       +----------Local Count, excluding submodules.
       |
      24 cells
      18   $scopeinfo
       2   SB_RAM40_4K
       4   SB_SPRAM256KA
    1176 submodules
     132   SB_CARRY
      92   SB_DFF
      78   SB_DFFE
     106   SB_DFFESR
       8   SB_DFFESS
      58   SB_DFFSR
       6   SB_DFFSS
     696   SB_LUT4
```

To see what the four shift and extend units cost here, we replaced them by plain wires (`assign data_out = data_in;`) and synthesised again: 352 LUTs instead of 696. So even in this step they account for half of the LUTs of the design. In step 09, where the processor uses all access sizes and all 64 bits, the same experiment gives 2843 against 2232 LUTs: there they cost about 600 LUTs, close to the $129 + 129 + 209 + 209 = 676$ of the report. The slowest path of the design, which limits it to 27.2 MHz, also runs through them: from the register `op` of the loader, through the switch, the selection of the data type and the shift and extend logic, to the display.

The other number worth noticing: `ICESTORM_SPRAM` 4 of 4. The memory of the ULM takes all SPRAMs of the chip.

9.6 Two Users, One Memory

08_dev_loader/dev_ram_switch.sv

```
`include "pkg_ram.sv"

module dev_ram_switch (
    input logic select,
    if_ram.server ram0,
    if_ram.server ram1,
    if_ram.client ram
);

    always_comb begin
        case (select)
            1'b0:
                begin
                    ram.op = ram0.op;
                    ram.data_type = ram0.data_type;
                    ram.addr = ram0.addr;
                    ram.data_in = ram0.data_in;
                    ram0.data_out = ram.data_out;
                    ram1.data_out = 0;
                end
            1'b1:
                begin
                    ram.op = ram1.op;
                    ram.data_type = ram1.data_type;
```

```
                    ram.addr = ram1.addr;
                    ram.data_in = ram1.data_in;
                    ram0.data_out = 0;
                    ram1.data_out = ram.data_out;
                end
        endcase

    end

endmodule
```

The switch is a multiplexer for a whole interface. Its ports are modports (Section 8.2), and their roles show the direction of the wires: towards the loader and the debugger the switch is a `server` (it receives requests and returns `data_out`), towards `dev_ram` it is a `client`. The `case` selects which user's requests go to the memory: 2-to-1 multiplexers for all bits of `op`, `data_type`, `addr` and `data_in`. The data from the memory goes to the selected user, and the other one gets 0.

The debugger:

08_dev_loader/dev_ram_debugger.sv

```
    logic [pkg_ram::RAM_ADDRW-1:0] addr = 0;

    // one-cycle pulses per press of the (bouncing) buttons
    logic rst_addr_press, fetch_press;

    debounce debounce_rst_addr(
        .clk(clk),
        .btn(rst_addr),
        .press(rst_addr_press)
    );

    debounce debounce_fetch(
        .clk(clk),
        .btn(fetch),
        .press(fetch_press)
    );

    initial begin
        ram.op = pkg_ram::RAM_NOP;
        ram.data_type = pkg_ram::RAM_BYTE;
        ram.addr = 0;
        ram.data_in = 0;
    end

    always_ff @ (posedge clk) begin
        ram.op <= pkg_ram::RAM_NOP;

        if (rst_addr_press) begin
            addr <= 1;
            ram.addr <= 0;
            ram.op <= pkg_ram::RAM_FETCH;
        end
        else if (fetch_press) begin
            if (ram.op == pkg_ram::RAM_NOP) begin
                ram.op <= pkg_ram::RAM_FETCH;
                ram.addr <= addr;
                addr <= addr + 1;
            end
        end
    end
```

The two buttons go through the debouncer of step 07 (Section 8.6), so each press gives one pulse of one cycle, however much the contacts bounce. `BTN1` fetches address 0 and sets the counter `addr` to 1; each `BTN2` fetches the address in `addr` and increments it. So the presses `BTN1`, `BTN2`, `BTN2`, … read the addresses 0, 1, 2, …. The output `out` is the address, or with `BTN3` the byte read, both extended to 64 bits. `LED4` lights if `out` has bits set above the low byte, which the display cannot show: without `BTN3`, from address 256 on.

The directory also contains `dev_io_switch.sv`, the same kind of switch for `if_io`. It is not used in this step; step 09 needs it.

9.7 The Loader

The loader reads one character at a time from `dev_io` through the `client` modport of `if_io`. First, a combinational circuit decides whether the character is a hexadecimal digit, and which value it has:

08_dev_loader/dev_loader.sv

```
    logic hex_char;
    logic [pkg_ram::RAM_NIBBLE_SIZE-1:0] nibble_val;

    always_comb begin
        {hex_char, nibble_val} = {1'b0, 4'h0};
        case (io.getc_char)
            "0": {hex_char, nibble_val} = {1'b1, 4'h0};
            "1": {hex_char, nibble_val} = {1'b1, 4'h1};
            "2": {hex_char, nibble_val} = {1'b1, 4'h2};
            "3": {hex_char, nibble_val} = {1'b1, 4'h3};
            "4": {hex_char, nibble_val} = {1'b1, 4'h4};
            "5": {hex_char, nibble_val} = {1'b1, 4'h5};
            "6": {hex_char, nibble_val} = {1'b1, 4'h6};
            "7": {hex_char, nibble_val} = {1'b1, 4'h7};
            "8": {hex_char, nibble_val} = {1'b1, 4'h8};
            "9": {hex_char, nibble_val} = {1'b1, 4'h9};
            "A": {hex_char, nibble_val} = {1'b1, 4'hA};
            "B": {hex_char, nibble_val} = {1'b1, 4'hB};
            "C": {hex_char, nibble_val} = {1'b1, 4'hC};
            "D": {hex_char, nibble_val} = {1'b1, 4'hD};
            "E": {hex_char, nibble_val} = {1'b1, 4'hE};
            "F": {hex_char, nibble_val} = {1'b1, 4'hF};
            "a": {hex_char, nibble_val} = {1'b1, 4'hA};
            "b": {hex_char, nibble_val} = {1'b1, 4'hB};
            "c": {hex_char, nibble_val} = {1'b1, 4'hC};
            "d": {hex_char, nibble_val} = {1'b1, 4'hD};
            "e": {hex_char, nibble_val} = {1'b1, 4'hE};
            "f": {hex_char, nibble_val} = {1'b1, 4'hF};
            default:
                ;
        endcase
    end
```

`"0"` is a string literal of one character, and in a comparison it stands for its 8-bit ASCII code, `8'h30`. So the `case` compares `io.getc_char` with 22 constants, and the block is a function of the 8 bits of the character with 5 output bits: a small decoder, a handful of LUTs, not a search through a table at run time. The first line gives the outputs their values for all other characters, so there are no missing cases.

Then two nibbles become a byte:

08_dev_loader/dev_loader.sv

```
    logic nibble_sel; // 1 = high, 0 = low
    logic [pkg_ram::RAM_NIBBLE_SIZE-1:0] low_nibble, high_nibble;
    logic new_byte;
    logic got_eof;

    assign byte_valid = nibble_sel;
    assign byte_val = {high_nibble, low_nibble};

    initial begin
        nibble_sel = 1;
        new_byte = 0;
        got_eof = 0;
        done = 0;
        io.putc_push = 0;
        io.putc_char = 0;
    end

    always_ff @ (posedge clk) begin
        io.getc_pop <= 0;
        new_byte <= 0;

        if (rst) begin
            nibble_sel <= 1;
            new_byte <= 0;
            got_eof <= 0;
            done <= 0;
        end
        else if (!done && got_eof) begin
            done <= 1;
        end
        else if (!done && !io.getc_pop && io.getc_en) begin
            io.getc_pop <= 1;
            if (hex_char) begin
                nibble_sel <= !nibble_sel;
                if (nibble_sel) begin
                    high_nibble <= nibble_val;
                    low_nibble <= 0;
                end else begin
                    low_nibble <= nibble_val;
                    new_byte <= 1;
                end
            end
            else if (io.getc_char == 8'h04) begin
                got_eof <= 1;
            end
        end
    end
```

The flip-flop `nibble_sel` says which half comes next, 1 for the high nibble. When a digit arrives, the loader pops it (`io.getc_pop <= 1`), stores its value in `high_nibble` or `low_nibble`, and toggles `nibble_sel`. After the low nibble, `new_byte` is 1 for one cycle. A character that is neither a digit nor `0x04` is popped and ignored. The character `0x04` is ASCII "end of transmission", what a terminal sends for Ctrl-D; it sets `got_eof`, one cycle later `done` is 1, and the loader stops reading. The condition `!io.getc_pop` keeps the loader from popping in

two consecutive cycles, as required by the FIFO (Section 8.4).

Finally, each new byte is stored at the next address:

08_dev_loader/dev_loader.sv

```
    logic [pkg_ram::RAM_ADDRW-1:0] addr;

    initial begin
        addr = 0;

        ram.op = pkg_ram::RAM_NOP;
        ram.data_type = pkg_ram::RAM_BYTE;
        ram.addr = 0;
        ram.data_in = 0;
    end

    always_ff @ (posedge clk) begin
        ram.op <= pkg_ram::RAM_NOP;
        io.putc_push <= 0;

        if (rst) begin
            addr <= 0;
        end
        else if (new_byte) begin
            ram.op <= pkg_ram::RAM_STORE;
            ram.addr <= addr;
            ram.data_in[pkg_ram::RAM_BYTE_SIZE-1:0] <= byte_val;
            addr <= addr + 1;
        end
    end
```

`ram.data_type` is set to `RAM_BYTE` in the `initial` block and never changed, and only the low 8 bits of `ram.data_in` are ever written: this is the constant input that made the shifters cheaper in Section 9.5.

How fast can the loader take characters? It decides at a clock edge to pop (`io.getc_pop <= 1`); in the next cycle the pop happens, and in the cycle after that `getc_en` is 0, because `getc_char` still shows the popped character (Section 8.6). So the loader takes at most one character every three cycles, and never a stale one.

In the simulation (`make sim`), the testbench delivers the characters `0` to `9`, `a`, `b` and `0x04`, one every two clock cycles, in the same way as in step 07. That is faster than the loader takes them, so several characters wait in the input buffer:

08_dev_loader/tb_test.cpp

```
// One clock cycle. Inputs set before the call are seen at the rising edge.
void
tick()
{
    dut->CLK = 1;
    dut->eval();
    ++posedge_cnt;

    // deliver the next character as if the receiver had got it
    dut->test->io0->uart_rx0->rx_ready = 0;
    if (posedge_cnt % 2 == 0 && posedge_cnt > 4 && numLoaded < prog_size) {
        dut->test->io0->uart_rx0->rx_ready = 1;
        dut->test->io0->uart_rx0->rx_data = prog[numLoaded++];
    }

    if (sim_time < MAX_TRACE_TIME) {
        m_trace->dump(sim_time);
    }
    ++sim_time;

    dut->CLK = 0;
    dut->eval();
    if (sim_time < MAX_TRACE_TIME) {
        m_trace->dump(sim_time);
    }
    ++sim_time;
}
```

In the waveform you see `ram.op` at `RAM_STORE` at time 28 with address 0 and data `0x01`, then every 12 time units (six clock cycles, two characters) the next byte, up to address 5 with `0xab` at time 88, and `done` becoming 1 at time 94. Then the testbench plays the user of the debugger: it presses `BTN1` once and `BTN2` five times, each press bouncing as in step 07, and after each press reads the address and, with `BTN3`, the byte from the debugger's output `debugger_out`, which `test.sv` marks as `/* verilator public */`:

```
$ make sim
...

### SIMULATING ###
loader done after 48 cycles
BTN1: address 00, byte 01 (expected 00, 01)
BTN2: address 01, byte 23 (expected 01, 23)
BTN2: address 02, byte 45 (expected 02, 45)
BTN2: address 03, byte 67 (expected 03, 67)
BTN2: address 04, byte 89 (expected 04, 89)
BTN2: address 05, byte ab (expected 05, ab)
PASSED
```

Verilator warns about `MULTIDRIVEN` signals in `dev_loader.sv`: as in step 02, these are the `initial` blocks that set power-up values.

Exercise 9.3. What does the loader store if you type `3 2 1` (return) `0` and then Ctrl-D? What if you type `123` and Ctrl-D? What does the display show, and is `LED4` on, before you press Ctrl-D in the second case?

Pitfall. A testbench that delivers characters at the speed of the real UART would never have found the problem of the pitfall in Section 8.6: at 9600 baud the loader has taken each character long before the next arrives. With the earlier version of `dev_io`, the testbench above shows the loader storing `00 12 34 56 78 9a` instead of `01 23 45 67 89 ab`: it reads the stale `'0'` a second time, pops the `'1'` without having read it, and so on. The `0x04` is popped unread as well, so the loader is never done. Test at the limits, not only at the speed you expect.

9.8 A Declaration Is Not a Connection

The reset of the loader is wired in `test.sv` in two lines:

```
logic rst;
assign rst = BTN1 & BTN3;
```

An earlier version said it in one line,

```
logic rst = BTN1 & BTN3;
```

which looks the same, but in SystemVerilog it is something else: `=` in the declaration of a variable gives it an *initial value*, computed once at the start, just like an `initial` block (Chapter 3). After that, nothing drives `rst`. A tiny test with Verilator:

```
module tb;
  logic a = 0, b = 0;
  logic r = a & b;
  initial begin
    #1 a = 1; b = 1;
    #1 $display("a=%b b=%b r=%b", a, b, r);
    $finish;
  end
endmodule
```

```
$ verilator --binary -Wno-fatal rinit_tb.sv --top-module tb
...
$ ./obj_dir/Vtb
a=1 b=1 r=0
```

yosys interpreted the line differently. Its log for the earlier version said

```
No latch inferred for signal `\test.\rst' from process `\test.$proc$test.v:714$454'.
```

which means it built combinational logic for `rst`, as for an `assign`. So on the board the two buttons did reset the loader, but in the simulation `rst` stayed 0 forever, whatever the testbench did with the buttons.

Pitfall. Never write `logic x = expression;` for a signal that should follow the expression. Write `logic x;` and `assign x = expression;`. With a constant, as in `logic rst = 0;` of step 07, the declaration is fine: it is a power-up value.

Exercise 9.4. Extend the testbench: after reading the memory, it presses `BTN1` and `BTN3` together for five clock cycles, sets `numLoaded` to 0 and replaces the characters in `prog` by `fedcba987654`, so that they are sent again, waits for `LED3`, and reads the memory again

with the debugger. What do you expect? Why may the reset come straight from the buttons, without a debouncer? And what would the simulation have shown with the earlier declaration of `rst`?

9.9 What Is Next

We now have everything around a processor: an I/O device with buffers, a memory of 128 KiB with byte, word, long and quad accesses, and a loader that puts a program into it. What is missing is the processor itself: registers, an ALU, and a control unit that fetches instructions from the memory, decodes and executes them. That is step 09, the ULM.

9.10 Solutions

Exercise 9.1. Offset 6: shifted left by 48 bits, the quad is `0xCDEF000000000000`, and the top 16 bits zero-extended give `0x000000000000CDEF`. Offset 5: shifted by 40 bits, `0xABCDEF0000000000`, so the result is `0xABCD`: unaligned fetches deliver the two bytes at addresses $8t+5$ and $8t+6$, as long as they are in the same quad. An unaligned word store writes nothing, since `quad_we_mask` has no case for a word at offset 5 and leaves the mask 0.

Exercise 9.2. $12 = 8 \cdot 1 + 4$: tag 1, offset 4. The mask is `16'h00FF`, so `we00` = `we01` = `4'b0000` and `we10` = `we11` = `4'b1111`. `quad_ext_low` gives `0xDEADBEEF00000000`, `quad_rshift` by 32 bits gives `0x00000000DEADBEEF`. So `spram00` and `spram01` get `0x0000` (and do not write it), `spram10` gets `0xDEAD` and `spram11` gets `0xBEEF`, both written at address 1.

Exercise 9.3. Spaces and the return character are skipped, so `3 2 1`, return, `0` stores `0x32` at address 0 and `0x10` at address 1, just like `3210`. With `123`, the loader stores `0x12` at address 0. The `3` becomes the high nibble of the next byte: `low_nibble` is set to 0, so the display shows `30`, and since `nibble_sel` is 0, `byte_valid` is 0 and `LED4` is on. Ctrl-D ends the loading anyway; the incomplete byte is never stored.

Exercise 9.4. `LED3` goes off with the reset, the loader is done again 43 cycles after the reset, and the debugger reads `fe dc ba 98 76 54` at the addresses 0 to 5: the reset cleared `done`, `nibble_sel` and the loader's `addr`, so the stores started at address 0 again. The five cycles of `BTN1` are far too short for the debouncer of the debugger, so it does not fetch. A reset needs no debouncer because it is idempotent (Chapter 5): resetting several times in a row, as a bouncing button would, has the same effect as resetting once. With the earlier declaration, `rst` would have stayed 0 in the simulation: `LED3` would stay on, no characters would be taken, and the debugger would read the old bytes `01 23 45 67 89 ab`.

Chapter 10

Step 09: The ULM on ICE

This is the last step: the ULM itself. Almost all of its parts are already there. Chapter 6 built a small ALU with a register file and a decoder, Chapter 8 the serial I/O with its FIFOs, Chapter 9 the RAM and the loader that fills it with a program typed into a terminal. What is new is the part that makes these parts a computer: a *control unit* that fetches an instruction from the RAM, decodes it, and lets the ALU, the register file, the RAM or the I/O execute it, over and over again. Everything is in `09_ulm_on_ice`; the files that are links to earlier steps are unchanged.

10.1 The Instruction Set

Every ULM instruction is a 32-bit word. Its top byte, bits 31 to 24, is the *opcode*; it says what to do. The other 24 bits are split into fields, and how they are split depends on the instruction. The file `examples/isa.txt` lists these splits as *formats*:

format	31–24	23–20	19–16	15–0	used by
`U8`	OP	*imm* (8 bits)			`halt 0x42`, `putc 'a'`
`R`	OP	x			`putc %x`, `getc %x`
`RR`	OP	y	x		`jmp %y, %x`
`REL_JMP`	OP	*offset* (24 bits, signed)			`jz`, `jmp`, …
`U20_R`	OP	*dest*	*imm* (20 bits)		`loadz imm, %dest`
`R_R_R`	OP	z	y	x (15–12)	`addq %x, %y, %z`
`U16_R_R`	OP	z	y	*imm*	`addq imm, %y, %z`
`MR_R`, `R_MR`	OP	*data*	*addr*	*offset* (signed)	`movq 8(%addr), %data`

Register numbers are 4 bits wide, for the 16 registers `%0` to `%15`. `%0` is special: it always reads as 0, and writing it has no effect. The instructions that the hardware understands are these ("imm" is a constant from the instruction, u and s mean that it is read as an unsigned or a signed number):

opcode	instruction	effect
01, 02	`halt imm`, `halt %x`	stop, with an exit code
05	`jmp offset`	$ip \leftarrow ip + 4 \cdot offset$
03, 04, 06	`jnz`, `jz`, `jb`	the same, if ZF $= 0$, ZF $= 1$, CF $= 1$
08–0D	`jbe`, `ja`, `jae`, `jge`, `jnge`, `jng`	conditional jumps on other flag combinations
07	`jmp %y, %x`	$\%x \leftarrow ip + 4$, $ip \leftarrow \%y$ (`call`, `ret`)
10, 11	`loadz imm, %dest`, `loads`	$\%dest \leftarrow u(imm)$, resp. $s(imm)$
12, 13	`addq imm, %y, %z`, `addq %x, %y, %z`	$\%z \leftarrow \%y + imm$, resp. $\%y + \%x$
14, 15	`subq` (same two forms)	$\%z \leftarrow \%y - imm$, resp. $\%y - \%x$
16, 17	`mulw` (same two forms)	product of the lowest 16 bits
18–1D	`andq`, `shrq`, `shlq` (both forms)	bitwise and, shift right, shift left
20, 21	`movzbq offset(%addr), %data`, `movsbq`	load one byte, zero- resp. sign-extended
22	`movb %data, offset(%addr)`	store one byte
23, 24	`movq` (load and store)	the same for 8 bytes
30, 31	`putc %x`, `putc imm`	send a character
32	`getc %x`	wait for a character, $\%x \leftarrow$ its code

All arithmetic is modulo 2^{64}. The flags are set by the last ALU instruction: ZF if its result was 0, SF to its bit 63, CF if an addition had a carry or a subtraction a borrow, OF if it

overflowed as a signed operation (the other ALU instructions clear CF and OF), so `subq %x, %y, %0` followed by `jb` jumps if $u(\%y) < u(\%x)$. Since a jump offset counts instructions, not bytes, the hardware multiplies it by 4.

The file `examples/isa.txt` describes exactly these instructions, in the input format of the `ulm-generator`: first the formats, then for each opcode its format, its assembler notation, and its effect as a few lines of C. From this file the `ulm-generator` builds an assembler (`ulmas`), which translates the `.s` files of `examples` into machine code (the files without extension contain the same bytes as hex digits), and a simulator of the ULM in software (`ulm`), in which a program can be tried out before it goes to the board.

10.2 The Architecture

Figure 10.1 shows how `test.sv` connects the parts. The I/O device `dev_io` and the RAM `dev_ram` have two possible users: the loader of Chapter 9 and the new control unit `dev_cu`. The switches `dev_io_switch` and `dev_ram_switch` are multiplexers that connect either the one or the other, and they are controlled by the loader's output `done`:

09_ulm_on_ice/test.sv

```
// ---------
// loader and CPU share access to RAM

if_ram ram_loader(), ram_cpu();

logic loader_done;
assign LED3 = loader_done;

dev_ram_switch dev_ram_switch0(
    .select(loader_done),
    .ram0(ram_loader.server),
    .ram1(ram_cpu.server),
    .ram(ram.client)
);

// ---------
// loader and CPU share access to IO

if_io io_loader(), io_cpu();

dev_io_switch dev_io_switch0(
    .select(loader_done),
    .io0(io_loader.server),
    .io1(io_cpu.server),
    .io(io.client)
);
```

The same signal enables the control unit (`.en(loader_done)`). So the board is in one of two modes. As long as `loader_done` is 0, the loader owns I/O and RAM, reads hex digits and writes the bytes to the RAM, starting at address 0; the control unit stands still. When the loader receives the character with code 4 (end of transmission, Ctrl-D), it sets `done`, the switches flip, and the control unit starts to execute the program at address 0. The button `BTN1` is the reset: it clears `done`, so the loader takes over again, and puts the control unit back to its start state.

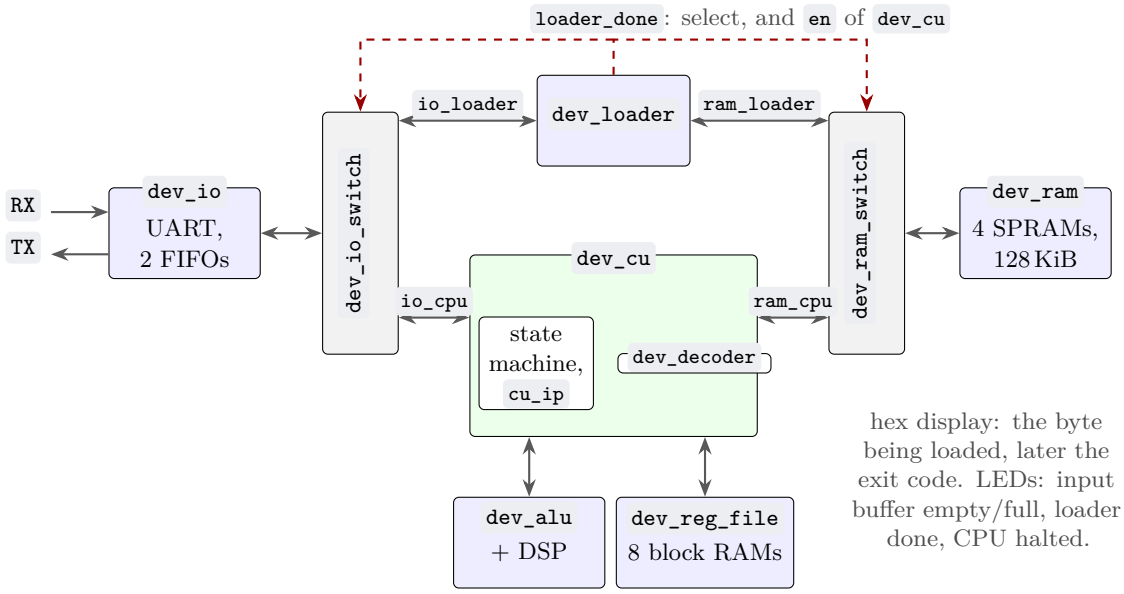


Figure 10.1: The ULM on the FPGA. The loader and the control unit share the I/O and the RAM through two switches; `loader_done` decides who is connected. The ALU and the register file are separate modules, used only by the control unit.

The arrows in Figure 10.1 are interfaces with modports, as in Chapter 8: `if_io`, `if_ram`, and two new ones for the ALU and the register file. Look at `if_reg_file.sv`: the register file has one write port (`addr_in`, `data_in`, and `op` to say whether to write) and two read ports (`addr_out0`, `data_out0` and `addr_out1`, `data_out1`), because an instruction like `addq %x, %y, %z` needs two registers at the same time. The ALU's interface `if_alu.sv` has the operation `op`, the operands `a` and `b`, the result `s` and the four flags. The packages `pkg_alu`, `pkg_bus`, `pkg_cu`, `pkg_io` and `pkg_reg` contain the `enum` types for the operations of each unit, and `pkg_reg` also the sizes:

09_ulm_on_ice/pkg_reg.sv

```
package pkg_reg;

typedef enum {
    REG_ADDRW = 4,
    REG_DEPTH = 16, // 2 ** REG_ADDRW,
    REG_WIDTH = 64
} size_t;

typedef enum {
    REG_READ_ONLY,
    REG_WRITE       // read and write
} op_t;

endpackage // pkg_reg
```

This is the trick of `pkg_ram` from Chapter 8 again: an `enum` with chosen values, used as a set of named constants.

10.3 The Register File and the ALU

The register file is an array of sixteen 64-bit words with synchronous reads:

09_ulm_on_ice/dev_reg_file.sv

```
`include "pkg_reg.sv"

module dev_reg_file (
    input logic clk,
    input if_reg_file.server reg_file
);

    (* no_rw_check *)
    logic [pkg_reg::REG_WIDTH-1:0] mem[pkg_reg::REG_DEPTH];

    initial begin
        mem[0] = 0;     // %0 is never written, so it always reads as 0
    end

    always @ (posedge clk) begin
        if (reg_file.op == pkg_reg::REG_WRITE && reg_file.addr_in != 0) begin
            mem[reg_file.addr_in] <= reg_file.data_in;
        end
        reg_file.data_out0 <= mem[reg_file.addr_out0];
        reg_file.data_out1 <= mem[reg_file.addr_out1];
    end

endmodule
```

As in Chapter 8, a memory array that is read inside a clocked block becomes block RAM. A block RAM of the iCE40 (`SB_RAM40_4K`) has one read port and one write port and is at most 16 bits wide. So yosys needs four of them side by side for 64 bits, and, since we have two read ports, a second set of four that holds a copy of the same data: both copies are written together, each is read through its own port. That makes eight of the 30 block RAMs, of which we use only 16 of 256 entries. `mem[0]` starts as 0, and writes to address 0 are suppressed, so address 0 always holds 0: that is how `%0` is always 0. The attribute `(* no_rw_check *)` is explained in Section 10.8.

The ALU computes all operations in combinational logic, with 64 bits, and selects one of them by `op`. Unlike the Mini-ALU of Chapter 6, which uses one adder for addition and subtraction (for a subtraction it inverts the second operand with $\dot{\vee}\, sub$ and adds the carry-in sub), it has a separate adder and subtractor:

09_ulm_on_ice/dev_alu.sv

```
`include "pkg_reg.sv"

module dev_alu #(
    localparam WIDTH = pkg_reg::REG_WIDTH
) (
    input logic clk,
    input if_alu.server alu
);
    logic [31:0] prod;
    logic [15:0] a0, b0;
    assign a0 = alu.a[15:0];
    assign b0 = alu.b[15:0];
```

```
assign prod = a0 * b0;

logic [WIDTH-1:0] res;
logic [WIDTH-1:0] acc = 0;

logic cf_res;
logic cf_acc = 0;

logic of_res;
logic of_acc = 0;

logic sf_acc = 0;

always_comb begin

    case (alu.op)
        pkg_alu::ALU_ADD:
            {cf_res, res} = alu.b + alu.a;
        pkg_alu::ALU_SUB:
            {cf_res, res} = alu.b - alu.a;
        pkg_alu::ALU_AND:
            {cf_res, res} = {1'b0, alu.b & alu.a};
        pkg_alu::ALU_SHR:
            {cf_res, res} = {1'b0, alu.b >> alu.a};
        pkg_alu::ALU_SHL:
            {cf_res, res} = {1'b0, alu.b << alu.a};
        pkg_alu::ALU_MULW:
            {cf_res, res} = {1'b0, {WIDTH-32{1'b0}}, prod};
        default:
            {cf_res, res} = 0;
    endcase

    case (alu.op)
        pkg_alu::ALU_ADD:
            of_res = res[WIDTH-1] && ~alu.b[WIDTH-1] && ~alu.a[WIDTH-1]
                  || ~res[WIDTH-1] && alu.b[WIDTH-1] && alu.a[WIDTH-1];
         pkg_alu::ALU_SUB:
            of_res = ~res[WIDTH-1] && alu.b[WIDTH-1] && ~alu.a[WIDTH-1]
                  || res[WIDTH-1] && ~alu.b[WIDTH-1] && alu.a[WIDTH-1];
        default: of_res = 0;
    endcase

    alu.s = alu.op == pkg_alu::ALU_NOP ? acc : res;
    //alu.cf = alu.op == pkg_alu::ALU_NOP ? cf_acc : cf_res;
    alu.cf = cf_acc;
    //alu.of = alu.op == pkg_alu::ALU_NOP ? of_acc : of_res;
    alu.of = of_acc;

    alu.zf = alu.s == 0;
    //alu.sf = alu.s[WIDTH-1];
    alu.sf = sf_acc;
end

always @ (posedge clk) begin
    if (alu.op != pkg_alu::ALU_NOP) begin
        acc <= res;
        cf_acc <= cf_res;
```

```
            of_acc <= of_res;
            sf_acc <= alu.s[WIDTH-1];
        end
    end

endmodule
```

New is the register `acc`. When `op` is not `ALU_NOP`, it takes over the result at the clock edge, and the flags `cf_acc`, `of_acc` and `sf_acc` take over theirs. When `op` is `ALU_NOP`, the output `alu.s` is `acc`. So the ALU remembers its last result, and the zero flag `alu.zf`, which is computed from `alu.s`, belongs to it. This is exactly what the conditional jumps need: they look at the flags of the last ALU instruction. The product for `mulw` uses only the lowest 16 bits of each operand. yosys maps it to one of the eight *DSP blocks* of the iCE40UP5K (`SB_MAC16`), a ready-made 16×16-bit multiplier, so the multiplication costs no logic cells at all. The netlist shows it as `dev_alu0.prod_SB_MAC16`.

10.4 The Decoder

The decoder in `dev_decoder.sv` takes the instruction word `ir` and splits it into four groups of signals, one for each unit that may be involved: `instr_alu`, `instr_bus` (RAM access), `instr_cu` (halt and jumps), and `instr_io`. Each group is an interface; this is the one for the ALU:

09_ulm_on_ice/if_instr_alu.sv

```
interface if_instr_alu;
    pkg_alu::op_t op;
    pkg_alu::sel_t a_sel;
    logic [pkg_reg::REG_ADDRW-1:0] s_reg;
    logic [pkg_reg::REG_ADDRW-1:0] b_reg;
    logic [pkg_reg::REG_WIDTH-1:0] a_imm;
    logic [pkg_reg::REG_ADDRW-1:0] a_reg;

    modport server(
        input op,
        input a_sel,
        input s_reg,
        input b_reg,
        input a_imm,
        input a_reg
    );
```

Here is how the decoder fills it:

09_ulm_on_ice/dev_decoder.sv

```
    if_instr_alu instr_alu_next();

    always_ff @ (posedge clk) begin
        if (rst) begin
            instr_alu.op <= pkg_alu::ALU_NOP;
        end
        else if (en) begin
            instr_alu.op <= instr_alu_next.op;
            instr_alu.a_sel <= instr_alu_next.a_sel;
            instr_alu.s_reg <= instr_alu_next.s_reg;
            instr_alu.b_reg <= instr_alu_next.b_reg;
            instr_alu.a_reg <= instr_alu_next.a_reg;
            instr_alu.a_imm <= instr_alu_next.a_imm;
        end
    end

    always_comb begin
        instr_alu_next.s_reg = ir[23:20];
        instr_alu_next.b_reg = ir[19:16];
        instr_alu_next.a_reg = ir[15:12];
        instr_alu_next.a_imm = {{64-16{1'b0}}, ir[15:0]};

        instr_alu_next.op = pkg_alu::ALU_NOP;
        instr_alu_next.a_sel = pkg_alu::ALU_REG;

        case (op)
            8'h10: // loadz a, %s  becomes u(a) + %0 -> u(%s)
                begin
                    instr_alu_next.op = pkg_alu::ALU_ADD;
                    instr_alu_next.a_sel = pkg_alu::ALU_IMM;
                    instr_alu_next.b_reg = 0;
                    instr_alu_next.a_reg = 0;
                    instr_alu_next.a_imm = {{64-20{1'b0}}, ir[19:0]};
                end
            8'h11: // loads a, %s  becomes s(a) + %0 -> s(%s)
                begin
                    instr_alu_next.op = pkg_alu::ALU_ADD;
                    instr_alu_next.a_sel = pkg_alu::ALU_IMM;
                    instr_alu_next.b_reg = 0;
                    instr_alu_next.a_reg = 0;
                    instr_alu_next.a_imm = {{64-20{ir[19]}}, ir[19:0]};
                end
            8'h12: // addq a, %b, %s
                begin
                    instr_alu_next.op = pkg_alu::ALU_ADD;
                    instr_alu_next.a_sel = pkg_alu::ALU_IMM;
                end
            8'h13: // addq %a, %b, %s
                begin
                    instr_alu_next.op = pkg_alu::ALU_ADD;
                end
```

Read it as a circuit. The `always_comb` block is the actual decoder: for the opcode `op = ir[31:24]`, a `case` computes the wires of `instr_alu_next`, with defaults for all opcodes that are not ALU instructions (`ALU_NOP`). The register numbers are simply cut out of the

fixed positions of the formats, `ir[23:20]` for z, `ir[19:16]` for y, `ir[15:12]` for x, whether the instruction uses them or not. The `always_ff` block is a register for the decoded instruction: when `en` is 1, it takes over the wires of `instr_alu_next`. The other three groups are built in the same way. For `loadz` the decoder plays a small trick: it turns the instruction into an addition $\%dest \leftarrow imm + \%0$, so the ALU needs no separate "load" operation.

Note how the interfaces are used here. `if_instr_alu instr_alu_next();` creates an interface inside the module, not as a port, only to have a name for a bundle of wires: `instr_alu_next.op`, `instr_alu_next.s_reg`, … In other SystemVerilog code you will often see a `struct` for this purpose:

```
typedef struct packed {
    pkg_alu::op_t op;
    logic [3:0]   s_reg, b_reg, a_reg;
    ...
} instr_alu_t;
```

A packed struct is a vector whose bits have names, so a single assignment can copy all fields at once (`instr_alu <= instr_alu_next;`). The ULM does not use structs; the interfaces do the same job, at the price of copying the fields one by one.

The conditional jumps are decided in the decoder too: for `jz`, the decoder sets `instr_cu_next.op` to `CU_REL_JMP` if `alu.zf` is 1 and to `CU_NOP` otherwise. This is why the decoder is connected to the ALU.

10.5 The Control Unit

The control unit `dev_cu` is a state machine (Chapter 7). Its states are in `pkg_cu`:

09_ulm_on_ice/pkg_cu.sv

```
typedef enum {
    CU_FETCH,
    CU_DECODE,
    CU_LOAD_OPERANDS,
    CU_EXECUTE,
    CU_INCREMENT,
    CU_HALTED
} state_t;
```

and its next-state logic is short:

09_ulm_on_ice/dev_cu.sv

```
    always_ff @ (posedge clk) begin
        if (rst) begin
            cu_state <= pkg_cu::CU_FETCH;
        end
        else if (en) begin
            cu_state <= cu_state_next;
        end
    end

    always_comb begin
        cu_state_next = cu_state;

        case (cu_state)
            pkg_cu::CU_FETCH:
                begin
                    if (instr_cu.op == pkg_cu::CU_HALT_IMM
                     || instr_cu.op == pkg_cu::CU_HALT_REG)
                    begin
                        cu_state_next = pkg_cu::CU_HALTED;
                    end
                    else begin
                        cu_state_next = pkg_cu::CU_DECODE;
                    end
                end
            pkg_cu::CU_DECODE:
                begin
                    cu_state_next = pkg_cu::CU_LOAD_OPERANDS;
                end
            pkg_cu::CU_LOAD_OPERANDS:
                begin
                    cu_state_next = pkg_cu::CU_EXECUTE;
                end
            pkg_cu::CU_EXECUTE:
                case (instr_io.op)
                    pkg_io::IO_GETC_REG:    // wait for a character
                        if (io.getc_en) begin
                            cu_state_next = pkg_cu::CU_INCREMENT;
                        end
                    pkg_io::IO_PUTC_REG, pkg_io::IO_PUTC_IMM:
                        if (io.putc_push_done) begin    // wait until sent
                            cu_state_next = pkg_cu::CU_INCREMENT;
                        end
                    default:
                        cu_state_next = pkg_cu::CU_INCREMENT;
                endcase
            pkg_cu::CU_INCREMENT:
                begin
                    cu_state_next = pkg_cu::CU_FETCH;
                end
        endcase
    end
```

Every instruction takes the same five states, FETCH, DECODE, LOAD_OPERANDS, EXECUTE, INCREMENT, one clock cycle each (Figure 10.2). There are three exceptions. In EXECUTE, a `getc` waits until the input buffer has a character (`io.getc_en`), and a `putc` waits

until `dev_io` has taken the character (`io.putc_push_done`, see below). And a `halt` is noticed only in the FETCH state after it, when the decoded `instr_cu.op` is a halt; then the machine goes to HALTED and stays there until reset. As long as `en` (that is, `loader_done`) is 0, the state register keeps its value.

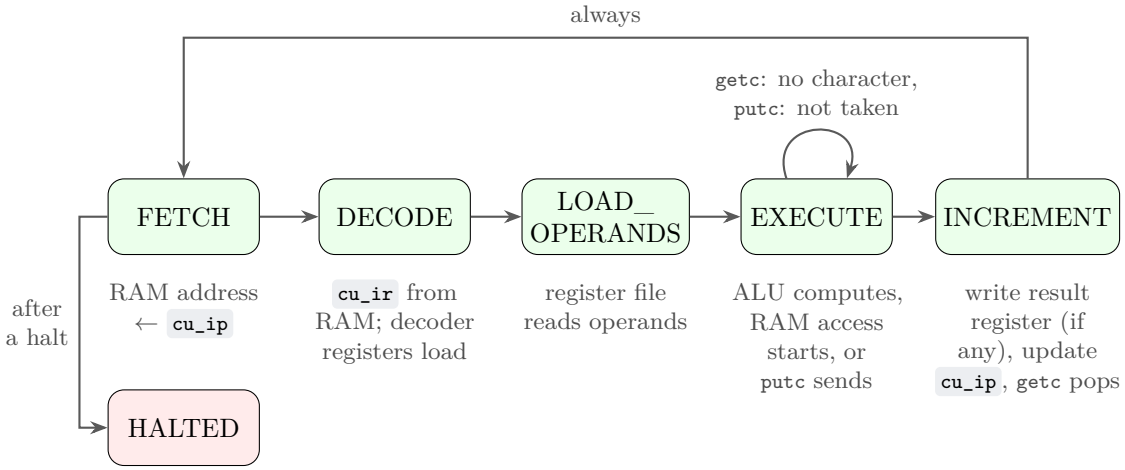


Figure 10.2: The state machine of the control unit, with what happens in each state. Each state lasts one clock cycle, except EXECUTE while `getc` or `putc` waits.

Why five cycles? Because every memory in the ULM is synchronous: the SPRAM and the block RAMs take an address at one clock edge and deliver the data after it. So the instruction, fetched with the address `cu_ip` in FETCH, is only available in DECODE; the decoded register numbers, latched at the end of DECODE, can only then be sent to the register file, whose data appear after LOAD_OPERANDS; the ALU's result is caught in `acc` at the end of EXECUTE, and written back in INCREMENT.

The rest of `dev_cu.sv` is combinational logic that says, for each state and each kind of instruction, what goes to the RAM, the register file, the ALU and the I/O. Instructions and data are in the same RAM. A second `dev_ram_switch`, inside the control unit, gives the RAM to the instruction fetch in FETCH and DECODE, and to the data access in all other states:

09_ulm_on_ice/dev_cu.sv

```
    logic ram_sel;
    if_ram ram_cu();        // used to fetch instructions
    if_ram ram_data();      // used to fetch/store data

    dev_ram_switch dev_ram_switch0(
        .select(ram_sel),
        .ram0(ram_cu.server),
        .ram1(ram_data.server),
        .ram(ram)
    );

    always_comb begin
        case (cu_state)
            pkg_cu::CU_FETCH, pkg_cu::CU_DECODE:
                ram_sel = 0;
            default:
                ram_sel = 1;
        endcase
    end

    //
    // Define how control unit accesses RAM
    //
    always_comb begin
        ram_cu.addr = cu_ip;
        ram_cu.data_type = pkg_ram::RAM_LONG;
        ram_cu.data_in = 0;
        cu_ir = ram_cu.data_out[31:0];
        ram_cu.op = pkg_ram::RAM_FETCH;
    end
```

So the very switch that separates loader and CPU is used once more, a nice example of a module as a reusable circuit. The instruction is fetched as a `RAM_LONG`, four bytes.

The register file is controlled like this:

09_ulm_on_ice/dev_cu.sv

```
    //
    // control register file operations
    //
    logic writes_reg;   // the instruction has a result register

    assign writes_reg = instr_alu.op != pkg_alu::ALU_NOP
                    || instr_io.op == pkg_io::IO_GETC_REG
                    || instr_cu.op == pkg_cu::CU_ABS_JMP
                    || instr_bus.op == pkg_bus::BUS_FETCH;

    always_comb begin
        reg_file.op = en && cu_state == pkg_cu::CU_INCREMENT && writes_reg
            ? pkg_reg::REG_WRITE
            : pkg_reg::REG_READ_ONLY;

        // defaults are for ALU instructions
        reg_file.addr_out0 = instr_alu.a_reg;
        reg_file.addr_out1 = instr_alu.b_reg;
        reg_file.addr_in = instr_alu.s_reg;
        reg_file.data_in = alu.s;

        if (instr_io.op == pkg_io::IO_PUTC_REG) begin
            reg_file.addr_out0 = instr_io.char_reg;
            reg_file.addr_out1 = 0;
            reg_file.addr_in = 0;
            reg_file.data_in = 0;
        end
        else if (instr_io.op == pkg_io::IO_GETC_REG) begin
            reg_file.addr_out0 = 0;
            reg_file.addr_out1 = 0;
            reg_file.addr_in = instr_io.char_reg;
            reg_file.data_in = {
                {pkg_ram::RAM_QUAD_SIZE - pkg_ram::RAM_BYTE_SIZE{1'b0}},
                io.getc_char
            };
        end
```

This is a multiplexer for the addresses and the write data of the register file. Its defaults are those of an ALU instruction: read registers `a_reg` and `b_reg`, write the ALU output `alu.s` to `s_reg`. The `if`-chain after it replaces them for the instructions that need something else (the listing shows two of them; further down follow `halt %x`, `jmp %y, %x`, and the loads and stores). The register file writes in the INCREMENT state, and only for an instruction that has a result register: `writes_reg` is 1 for the ALU instructions, `getc`, `jmp %y, %x` (which saves the return address in $\%x$) and the loads. This condition matters. The register numbers are cut out of the instruction word whether the instruction uses them or not, so for `putc 'A'` (`0x31410000`) `s_reg` is 4, and for `jmp fetch` (`0x05FFFFFB`) it is 15. Without `writes_reg`, these instructions would write the ALU's last result into `%4` and `%15` (Exercise 10.4).

The ALU gets its operands from the register file or, for an immediate operand, from the decoded instruction, and it operates only in EXECUTE; in all other states it gets `ALU_NOP` and keeps `acc`:

09_ulm_on_ice/dev_cu.sv

```
    //
    // control ALU operations
    //
    always_comb begin
        alu.a = instr_alu.a_sel == pkg_alu::ALU_REG
            ? reg_file.data_out0
            : instr_alu.a_imm;
        alu.b = reg_file.data_out1;

        if (cu_state == pkg_cu::CU_EXECUTE) begin
            alu.op = instr_alu.op;
        end
        else begin
            alu.op = pkg_alu::ALU_NOP;
        end
    end
```

The I/O instructions are controlled by this block:

09_ulm_on_ice/dev_cu.sv

```
    //
    // control IO instructions
    //
    always_comb begin
        io.putc_push = 0;
        io.putc_char = 0;
        io.getc_pop = 0;

        case ({cu_state, instr_io.op})
            {pkg_cu::CU_EXECUTE, pkg_io::IO_PUTC_REG}:
                begin
                    io.putc_push = 1;
                    io.putc_char
                        = reg_file.data_out0[pkg_ram::RAM_BYTE_SIZE-1:0];
                end
            {pkg_cu::CU_EXECUTE, pkg_io::IO_PUTC_IMM}:
                begin
                    io.putc_push = 1;
                    io.putc_char = instr_io.char_imm;
                end
            {pkg_cu::CU_INCREMENT, pkg_io::IO_GETC_REG}:
                begin
                    io.getc_pop = 1;
                end
            default:
                ;
        endcase
    end
```

`getc` takes its character from the input buffer in INCREMENT, after EXECUTE has waited for `io.getc_en`. `putc` offers its character in EXECUTE (`io.putc_push`), and the state machine stays in EXECUTE until `dev_io` answers with `io.putc_push_done` (Chapter 8). `dev_io` does not always take the character at once: not while the output buffer is full, and not in a cycle in which it pushes the echo of a received byte. In that case `putc_push_done` stays 0, and the

control unit simply offers the character again in the next cycle. When `dev_io` takes it, it sets `putc_push_done` for one cycle; in that cycle `putc_push` is still 1, but `dev_io` is busy with its own push and ignores it, so each character goes into the buffer exactly once. Without this handshake, a program that sends more than the 512 bytes the output buffer holds would lose characters.

The instruction pointer `cu_ip` is a 17-bit register (the RAM has 2^{17} bytes). In INCREMENT it becomes $ip + 4$, or $ip + 4 \cdot offset$ for a jump that is taken (the decoder already appended two zero bits to the offset), or the content of a register for `jmp %y, %x`. All of this is computed modulo 2^{17}, so a negative offset in two's complement simply works.

10.6 One Instruction, Cycle by Cycle

Let us follow the instruction `addq 1, %p, %p` of `01_hello.s` (below) through the hardware. Here `%p` is `%1`, the instruction is at address `0x08`, its code is `0x12110001` (opcode `12`, $z = 1$, $y = 1$, $imm = 1$), and `%1` contains `0x20`. The values in Table 10.1 are taken from a simulation of this program (Section 10.7).

state	during the cycle	at the rising edge at its end
FETCH	RAM switch to instruction fetch; RAM address `cu_ip` = `0x08`, `RAM_FETCH`	SPRAM takes over the address
DECODE	`cu_ir` = `0x12110001`; the decoder computes `ALU_ADD`, `ALU_IMM`, `s_reg` = 1, `b_reg` = 1, `a_imm` = 1; `decoder_en` = 1	`instr_alu` takes over these values
LOAD_OPS	register file: `addr_out1` = `b_reg` = 1; `alu.a` = `a_imm` = 1	`data_out1` ← %1 = `0x20`
EXECUTE	`alu.op` = `ALU_ADD`, `alu.b` = `0x20`, `alu.s` = `0x21`, ZF = 0	`acc` ← `0x21`
INCREMENT	`alu.op` = `ALU_NOP`, so `alu.s` = `acc` = `0x21`; register file: `op` = `REG_WRITE`, `addr_in` = 1, `data_in` = `0x21`; `cu_ip_next` = `0x0c`	%1 ← `0x21`, `cu_ip` ← `0x0c`, next state FETCH

Table 10.1: `addq 1, %p, %p` in five clock cycles.

The load `movzbq (%p), %ch` before it, at address `0x04`, takes the same five states but uses other paths. In LOAD_OPERANDS the register file reads `%1` on port 0 (the address register). In EXECUTE the RAM switch of the control unit has changed sides, and the data access sends the address %1 + $offset$ = `0x20` with `RAM_FETCH` and `RAM_BYTE` to the RAM. In INCREMENT the RAM delivers the byte `0x68` (an `h`), zero-extended to 64 bits by the byte logic of Chapter 9, and the register file writes it to `%2`. For `movsbq` the decoder sets an additional signal `instr_bus.sign_ext`, and the control unit replaces bits 63 to 8 of the byte by copies of its bit 7 before it goes to the register file.

At 12 MHz and five cycles per instruction the ULM executes 2.4 million instructions per second.

Exercise 10.1. (a) The word `0x05FFFFFB` is at address `0x18` of `examples/01_hello`. Which instruction is it, and where does it jump? (b) Encode `subq %3, %4, %5`. Which register gets which difference?

10.7 Running a Program

This is the first example program:

examples/01_hello.s

```
        .equ    p,      1
        .equ    ch,     p + 1

        .data
msg     .string "hello, world! 🍺 \n你好世界\nHallo, Welt\nGrüße aus Ulm ;-)\n"

        .text
        loadz   msg,    %p
fetch   movzbq  (%p),   %ch
        addq    1,      %p,     %p
        subq    0,      %ch,    %0
        jz      halt
        putc    %ch
        jmp     fetch
halt    halt    0x42
```

Line 5 puts a string of 65 bytes and a terminating zero byte into the data segment: UTF-8 encodes the beer mug emoji in four bytes and each Chinese character in three, and `\n` is the newline character. The program sets `%p` to the address of the string, loads one byte after another, and sends it with `putc`, until it finds the terminating zero byte. The comparison with zero is a subtraction whose result goes to `%0`: the result itself is thrown away, but the zero flag remains. Assembled, the program is this sequence of hex digits, `examples/01_hello`:

```
10100020202100001211000114020000040000033020000005FFFFFB0142000068656C6C6F
2C20776F726C642120F09F8DBA200AE4BDA0E5A5BDE4B896E7958C0A48616C6C6F2C205765
6C740A4772C3BCC39F652061757320556C6D203B2D290A00
```

(the file is one line; we broke it here). The first eight words are the instructions; from address `0x20` on follows the string, byte by byte. UTF-8 needs several bytes for the emoji and for the Chinese characters, and the ULM does not care: it sends bytes, and the terminal puts them together.

On the board

1. In `09_ulm_on_ice`, `make prog` writes the ULM to the board.
2. Start a serial terminal, for instance `06_io_mirror/kermit-serial` (Chapter 7; adjust the device in `set line`). It talks at 9600 baud.
3. Paste the contents of `examples/01_hello` into the terminal. Each character comes back as an echo, since `dev_io` copies every received byte to the output (Chapter 8). The loader turns pairs of hex digits into bytes and ignores everything that is not a hex digit, such as line breaks. Meanwhile the display shows the byte being assembled.
4. Press Ctrl-D. This sends the byte 4; the loader sets `done`, LED3 goes on, the display goes dark, and the ULM runs. The text appears in the terminal, then the ULM halts: LED4 goes on, and the display shows the exit code `42` of `halt 0x42`.
5. To run the next program, press `BTN1` and start again with step 3. The registers and the RAM are not cleared, the new program simply overwrites the old one from address 0 on.

The other examples read numbers with `getc` and print results: `02_uint64_hex_io` works with hexadecimal numbers, `03_uint64_dec_io` with decimal ones, and `04_factorial` computes factorials. Their source files show how the ULM calls functions with `jmp %y, %x` and a stack in the RAM.

Pitfall. A first version of `test.sv` declared the reset as `logic rst = BTN1;`. In SystemVerilog this is a declaration with an *initial value*, not an `assign`: the variable gets the value of `BTN1` once, at time 0. yosys turned it into a connection to `BTN1`, so the button worked on the board; Verilator follows the standard, so in the simulation `rst` never changed. (We checked both with a two-line module.) If a signal is meant to follow another one, write `logic rst;` and `assign rst = BTN1;`, as `test.sv` does now. The testbench below relies on it: it presses `BTN1` before each program.

In the simulation

`make sim` works as in all earlier steps. The testbench `tb_test.cpp` does not go through the serial line: at 9600 baud a byte takes $12 \cdot 10^6 \cdot 11/9600 = 13\,750$ clock cycles, far too many for a simulation. Instead it plays the role of the UART receiver (Chapter 7) itself, in the function that simulates one clock cycle:

09_ulm_on_ice/tb_test.cpp

```
// One clock cycle. Inputs set before the call are seen at the rising edge.
void
tick()
{
    dut->CLK = 1;
    dut->eval();
    ++posedge_cnt;
    if (sim_time < MAX_TRACE_TIME) {
        m_trace->dump(sim_time);
    }
    ++sim_time;

    // collect what the CPU sends
    if (dut->test->cpu_putc_done) {
        output += char(dut->test->cpu_putc_char);
    }

    // every fourth cycle deliver the next character, as if the UART
    // receiver had got it
    dut->test->io0->uart_rx0->rx_ready = 0;
    if (posedge_cnt % 4 == 0 && numLoaded < input.size()) {
        dut->test->io0->uart_rx0->rx_ready = 1;
        dut->test->io0->uart_rx0->rx_data = input[numLoaded++];
    }

    dut->CLK = 0;
    dut->eval();
    if (sim_time < MAX_TRACE_TIME) {
        m_trace->dump(sim_time);
    }
    ++sim_time;
}
```

Every fourth clock cycle it sets the outputs `rx_ready` and `rx_data` of `uart_rx0`, as if a byte had just arrived. Verilator makes this possible: the C++ class of the design has members for the submodules, and the testbench may overwrite their signals. In the other direction, the testbench reads four signals of `test.sv` that are marked with the comment `/* verilator public */`: `cpu_halted`, `cpu_exit_code`, and `cpu_putc_done` and `cpu_putc_char`, which are `putc_push_done` and `putc_char` of the CPU's I/O interface. Whenever `cpu_putc_done` is 1, `dev_io` has just taken a character from a `putc`, and the testbench appends it to `output`. This is much faster than decoding the serial signal on `TX`.

The function `run` presses `BTN1` for a few cycles, sends a program as hex digits followed by the byte 4, waits until the loader is done (`LED3`) and then until the CPU has halted, and compares the exit code and the output with the expected ones. The testbench runs four programs this way: `examples/01_hello`, which it reads from the file, and three short programs that check details of the control unit: that `putc 'A'` does not write `%4`, that the backward jump `jnz` does not write `%15`, and that `movsbq` extends the sign. Only the first program goes into `waveform.vcd`. The output, without the build steps (the lines of `01_hello` with Chinese characters and the emoji are left out here):

```
$ make sim
...

### SIMULATING ###
== examples/01_hello
loaded after 802 cycles, halted 2046 cycles later
exit code 0x42 (expected 0x42)
output (65 bytes):
...
Hallo, Welt
Grüße aus Ulm ;-)

PASSED

== putc 'A' keeps %4
loaded after 138 cycles, halted 22 cycles later
exit code 0x07 (expected 0x07)
output (1 bytes):
A
PASSED

== jnz keeps %15
loaded after 170 cycles, halted 46 cycles later
exit code 0x05 (expected 0x05)
output (0 bytes):

PASSED

== movsbq
loaded after 210 cycles, halted 31 cycles later
exit code 0xfd (expected 0xfd)
output (0 bytes):

PASSED

ALL PASSED
```

The loader is done after 802 clock cycles, and the ULM then sends the 65 bytes of the string and halts 2046 cycles later with exit code `0x42`. Table 10.1 was read off this simulation.

Exercise 10.2. Explain the 2046 cycles from the program and Figure 10.2. How long does it take the UART to send the 65 bytes at 9600 baud, with one start bit, eight data bits and two stop bits each? Where do the bytes wait in the meantime?

10.8 Resources and Speed

`make report` (Appendix A) for the whole ULM:

```
$ make report
Resource report for test in 09_ulm_on_ice

== Device utilisation (nextpnr, iCE40 UP5K) ==
   ICESTORM_LC = logic cell (4-input LUT + flip-flop)

    ICESTORM_LC:      3036/   5280   57%
    ICESTORM_RAM:       10/     30   33%
    SB_IO:              16/     39   41%
    SB_GB:               4/      8   50%
    ICESTORM_DSP:        1/      8   12%
    ICESTORM_SPRAM:      4/      4  100%

Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 15.41 MHz (PASS at 12.00 MHz)

== Cells per module (yosys) ==
   first line: the whole design; below it: the cells of each submodule
   itself, without its own submodules (indented under it)
   cells = LUTs, flip-flops, carry cells, RAMs, ... (before packing into logic cells)
   modules with interface ports are merged into their parent by sv2v

     3707 test
       65 uart_rx
       58   baud_tick_gen
       29 uart_tx
       58   baud_tick_gen
        1 dp_bram4096
       47 dev_hex
      129 quad_ext_high
      129 quad_ext_low
      209 quad_lshift
      209 quad_rshift
       53 quad_we_mask
        2 spram
```

The ULM fills 57 % of the logic cells, all four SPRAMs, ten of the block RAMs (eight for the registers, two for the FIFOs) and one DSP block. The per-module part of the report is of little help this time: almost all modules of this step have interfaces as ports, sv2v merges those into their parent, and so the ALU, the register file and the control unit all disappear into `test`. The floorplan (`make floorplan`) assigns logic cells to instances by the names of the cells, and its table gives a better picture:

```
Instance           Logic cells  Share  Blocks         Largest parts
dev_alu0                  1546  50.9%  1 DSP
ram0                       454  15.0%  4 SPRAMs       spram10 126, spram11 104, spram01 55, spram00 19
dev_reg_file0              339  11.2%  8 block RAMs   mem 339
dev_cu0                    331  10.9%                 instr_alu 156, instr_bus 34, instr_io 16, instr_cu 15
io0                        196   6.5%  2 block RAMs   uart_tx0 35, uart_rx0 32, outbuf 29, ...
ram_loader                  44   1.4%
dev_loader0                 44   1.4%
Other                       19   0.6%                 io_loader 7, dev_hex0 6, io 6
(top-level logic)           63   2.1%  16 I/O pins    route-through 13
```

The ALU is the largest part. To find out why, we synthesized `dev_alu` alone, with a small wrapper module that connects its interface to ports: 1375 LUTs, 128 carry cells, 67 flip-flops, one DSP. Without the two shift operations it needs 540 LUTs. So the two 64-bit shifters cost about 835 LUTs: a shifter that can shift by any amount from 0 to 63 is a cascade of six layers of 64 multiplexers (shift by 1, 2, 4, 8, 16, 32 or not), for each direction. The 128 carry cells are two 64-bit carry chains, one for the adder and one for the subtractor.

The register file gets 339 logic cells in the floorplan, for something that lives in block RAMs.

This is an artefact of how the floorplan assigns cells: yosys names a cell after a net it is connected to, and many LUTs of the ALU's input logic are named after the read data of the block RAMs. Synthesized alone, with a small wrapper module, the register file gives

```
=== top ===
      11 cells
       3   SB_LUT4
       8   SB_RAM40_4K
```

that is, the eight block RAMs and three LUTs for the write enable (`op` is `REG_WRITE` and `addr_in` is not 0). Two details of `dev_reg_file.sv` keep it this small:

- `%0` reads as 0 without any logic. Address 0 is never written, and `mem[0]` starts as 0 (the initial contents of a block RAM are part of the bitstream, Chapter 8), so both copies of the memory always hold 0 there. A test like `addr_out0 != 0 ? mem[...] : 0` would need a multiplexer behind the block RAM, one LUT for each of the 64 bits of each read port.
- The attribute `(* no_rw_check *)` on `mem` tells yosys not to care about a read and a write of the same address at the same clock edge (Chapter 8). Without it, yosys delays the write by one cycle (flip-flops for the 64 data bits, the address and the write enable) and adds a bypass, so that a read still sees the value the description promises. In the ULM, the register file is written only at the end of INCREMENT, and what its read ports deliver after that edge is not used: in the next instruction, the operands are read again in LOAD_OPERANDS.

Exercise 10.3 measures what each of them saves.

The clock. nextpnr reports a maximum frequency of 15.41 MHz, against the 12 MHz of the board: the design works, with about 25 % of margin. The critical path is this one (shortened):

```
Info: Critical path report for clock 'CLK$SB_IO_IN_$glb_clk' (posedge -> posedge):
Info:       type curr  total name
Info:   clk-to-q  1.39  1.39 Source dev_cu0.instr_alu.a_imm_SB_DFFESR_Q_12_D_SB_LUT4_O_LC.O
Info:    routing  2.95  4.34 Net dev_cu0.instr_alu.a_imm[20] (9,18) -> (9,14)
Info:      logic  1.23  5.57 Source dev_alu0.cf_acc_SB_DFFESR_Q_D_SB_LUT4_O_I3_SB_CARRY_CO_CI_S...
Info:    routing  1.76  7.33 Net dev_alu0.cf_acc_SB_DFFESR_Q_D_SB_LUT4_O_I3_SB_CARRY_CO_CI_SB_C...
Info:   ...
Info:    routing  4.14  61.08 Net dev_alu0.sf_acc_SB_LUT4_I3_O_SB_LUT4_I1_O_SB_LUT4_I3_O_SB_LUT...
Info:      logic  0.87  61.96 Source dev_alu0.cf_acc_SB_LUT4_I0_O_SB_LUT4_O_LC.O
Info:    routing  1.76  63.72 Net dev_alu0.cf_acc_SB_LUT4_I0_O[2] (4,19) -> (4,19)
Info:      setup  1.15  64.87 Source dev_cu0.instr_cu.op_SB_DFFESR_Q_D_SB_LUT4_O_LC.I2
Info: 19.89 ns logic, 44.98 ns routing
```

It starts at the flip-flops of the decoded instruction (`instr_alu.a_imm`), goes through the 64-bit adder of the ALU and the test whether the result is zero (`alu.zf`), and ends at the register in the decoder that decides whether a conditional jump is taken (`instr_cu.op`): 64.9 ns of the 83.3 ns that a clock cycle at 12 MHz has. As in step 01, most of the time is spent in the wires (45.0 ns), not in the logic (19.9 ns). Strictly speaking, this path is never used within one clock cycle: the ALU computes only in EXECUTE, and the decoder takes over its inputs only in DECODE, when the ALU gets `ALU_NOP` and `alu.zf` comes from the register `acc`. nextpnr does not know the state machine, so it has to assume the worst case. Such a path is called a *false path*.

Exercise 10.3. In `dev_reg_file.sv`, (1) remove `(* no_rw_check *)`; (2) instead, keep the attribute and read with `reg_file.addr_out0 != 0 ? mem[reg_file.addr_out0] : 0` (and the same for port 1). Predict what happens to the cell count of the register file and of the whole design, then check with `make report`. Does the ULM still work (`make sim`)?

Exercise 10.4. The testbench loads this program:

```
1040000712540001314100000240000 0
```

It is `loadz 7, %4`; `addq 1, %4, %5`; `putc 'A'`; `halt %4`, and it halts with exit code `07`. Suppose `dev_cu.sv` wrote the register file in every INCREMENT state, that is, without `&& writes_reg`. Which exit code would the program give? Which other instructions would change a register they do not name?

10.9 What Could Be Improved

The README of the repository calls the ULM on ICE a working prototype, written while its author met SystemVerilog and FPGAs for the first time. It does what it was built for: it runs the programs of the course. Having gone through all nine steps, you are now in a position to improve it. Some ideas, from small to large:

- **More instructions.** The instruction set has no `or`, no `not`, no arithmetic shift right, no division, and only byte and quad word loads and stores. Each new instruction is an opcode in `dev_decoder.sv`, perhaps an operation in `dev_alu.sv`, and an entry in `isa.txt`, so that the assembler knows it.
- **Fewer logic cells.** The adder and the subtractor can share one carry chain, since $b - a = b + \neg a + 1$, as in the Mini-ALU of Chapter 6. The shifters could shift by one bit per clock cycle, at the price of up to 63 cycles per shift. With more than 40 % of the logic cells free, there is room for more instructions, such as a full 64-bit multiplication from four of the DSP blocks.
- **More speed.** Five cycles per instruction is slow. The next instruction could be fetched while the current one is executed; this is *pipelining*, and every modern processor does it. It needs care: a jump changes which instruction comes next, and an instruction may need the result of the one before. And the clock could be faster: the iCE40UP5K has a PLL that can make a faster clock from the 12 MHz, but first the critical paths must become shorter, and nextpnr must be told which paths are false ones, or the design must be changed so that they disappear.

We started with two buttons and an AND gate. We end with a computer that we can describe completely, from the lookup table to the instruction set, and that runs programs we write. Every step on the way was a circuit, and so is the ULM: registers, multiplexers, adders and memory, connected by wires.

10.10 Solutions

Exercise 10.1. (a) Opcode `05` is `jmp`. The 24-bit offset `0xFFFFFB` is -5 in two's complement, so the target is $\texttt{0x18} - 5 \cdot 4 = \texttt{0x04}$, the instruction `fetch movzbq (%p), %ch`: this is the `jmp fetch` that closes the loop. In hardware, the decoder appends two zero bits, and `dev_cu` adds the lowest 17 bits to `cu_ip`: $\texttt{0x18} + (2^{17} - 20) \equiv \texttt{0x04} \pmod{2^{17}}$.

(b) The register form of `subq` has opcode `15` and format `R_R_R` with $z = 5$, $y = 4$, $x = 3$: `0x15543000`. It computes $\%5 \leftarrow \%4 - \%3$ (the ALU computes `b - a` with `b` from y and `a` from x).

Exercise 10.2. The program executes `loadz` once, then for each of the 65 bytes the six instructions `movzbq`, `addq`, `subq`, `jz` (not taken), `putc`, `jmp`, then `movzbq`, `addq`, `subq` and the taken `jz` for the zero byte, and finally `halt`. That is $1 + 6 \cdot 65 + 4 + 1 = 396$ instructions of five cycles, 1980 cycles. Each `putc` stays one cycle longer in EXECUTE: `dev_io` takes the character at the end of the first EXECUTE cycle and answers with `putc_push_done` in the second. That adds 65 cycles, and the FETCH cycle in which the halt is noticed one more: 2046. That is 0.17 ms at 12 MHz. The UART needs 11 bit times per byte, so $65 \cdot 11/9600\,\text{s} \approx 74.5\,\text{ms}$, more than 400 times as long. The bytes wait in the output FIFO of `dev_io`, which holds 512 bytes, and the ULM has halted long before the terminal shows the text.

Exercise 10.3. (1) Without `(* no_rw_check *)`, the register file alone grows from 11 to 284 cells: 135 flip-flops and 141 LUTs for the delayed write and the bypass come in addition to the block RAMs. The whole design grows from 3036 to 3228 logic cells, and its maximum frequency drops from 15.41 to 14.18 MHz. (2) With the test for address 0, the register file alone has 143 cells: the multiplexers behind the block RAMs. The whole design needs 3070 logic cells and reaches 14.98 MHz. The saving in the whole design is smaller than in the register file alone because nextpnr can put a LUT and an unrelated flip-flop into the same logic cell. In both cases `make sim` still passes all four programs: the extra logic does what it should, it is just not needed in the ULM.

Exercise 10.4. Without `writes_reg`, the defaults of the `always_comb` block would apply to `putc 'A'`: write `alu.s` to register `instr_alu.s_reg`, which the decoder always cuts out of `ir[23:20]`. `putc 'A'` is `0x31410000`, so `s_reg` is 4, and the ALU's last result, `acc` $= 7 + 1 = 8$, would be written to `%4`: exit code `08`. (An earlier version of `dev_cu.sv` had exactly this mistake.) The same would happen for `halt imm` (register $\lfloor imm/16 \rfloor$) and for the relative jumps, taken or not: `jmp fetch` in `01_hello` is `0x05FFFFFB`, which would write `acc` to `%15`, and so does every backward jump, since its offset is negative and starts with the bits `1111`. This is what the third program of the testbench checks. The examples would survive it, because their `putc` constants are `'\n'` = `0x0A`, which hits `%0`, and they do not use `%15`.

Appendix A

The make Targets

Every step directory contains the same `Makefile`, a link to `01_logic_gates/Makefile`. It expects the top module `test` in `test.sv`, passes *all* `.sv` files of the directory to the tools, and uses the testbench `tb_test.cpp` and the pin file `../icebreaker.pcf`. All files it creates start with `test`, except the simulation files. `make` without a target is `make sim`.

A.1 Overview

target	what it does	tools	creates
Simulation			
`sim`	verilates, builds and runs the simulation (the default)	Verilator, C++ compiler	`.stamp.verilate`, `obj_dir/`, `waveform.vcd`
`verilate`	only translates the design to C++	Verilator	`.stamp.verilate`, `obj_dir/`
`build`	verilates and builds, but does not run	Verilator, C++ compiler	`.stamp.verilate`, `obj_dir/`
`waves`	shows `waveform.vcd`	GTKWave	
`surfer`	shows `waveform.vcd`	Surfer	
`lint`	checks the SystemVerilog without simulating	Verilator	
Synthesis and programming			
`test.v`	translates to Verilog	sv2v	`test.v`
`test.json`	synthesizes	yosys	`test.json`, `test.log`
`test.asc`	places and routes	nextpnr	`test.asc`
`synth`	all of the above, then packs the bitstream	icepack	`test.bin`
`prog`	`synth`, then writes the flash	iceprog	
`gui-pnr`	nextpnr with its graphical interface	nextpnr	
Looking at a design			
`svg`, `png`	draws the circuit	yosys, netlistsvg, rsvg-convert	`test_svg.json`, `test.svg`, `test.png`
`report`	logic cells, Fmax, cells per module	nextpnr, yosys	`test_report.txt`, `test_report_pnr.log`, `test_report_stat.txt`
`floorplan`	where the design lies on the chip	nextpnr, Python 3	`test_pnr_dump.json`, `test_floorplan.html`
`clean`	removes all of the files above		

A.2 Simulation

`make sim` runs three stages, each printing a header (Section 2.4):

VERILATING runs

```
verilator -Wall --trace --x-assign unique --x-initial unique -Wno-fatal
          -cc *.sv --top-module test --exe tb_test.cpp
```

`-Wall` turns on all warnings, `-Wno-fatal` keeps them from stopping the build, `--trace` adds the code that writes the VCD file. The other two options make Verilator give signals without a defined value (no initial value, no reset) a random value instead of 0, so that a design that relies on a value it never set is more likely to show it. The time stamp `.stamp.verilate` records that this stage is done; it is repeated only when a `.sv` file or the testbench changes.

BUILDING SIM compiles the generated C++ code with the testbench into the program `obj_dir/Vtest`.

SIMULATING runs `obj_dir/Vtest +verilator+rand+reset+2`, which chooses those random values at the start. The option reaches Verilator only if the testbench passes its command line on with `Verilated::commandArgs(argc, argv);`, as the testbench of step 05 does; otherwise such signals start at 0. The testbench writes `waveform.vcd`.

`make verilate` stops after the first stage. `make waves` and `make surfer` open `waveform.vcd` in GTKWave or Surfer, and run the simulation first if the file is missing. `make lint` runs `verilator --lint-only`, a quick check of the syntax and of the obvious errors, without `-Wall` and without building anything. `make build` verilates and builds the program `obj_dir/Vtest` without running it.

A.3 Synthesis and Programming

The targets follow the tool flow of Figure 1.3, each file made from the previous one, and `make` rebuilds only what is older than its input:

```
sv2v -v --define=SYNTHESIS --write=test.v *.sv
yosys -ql test.log -p 'synth_ice40 -abc9 -dffe_min_ce_use 4 -device u -dsp
      -top test -json test.json' test.v
nextpnr-ice40 --pcf-allow-unconstrained --up5k --package sg48 --json test.json
      --pcf ../icebreaker.pcf --asc test.asc
icepack test.asc test.bin
iceprog test.bin
```

The options of yosys: `synth_ice40` is the complete synthesis for the iCE40 family, `-abc9` selects the newer logic optimizer, `-device u` the UltraPlus devices (our UP5K), `-dsp` allows the multipliers (DSP blocks) to be used, and `-dffe_min_ce_use 4` uses the enable input of the flip-flops only where at least four flip-flops share an enable (Section 5.3). The output goes to `test.log`; its end has the statistics we show in every chapter. nextpnr gets the device (`--up5k`, package `sg48`) and the pin file, and `--pcf-allow-unconstrained` turns a missing pin into a warning instead of an error (Section 3.5). nextpnr prints its report, with the utilisation and the maximal frequency, to the terminal.

`make synth` produces `test.bin`, `make prog` also writes it into the flash of the board.

make gui-pnr

runs nextpnr with the same options and `--gui`. A window opens with the chip and the netlist loaded, and nothing done yet. The buttons *Pack*, *Place* and *Route* in the toolbar run the three phases of nextpnr one at a time, so you can watch the logic cells appear on the chip during placement and the wires during routing, and click on a cell or a wire to see what it is. For small designs like step 01 or 02 this is the best way to see what place and route means. Build the bitstream for the board with `make synth`, not here.

A.4 Pictures of a Design

`make svg` runs yosys with `prep -top test` instead of `synth_ice40`, so the netlist keeps the gates, adders, multiplexers and flip-flops as we wrote them, and keeps submodules as boxes. netlistsvg draws this netlist as `test.svg`. `make png` converts it into `test.png`, 2000 pixels high. Figures 2.1, 3.1 and 4.1 were made this way. From step 05 on, the pictures become too large to read.

A.5 make report

`make report` prints a summary of the resources a design uses. It runs nextpnr once more, without writing a bitstream, and keeps its log in `test_report_pnr.log`. Then it runs yosys a second time with `synth_ice40 -noflatten`, which keeps the modules apart, and keeps the statistics in `test_report_stat.txt`. From both it writes `test_report.txt` and prints it. For the ULM of step 09:

```
$ make report
### PLACE AND ROUTE (for report) ###
### SYNTHESIS PER MODULE (for report) ###
Resource report for test in 09_ulm_on_ice

== Device utilisation (nextpnr, iCE40 UP5K) ==
   ICESTORM_LC = logic cell (4-input LUT + flip-flop)

    ICESTORM_LC:      3036/   5280   57%
    ICESTORM_RAM:       10/     30   33%
    SB_IO:              16/     39   41%
    SB_GB:               4/      8   50%
    ICESTORM_DSP:        1/      8   12%
    ICESTORM_SPRAM:      4/      4  100%

Max frequency for clock 'CLK$SB_IO_IN_$glb_clk': 15.41 MHz (PASS at 12.00 MHz)

== Cells per module (yosys) ==
   first line: the whole design; below it: the cells of each submodule
   itself, without its own submodules (indented under it)
   cells = LUTs, flip-flops, carry cells, RAMs, ... (before packing into logic cells)
   modules with interface ports are merged into their parent by sv2v

     3707 test
       65 uart_rx
       58   baud_tick_gen
       29 uart_tx
       58   baud_tick_gen
        1 dp_bram4096
       47 dev_hex
      129 quad_ext_high
      129 quad_ext_low
      209 quad_lshift
      209 quad_rshift
       53 quad_we_mask
        2 spram
```

The first part is the utilisation reported by nextpnr, without the resources the design does not use: logic cells, block RAMs (`ICESTORM_RAM`), I/O pins, global buffers for clocks (`SB_GB`), DSP blocks and the large single-port RAMs (`ICESTORM_SPRAM`). Then the maximal clock frequency of the final placement, and if the design has no clock, a line saying so.

The second part comes from the non-flattened synthesis. Its first line is the whole design, `test` with everything in it. Every line below counts the cells of one submodule *itself*, without its own submodules, which are indented under it: `uart_rx` has 65 cells of its own, and its `baud_tick_gen` 58 more. These are cells before packing (a LUT and a flip-flop that later share

one logic cell count as two), so the numbers are larger than the logic cells of the first part. Two more things to know. A module whose ports are interfaces (Chapter 8) does not appear: sv2v merges it into its parent, so its cells are counted with the parent, here with `test`. And each module is listed once, with the cells of one instance, even if it has several instances: `dp_bram4096` is used twice in the ULM. yosys names modules with parameters `$paramod$...`; the report strips that prefix.

A.6 make floorplan

`make floorplan` shows where the design ends up on the chip. It runs nextpnr with the option `--post-route tools/pnr_dump.py`. nextpnr has a Python interpreter built in, and after routing it runs this script, which writes every site of the chip (every logic cell, block RAM, I/O pin, …) with its position and the name of the cell placed there into `test_pnr_dump.json`. The script `tools/floorplan.py` turns this into the page `test_floorplan.html`, and on a Mac, `make` opens it in the browser. The page shows three things:

- **The chip.** Each square is a logic tile with its eight logic cells, drawn as eight thin bars, colored by the top-level instance a cell belongs to; free cells are grey. The tall boxes are block RAMs and DSPs, the small squares SPRAMs and I/O pins, dashed if unused. Hovering over a tile lists what is in it.
- **A treemap.** One rectangle per top-level instance, with an area proportional to its number of logic cells, divided into its sub-instances.
- **A table** with the same numbers, the blocks each instance uses (block RAMs, DSPs, SPRAMs, I/O pins), and its largest parts.

The seven largest instances get a color of their own, the rest is shown as "Other". Figure A.1 shows the chip for step 09.

Pitfall. The page knows only the names of the cells. yosys names a cell after the net it drives, and after flattening this name begins with the instance path, like `dev_hex0.counter[3]` in Section 4.6. The page counts a cell for the instance at the beginning of its name. This is a good approximation, but a cell at the boundary between two modules, for instance a LUT that computes an input of one module from outputs of another, may be named after the other module and be counted there. Cells whose names have no instance path, and the extra cells nextpnr adds itself, appear as "(top-level logic)". So the numbers of the page and of `make report` differ a little.

A.7 make clean

`make clean` removes everything the Makefile creates: `.stamp.*`, `obj_dir`, `waveform.vcd`, `test.v`, the files of synthesis, place and route and programming, the pictures, the report and the floorplan. The sources (`*.sv`, `tb_test.cpp`) are never touched.

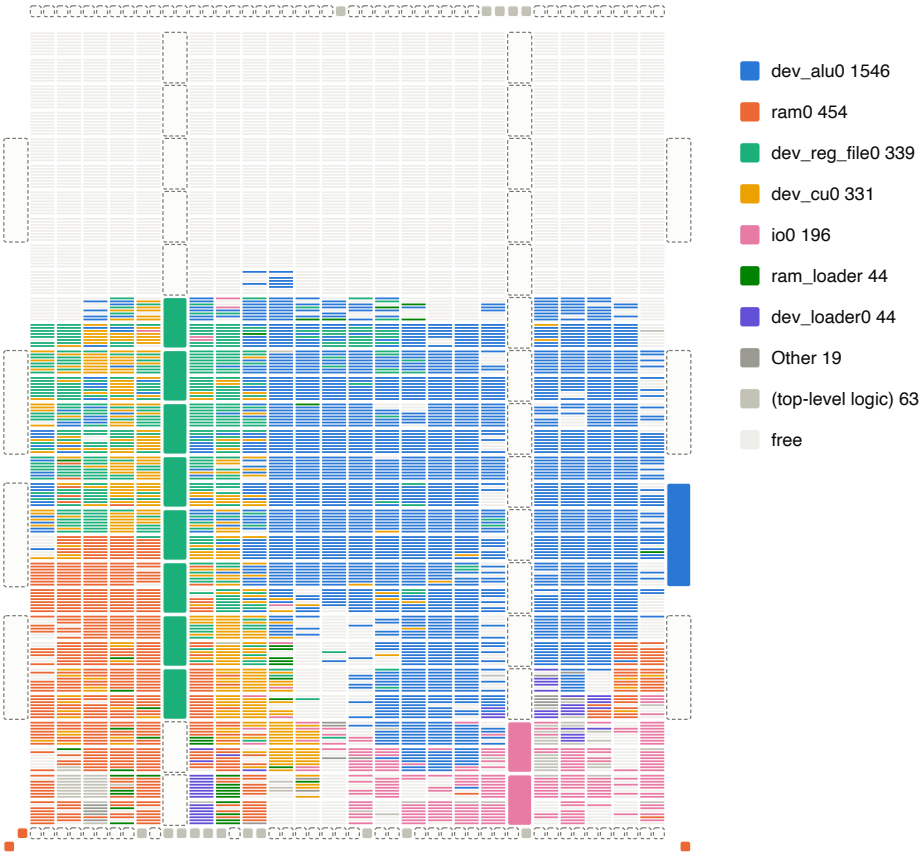


Figure A.1: The chip view of `make floorplan` for step 09, with the number of logic cells per top-level instance. The block RAMs of the register file (`dev_reg_file0`) form a column in the left half; the DSP used by the ALU is the box on the right edge.

Appendix B

Glossary

The chapter in parentheses is where a term is introduced.

`always_comb` A block describing combinational logic: its outputs follow its inputs, without memory. Assignments with `=`. (Chapter 2)

`always_ff` A block describing flip-flops, here always written as `always_ff @ (posedge clock)`: the signals assigned in it take their new values at the rising edge of the clock. Assignments with `<=`. (Chapter 3)

`assign` A permanent connection: the left side is a wire that always carries the value of the right side. (Chapter 2)

Baud rate The number of bits per second on a serial line. (Chapter 7)

Attribute A hint for the tools, attached to a declaration and written between `(*` and `*)`, for example `(* no_rw_check *)` for a memory. (Chapter 8)

Bitstream The configuration of the FPGA as a bit pattern, written by icepack into `test.bin`. (Chapter 1)

Block RAM One of the 30 memory blocks of 4 kbit of the iCE40UP5K (`SB_RAM40_4K`). yosys uses them for arrays that are read through a register. (Chapter 8)

Bouncing A mechanical button opens and closes several times within a few milliseconds before it settles. (Chapter 3)

Carry chain Dedicated wiring between neighbouring logic cells for the carry bits of an adder (`SB_CARRY`). (Chapter 4)

Clock domain All flip-flops driven by the same clock. (Chapter 6)

Clock enable An input of a flip-flop that decides whether it takes over its data input at a clock edge (`SB_DFFE`). A counter with an enable divides the clock without creating a new clock. (Chapter 4)

Combinational logic Logic without memory: its outputs are a function of its current inputs. (Chapter 2)

Concatenation `\{a, b\}`: the bits of `a` followed by those of `b`, as one vector. (Chapter 4)

Control unit The part of the processor that fetches, decodes and executes instructions, as a state machine. (Chapter 10)

Critical path The slowest path between two flip-flops (or pins). Its delay determines the highest possible clock frequency. (Chapter 2)

Debouncer A circuit that accepts a new level of a button only after it has been stable for some milliseconds, and reports each press as a single-cycle pulse. (Chapters 6 and 8)

Decoder The combinational logic that turns the bits of an instruction into the control signals of the datapath. (Chapters 6 and 10)

DSP block A hard multiplier on the chip (`SB_MAC16`); the iCE40UP5K has eight. (Chapter 10)

FIFO A first-in, first-out buffer: a memory with a write pointer and a read pointer, used as a ring. (Chapter 8)

Flip-flop A one-bit memory that takes over its input at the rising edge of its clock. Every logic cell has one. (Chapter 3)

Fmax The highest clock frequency at which the critical path still settles in time, as computed by nextpnr. (Chapter 3)

FPGA Field programmable gate array: a chip of configurable logic cells, routing, I/O pins and memory blocks. (Chapter 1)

`function` A function in SystemVerilog. In our designs it computes constants while the tools process the design. (Chapter 7)

Global network One of eight wires that reach every flip-flop of the chip with little delay, used for clocks (`SB_GB`). (Chapter 4)

`initial` In our designs: the value a flip-flop has right after the FPGA is configured. (Chapter 3)

Instantiation Using a module inside another one, like a subcircuit in CircuitVerse. Each instance is its own copy of the hardware. (Chapter 4)

`interface` A named bundle of wires, connected as a whole. (Chapter 8)

Latch A memory element that is transparent while its enable is active. yosys infers one when an `always_comb` block does not assign a signal in every case, which is almost always a mistake. (Chapters 4 and 7)

Logic cell The basic building block of the FPGA: a LUT with four inputs, a flip-flop, and carry logic. The iCE40UP5K has 5280 (`ICESTORM_LC`). (Chapter 1)

`logic` The type of a signal. Whether it becomes a wire or the output of a flip-flop follows from how it is assigned. (Chapter 2)

LUT Lookup table: a 16-to-1 multiplexer whose data inputs are configuration bits. It computes any Boolean function of four inputs. (Chapter 1)

`modport` The view of an interface from one side: which of its wires are inputs and which are outputs there. (Chapter 8)

Metastability A flip-flop whose input changes right at the clock edge may hang between 0 and 1 for a while. (Chapter 5)

Module A circuit with ports, between `module` and `endmodule`. (Chapter 2)

Netlist A list of building blocks and of the wires between them, the result of synthesis (`test.json`). (Chapter 1)

Non-blocking assignment `<=` in `always_ff`: all right-hand sides are taken from before the clock edge. (Chapter 3)

Part-select `x[3:0]` or `x[i +: 3]`: some bits of a vector. (Chapter 4)

`package` A collection of constants and types, used with `import` or `pkg::name`. (Chapter 8)

`parameter` A constant of a module that can be set differently for each instance. (Chapter 7)

Pin file `icebreaker.pcf`: the names of the pins of the chip that a top module may use as ports. (Chapter 1)

Place and route Choosing a physical block for every building block of the netlist and switches for the wires between them; done by nextpnr. (Chapter 1)

PMOD A simple connector standard for small extension modules. (Chapter 1)

Reduction `&x`, `|x`, `^x`: the AND, OR or XOR of all bits of a vector. (Chapter 8)

Register file The registers of a processor, with ports to read and to write them. (Chapters 6 and 10)

Replication `\{n\{a\}\}`: `a` repeated n times, n wires connected to the same signal. (Chapter 6)

Seven-segment display Seven LEDs arranged as an 8. Our display has two digits that share the segment wires and are switched on in turn (multiplexing). (Chapter 4)

Simulation Computing how all signals of a design change over time, here with Verilator and a testbench. (Chapter 2)

Sized literal A constant with a given number of bits, such as `8'h0f` or `1'b1`. (Chapter 4)

SPRAM One of the four single-port memory blocks of 256 kbit of the iCE40UP5K (`SB_SPRAM256KA`), together 128 KiB. (Chapter 9)

State machine A register holding a state, and combinational logic computing the next state. (Chapters 3 and 7)

Synchroniser Flip-flops that bring an asynchronous input, such as a button or the serial line, into the clock domain of the design. (Chapters 5 and 7)

Synthesis Turning a description into a netlist of the building blocks of the FPGA; done by yosys. (Chapter 1)

Testbench A program that sets the inputs of a simulated design and records its signals. (Chapter 2)

Top module The outermost module of a design. Its ports are pins of the chip. (Chapter 2)

`typedef enum` A type whose values have names, used for the states of state machines. (Chapter 7)

UART The circuit for a serial line: a start bit, eight data bits (least significant first), a stop bit. (Chapter 7)

Vector A signal of several bits, such as `logic [7:0] x`. (Chapter 4)

Waveform The values of signals over time, as recorded in `waveform.vcd` and shown by Surfer or GTKWave. (Chapter 2)